

Documentation But et Etapes

Le probleme resolu, puis le code explique etape par etape (pas ligne par ligne)
Dossier Talent-Performer-Projects-main — projets 8 a 36

Pour chaque projet, ce document explique d'abord **le but** : le probleme reel resolu et la technique demontree, en langage simple, independamment du code. Ensuite, le code de chaque fichier est decoupe en quelques etapes logiques (et non en des dizaines de lignes individuelles) : chaque etape montre l'extrait de code reel concerne et explique ce qu'il fait et pourquoi. Chaque projet est traite de maniere totalement independante des autres.

Sommaire

#	Projet
8	8-mistral_OCR
9	9-Arize
10	10-Workflow_Builder
11	11-Notion_RAG
14	14-Multi_AgentAGNO
16	16-Whatsapp
18	18-Memory_agent
20	20-Agno-Team
21	21-SDV-synthetise
22	22-Cyber_Workflow
23	23-Whatsapp-lette
24	24-ResearchAnalyst
25	25-PriceAndPositioning
26	26-PlateDetection
27	27-ImportExportSpecialist
29	29-Investment_insight
30	30-compliance_agent
31	31-Procurement-agent
32	32-Advance-Research-agent
33	33-FreightAgent
34	34-ImportExportSpecialist
35	35-HR-Assistant

#	Projet
36	36-rfp-Agent

Projet 8 — 8-mistral_OCR

Legal Copilot : OCR de document juridique + questions-reponses

LE BUT DE CE PROJET

Le but est de permettre a quelqu'un de "discuter" avec un document juridique long (un contrat, un reglement) sans le lire en entier, en posant des questions en langage naturel et en obtenant une reponse fiable et sourcee -- pas une reponse inventee.

Le probleme que ca resout est double. D'abord, un PDF juridique n'est pas toujours du texte "propre" : mise en page en colonnes, tableaux, parfois des pages scannees -- une extraction de texte classique s'en sort mal, d'ou le recours a un veritable moteur d'OCR (reconnaissance optique de caracteres) plutot qu'un simple parseur PDF. Ensuite, un contrat peut faire des dizaines de pages : envoyer tout le texte a chaque question au modele de langage serait lent, couteux, et noierait la question dans du contenu non pertinent. La solution retenue est un systeme RAG (Retrieval-Augmented Generation) : le document est decoupe en petits morceaux, chacun transforme en vecteur numerique (embedding), et pour chaque question, seuls les 2 morceaux dont le sens est le plus proche de la question sont donnees au modele -- qui doit repondre UNIQUEMENT a partir de ces extraits, jamais de sa memoire generale, ce qui limite fortement le risque d'halluciner une reponse juridiquement fausse.

8-mistral_OCR/legal_copilot.py

Classe OCRService qui centralise l'OCR, les embeddings, la recherche vectorielle et le chat, via l'API Mistral.

Etape 1 — Authentification aupres de l'API Mistral

Lignes 15-17

```
class OCRService:
    def __init__(self):
        self.client = Mistral(api_key=MISTRAL_API_KEY)
```

Des la creation d'un objet OCRService, un client Mistral authentifie est cree une seule fois et stocke sur l'instance. Toutes les methodes suivantes (OCR, embeddings, chat) reutilisent ce meme client plutot que d'en recreer un a chaque appel -- plus simple et plus efficace.

Etape 2 — Transformer le PDF en texte structure (OCR)

Lignes 19-43

```
def ocr_pdf(self, file_path):
    uploaded_pdf = self.client.files.upload(
        file={
            "file_name": file_path.split("/")[-1],
            "content": open(file_path, "rb"),
        },
        purpose="ocr"
    )

    signed_url = self.client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = self.client.ocr.process(
        model="mistral-ocr-latest",
        document={
```

```

        "type": "document_url",
        "document_url": signed_url.url,
    },
    include_image_base64=True
)

with open("ocr_response.md", "w", encoding="utf-8") as f:
    f.write("\n".join([page.markdown for page in ocr_response.pages]))

with open("ocr_meta.txt", "w") as meta:
    meta.write(file_path)

```

Cette étape envoie le PDF vers les serveurs Mistral (upload), obtient une URL temporaire sécurisée vers ce fichier, puis demande au modèle OCR de le traiter page par page. Le résultat -- du texte au format Markdown, qui préserve titres/tableaux/listes plutôt qu'un bloc de texte brut -- est sauvegardé sur disque. Le chemin du fichier source est aussi noté dans un petit fichier de cache (ocr_meta.txt), pour éviter de relancer un OCR coûteux si l'utilisateur re-upload le même document.

Etape 3 — Relire le texte OCRisé en gérant l'encodage

[Lignes 45-50](#)

```

def read_markdown_file(self, path):
    with open(path, 'rb') as f:
        raw_data = f.read()
        result = chardet.detect(raw_data)
        encoding = result['encoding']
        return raw_data.decode(encoding, errors='replace')

```

Petite fonction utilitaire qui relit un fichier texte sans supposer un encodage fixe : elle détecte automatiquement l'encodage réel du fichier avant de le decoder, ce qui évite un plantage si le fichier n'est pas en UTF-8 pur.

Etape 4 — Transformer un morceau de texte en vecteur (embedding)

[Lignes 52-57](#)

```

def get_text_embedding(self, input):
    embeddings_batch_response = self.client.embeddings.create(
        model="mistral-embed",
        inputs=input
    )
    return embeddings_batch_response.data[0].embedding

```

Cette fonction est le cœur technique du RAG : elle convertit un texte en une liste de plusieurs centaines de nombres (un "embedding") qui capture son sens. Deux textes de sens proche produiront des vecteurs proches, ce qui permettra plus loin de comparer mathématiquement une question à des passages du document.

Etape 5 — Envoyer un prompt au modèle de chat

[Lignes 59-62](#)

```

def run_mistral(self, user_message, model="mistral-large-latest"):
    messages = [{"role": "user", "content": user_message}]
    chat_response = self.client.chat.complete(model=model, messages=messages)
    return chat_response.choices[0].message.content

```

Fonction générique réutilisée pour les deux usages du modèle de langage dans ce projet : générer un résumé du document entier, ou répondre à une question à partir des passages retrouvés.

Etape 6 — Vérifier que le cache OCR correspond au bon fichier

[Lignes 64-69](#)

```

def ocr_is_valid(self, file_path):
    if not os.path.exists("ocr_meta.txt"):
        return False
    with open("ocr_meta.txt", "r") as meta:
        last_file = meta.read().strip()

```

```
return file_path == last_file
```

Avant d'afficher un resultat OCR ou de repondre a une question, l'interface verifie que le PDF actuellement uploadé est bien celui pour lequel l'OCR a été fait en dernier -- sinon, elle affiche une reponse basée sur un document périmé.

Etape 7 — Repondre a une question via RAG : decouper et vectoriser

Lignes 71-83

```
def process_question(self, question, file_path):
    text = self.read_markdown_file("ocr_response.md")
    chunk_size = 2048
    chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]

    text_embeddings = []
    for chunk in chunks:
        embedding = self.get_text_embedding(chunk)
        text_embeddings.append(embedding)
        time.sleep(2)

    text_embeddings = np.array(text_embeddings)
    question_embedding = np.array([self.get_text_embedding(question)])
```

Première moitié de la méthode centrale du fichier : le document OCRisé est découpé en morceaux de 2048 caractères, et chaque morceau est transformé en vecteur (avec une pause de 2 secondes entre chaque appel pour respecter les limites de débit de l'API). La question de l'utilisateur est elle aussi transformée en vecteur, dans le même espace numérique que les morceaux du document -- c'est ce qui va permettre de les comparer.

Etape 8 — Repondre a une question via RAG : rechercher et generer

Lignes 85-102

```
d = text_embeddings.shape[1]
index = faiss.IndexFlatL2(d)
index.add(text_embeddings)

D, I = index.search(question_embedding, k=2)
retrieved_chunk = [chunks[i] for i in I.tolist()[0]]

prompt = f"""
Context information is below.
-----
{retrieved_chunk}
-----
Given the context information and not prior knowledge, answer the query.
Query: {question}
Answer:
"""

return self.run_mistral(prompt)
```

Seconde moitié : un index FAISS (une structure spécialisée dans la recherche rapide de vecteurs proches) est construit à partir des vecteurs du document, puis interrogé pour retrouver les 2 morceaux de texte les plus proches en sens de la question. Ces 2 morceaux -- pas le document entier -- sont insérés dans un prompt qui ordonne explicitement au modèle de répondre "à partir du contexte fourni, pas de connaissances préalables", avant d'envoyer ce prompt au modèle de chat. C'est cette dernière instruction qui réduit concrètement le risque de réponse inventée.

8-mistral_OCR/app.py

Interface Streamlit qui pilote OCRService : upload, OCR, resume, questions.

Etape 1 — Upload et sauvegarde du PDF

Lignes 5-17

```
st.set_page_config(page_title="Legal Copilot", layout="wide")
st.title("Legal Copilot - PDF OCR and Legal QA")

ocr_service = OCRService()
uploaded_file = st.file_uploader("Upload a legal PDF file", type=["pdf"])
ocr_performed = False

if uploaded_file:
    os.makedirs("pdf", exist_ok=True)
    file_path = f"pdf/{uploaded_file.name}"

    with open(file_path, "wb") as f:
        f.write(uploaded_file.read())
```

Mise en place de la page et du service OCR, puis reception du fichier uploadé par l'utilisateur : Streamlit garde le fichier en memoire, donc il est d'abord écrit sur le disque local (dossier "pdf/") car l'API OCR de Mistral attend un chemin de fichier ouvrable.

Etape 2 — Lancer l'OCR au clic d'un bouton

Lignes 19-25

```
if st.button("Run OCR"):
    try:
        with st.spinner("OCR en cours..."):
            ocr_service.ocr_pdf(file_path)
            ocr_performed = True
    except Exception as e:
        st.error(f"Error during OCR: {e}")
```

L'OCR n'est déclenché qu'à la demande explicite de l'utilisateur (pas automatiquement à l'upload), car chaque appel consomme des crédits API. Un spinner informe l'utilisateur pendant le traitement, et toute erreur est capturée et affichée proprement plutôt que de faire planter la page.

Etape 3 — Afficher le texte extrait (optionnel)

Lignes 27-34

```
if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.checkbox("Show extracted OCR text"):
    try:
        text = ocr_service.read_markdown_file("ocr_response.md")
        st.markdown(text)
    except Exception:
        st.warning("Error reading OCR content.")
elif not uploaded_file:
    st.info("Upload a PDF file to enable OCR analysis.")
```

Une case à cocher, visible seulement si un OCR valide existe déjà pour ce fichier, permet à l'utilisateur de vérifier lui-même le texte brut extrait -- utile pour détecter une erreur d'OCR avant de faire confiance aux réponses générées dessus.

Etape 4 — Générer un résumé du document entier

Lignes 36-45

```
st.markdown("### Document Summary")
if uploaded_file and ocr_service.ocr_is_valid(file_path) and st.button("Generate Summary"):
    text = ocr_service.read_markdown_file("ocr_response.md")
    prompt = f"Summarize this legal document clearly and concisely:\n\n{text}"
    try:
        summary = ocr_service.run_mistral(prompt)
        st.markdown("#### Summary:")
        st.markdown(summary)
    except Exception as e:
        st.error(f"Error during summarization: {e}")
```

Contrairement aux questions ponctuelles (qui passent par le RAG pour ne cibler que 2 extraits), le resume doit couvrir tout le document : le texte complet est donc envoye d'un bloc au modele, sans etape de recherche vectorielle.

Etape 5 — Poser une question sur le document

[Lignes 47-59](#)

```
st.markdown("### Ask a question about the document content")
question = st.text_input("Your question:")

if question:
    if not os.path.exists("ocr_response.md") or not ocr_service.ocr_is_valid(file_path):
        st.error("No valid OCR data found. Please run OCR first.")
    else:
        try:
            response = ocr_service.process_question(question, file_path)
            st.markdown("### Response")
            st.markdown(response)
        except Exception as e:
            st.error(f"Error while processing: {e}")
```

Dernier bloc de l'interface : la question saisie declenche la methode RAG complete (decoupage, embeddings, recherche FAISS, generation) et affiche la reponse. Une verification prealable empeche de poser une question s'il n'existe pas encore d'OCR valide pour ce fichier.

Projet 9 — 9-Arize

Pipeline multi-agents avec auto-evaluation independante

LE BUT DE CE PROJET

Le but est de montrer comment un pipeline de plusieurs agents IA specialises, coordonnees automatiquement, peut produire un rapport d'analyse complexe (ici sur les politiques agricoles britanniques) de meilleure qualite qu'un seul agent generaliste -- et surtout, comment verifier la qualite de ce rapport de maniere independante plutot que de faire confiance aveuglement au systeme.

Le probleme que ca resout est double. D'abord, demander a un seul agent de "rechercher des donnees ET les analyser ET juger la qualite de son propre travail" melange des roles tres differents (collecte factuelle, synthese, esprit critique), avec le risque qu'il fasse tout un peu superficiellement. La solution est de decouper le travail entre 3 agents specialises, chacun avec un role et des instructions precises, coordonnees par un "chef d'equipe" (un Team) qui decide qui fait quoi et dans quel ordre. Ensuite, si c'est un agent DE la meme equipe qui evalue la qualite du rapport final, il risque d'etre indulgent envers son propre travail -- d'ou l'ajout d'un second juge totalement independant (technique LLM-as-judge, via la plateforme Arize Phoenix), qui ne voit que le rapport final et donne un avis impartial sur son impact reel.

9-Arize/policy_pipeline.py

Moteur partage : definit les 3 agents specialises, le Team qui les orchestre, et l'auto-evaluation independante.

Etape 1 — Rendre le tracing d'observabilite optionnel

Lignes 38-49

```
ARIZE_SPACE_ID = os.getenv("ARIZE_SPACE_ID")
ARIZE_API_KEY = os.getenv("ARIZE_API_KEY")
ARIZE_TRACING_ACTIF = bool(ARIZE_SPACE_ID and ARIZE_API_KEY)

if ARIZE_TRACING_ACTIF:
    tracer_provider = register(
        space_id=ARIZE_SPACE_ID,
        api_key=ARIZE_API_KEY,
        project_name="agent-politique-agricole",
    )
    AgnoInstrumentor().instrument(tracer_provider=tracer_provider)
    OpenAIInstrumentor().instrument(tracer_provider=tracer_provider)
```

Le pipeline peut envoyer des traces d'execution detaillees (quel agent a tourne, avec quel prompt, combien de temps) vers la plateforme d'observabilite Arize, pour visualiser et deboguer le comportement des agents -- mais seulement si des identifiants Arize sont configures. Sans eux, le pipeline continue de fonctionner normalement, juste sans cette couche d'observabilite optionnelle.

Etape 2 — Definir l'agent de recherche (collecte web)

Lignes 57-118

```
AgentRecherche = Agent(
    name="AgentRecherche",
    model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
```

```

tools=[DuckDuckGoTools()],
description=dedent("""
    Vous etes un assistant de recherche politique specialise dans la collecte de donnees recentes et
    credibles sur :
    - Les politiques agricoles du Royaume-Uni, les programmes de subventions, les schemas de
    developpement rural et les reglementations environnementales
    - Votre objectif est de recuperer des informations exploitables et des rapports verifies a partir
    de sources officielles.
    """),
instructions=dedent("""
    1. Decouverte des Sources : rechercher les documents et rapports les plus recentes.
    2. Filtrage des Informations : eliminer le contenu duplique ou obsolete.
    3. Extraction du Contexte Brut : extraire les faits cles, sans analyse.
    Limite-toi a 4-5 recherches ciblees maximum, puis redige la synthese avec ce que tu as trouve.
    """),
markdown=True,
debug_mode=True,
compress_tool_results=True,
tool_call_limit=6,
)

```

Premier agent de la chaine : son seul travail est de collecter des informations recentes via une recherche web (outil DuckDuckGo), sans jamais les analyser -- cette separation stricte des roles (collecter puis analyser puis juger, jamais tout a la fois) est le principe fondateur du pipeline. Deux garde-fous limitent son execution : `compress_tool_results` resume les resultats de recherche avant qu'ils ne saturent le contexte du modele, et `tool_call_limit` l'empeche de chercher indefiniment sur un sujet aussi vaste.

Etape 3 — Définir l'agent d'analyse (synthese structuree)

[Lignes 121-182](#)

```

AgentAnalyse = Agent(
    name="AgentAnalyse",
    model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
    description=dedent("""
        Vous etes un analyste senior de politique agricole qui synthetise des evaluations structurees.
        """),
    instructions=dedent("""
        1. Revision Politique : analyser les donnees brutes et identifier les principaux moteurs
        politiques.
        2. Evaluation des Resultats : quantifier l'impact sur les communautes rurales.
        3. Recommandations : proposer des plans d'action multi-echelles.
        4. Format et Clarte : Markdown clair avec des en-tetes structures.
        """),
    expected_output=dedent("""
        # Analyse de Politique Agricole du Royaume-Uni
        ## Resume Executif
        | Region | Types d'Exploitations| Enjeux Cles | Schemas de Soutien |
        ## Conclusions Cles
        ## Recommandations
        """),
    markdown=True,
)

```

Deuxieme agent : il ne recherche rien lui-meme (pas d'outil attache), il transforme les donnees brutes recues du premier agent en une analyse structuree, avec un gabarit de sortie (`expected_output`) tres precis -- donner un format-cible au modele augmente fortement la chance que sa reponse soit reellement structuree et exploitable en aval.

Etape 4 — Définir l'agent évaluateur (jugement d'impact)

[Lignes 185-223](#)

```

AgentEvalueur = Agent(
    name="AgentEvalueur",
    model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
    description=dedent("""

```

```

    Vous etes un auditeur d'impact politique evaluant si un rapport repond aux besoins des
    communautés agricoles.
    """),
    instructions=dedent("""
        1. Evaluer l'utilite globale du rapport politique.
        2. Utiliser uniquement "impactant" ou "non impactant" comme etiquette finale, en etant strict.
        3. Rediger une explication etape par etape apres l'etiquetage.
        La sortie doit commencer par un seul mot : impactant ou non impactant.
    """),
    markdown=True,
)

```

Troisième agent : il juge le rapport produit par les deux précédents, avec une contrainte de format très stricte (un seul mot en première ligne) pour que sa décision soit facilement exploitable par du code sans parsing complexe. Ce jugement fait partie de la même équipe que les agents qu'il évalue -- d'où l'ajout, plus loin, d'un second jugement totalement indépendant.

Etape 5 — Orchestrer les 3 agents avec un Team

[Lignes 232-257](#)

```

AgentGestionnaire = Team(
    name="AgentGestionnaire",
    model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),
    members=[AgentRecherche, AgentAnalyse, AgentEvalueur],
    description="Un agent orchestrateur coordonnant le flux d'évaluation complet de la politique agricole
    britannique.",
    instructions=dedent("""
        Etapes :
        1. Appeler AgentRecherche pour rassembler le contexte pertinent.
        2. Transmettre le contexte brut à AgentAnalyse pour produire une analyse structurée.
        3. Demander à AgentEvalueur d'évaluer l'impact du rapport politique.
        Ne pas générer de texte vous-même. Router chaque tâche vers l'agent correct.
    """),
    markdown=True,
    debug_mode=True,
    compress_tool_results=True,
)

```

Un Team agno est lui-même pilote par un modèle de langage (son "leader") qui décide à chaque étape à quel agent membre déléguer la suite du travail -- c'est cette couche de décision qui transforme 3 agents isolés en un véritable pipeline coordonné. Le leader utilise volontairement un modèle plus léger (20 milliards de paramètres contre 120 milliards pour les agents spécialisés) : son rôle de routage/synthèse est moins exigeant, et cela lui laisse un quota de tokens plus confortable pour relire d'un coup les rapports combinés des deux premiers agents.

Etape 6 — Executer le pipeline en suivant la progression en temps réel

[Lignes 266-287](#)

```

def run_multi_agent_pipeline(saisie_utilisateur, progress_callback=None):
    final_content = None
    for event in AgentGestionnaire.run(saisie_utilisateur, stream=True, stream_events=True):
        tool = getattr(event, "tool", None)
        if tool is not None and tool.tool_name == "delegate_task_to_member" and progress_callback:
            member_id = (tool.tool_args or {}).get("member_id", "")
            progress_callback(MEMBER_LABELS.get(member_id, f"{member_id} en cours..."))

        if getattr(event, "event", None) == "TeamRunCompleted" and getattr(event, "content", None):
            final_content = event.content

    return final_content or "(Aucune réponse générée.)"

```

En exécutant le Team en mode streaming, le code peut intercepter chaque événement de délégation ("le leader vient de choisir tel agent") pour mettre à jour une barre de progression en temps réel, plutôt que de laisser l'utilisateur devant un écran de chargement bloquant pendant plusieurs minutes. La réponse finale est isolée via un événement précis ("TeamRunCompleted"), émis uniquement par le Team lui-même, jamais par un membre

delegate.

Etape 7 — Faire juger le rapport final par un second modele independant

Lignes 311-343

```
def run_auto_evaluation(saisie_utilisateur, contenu_final):
    donnees_reponse = {
        "input": [saisie_utilisateur],
        "actual_output": [contenu_final],
        "expected_output": [AgentAnalyse.expected_output],
    }
    df_reponse = pd.DataFrame(donnees_reponse)
    df_reponse["formatted_prompt"] = df_reponse.apply(format_template, axis=1)

    return llm_classify(
        dataframe=df_reponse,
        template=MODELE_EVAL_ROUTER,
        model=OpenAIModel(api_key=GROQ_API_KEY, base_url="https://api.groq.com/openai/v1",
model=GROQ_MODEL_ID),
        rails=["not impactful", "impactful"],
        provide_explanation=True,
        concurrency=1,
    )
```

C'est l'etape cle du "LLM-as-judge" : la librairie Phoenix (llm_classify) fait juger le rapport final par un modele independant du Team qui l'a redige, en le forçant a choisir entre exactement deux etiquettes (rails). L'interet d'un juge externe est d'obtenir un avis moins susceptible d'etre biaise en faveur du propre travail de l'equipe qu'un jugement fait par un agent de la meme equipe.

9-Arize/evaluation_agent.py

Script CLI qui lance le pipeline puis l'auto-evaluation, et sauvegarde les resultats.

Etape 1 — Lancer le pipeline puis l'auto-evaluation

Lignes 19-34

```
saisie_utilisateur = "Analyser l'etat actuel et les implications futures des politiques agricoles britanniques et leur impact sur les communautes agricoles"

contenu_final = run_multi_agent_pipeline(saisie_utilisateur, progress_callback=lambda label: print(f"-> {label}"))
print("\n===== SORTIE FINALE =====")
print(contenu_final)

df_eval_router = run_auto_evaluation(saisie_utilisateur, contenu_final)

print("\n===== RESULTATS D'EVALUATION =====")
print(df_eval_router[["label", "explanation"]])
df_eval_router.to_csv("evaluation_analyse_politique.csv", index=False)
```

Point d'entree en ligne de commande : il enchaîne simplement les deux fonctions definies dans le moteur partage (pipeline puis evaluation), affiche les resultats en console, et les exporte en CSV -- toute la logique reste dans policy_pipeline.py pour rester partagee avec l'interface Streamlit.

9-Arize/app.py

Interface Streamlit qui pilote le meme moteur partage, avec barre de progression et export CSV.

Etape 1 — Lancer le pipeline avec suivi de progression

Lignes 44-67

```
if lancer:
    if not saisie_utilisateur.strip():
```

```

        st.warning("Decris un sujet a analyser avant de lancer le pipeline.")
    else:
        progress_placeholder = st.empty()
        progress_placeholder.info("Demarrage du pipeline...")
        try:
            contenu_final = run_multi_agent_pipeline(
                saisie_utilisateur,
                progress_callback=lambda label: progress_placeholder.info(label),
            )
            progress_placeholder.info("Auto-evaluation du rapport (Phoenix)...")
            df_eval = run_auto_evaluation(saisie_utilisateur, contenu_final)
            progress_placeholder.empty()
            st.session_state.resultat = {...}
        except Exception as e:
            progress_placeholder.empty()
            st.error(f"Erreur pendant le pipeline : {e}")

```

Le clic sur le bouton de lancement declenche le meme pipeline que le script CLI, mais le callback de progression met a jour un emplacement Streamlit reutilisable en temps reel, pour que l'utilisateur voie concretement quel agent travaille pendant les 3 a 8 minutes d'execution. Le resultat est stocke dans `st.session_state` pour survivre aux re-executions du script que Streamlit declenche a chaque interaction.

Etape 2 — Afficher le verdict et permettre l'export

[Lignes 76-92](#)

```

label = resultat["label"]
if label == "impactful":
    st.success(f"Evaluation automatique : **{label}**")
else:
    st.warning(f"Evaluation automatique : **{label}**")
st.markdown(f"***Justification : ** {resultat['explanation']}**")

st.download_button(
    "Exporter les resultats en CSV",
    data=resultat["df"].to_csv(index=False).encode("utf-8-sig"),
    file_name="evaluation_analyse_politique.csv",
    mime="text/csv",
)

st.markdown("### Rapport final")
st.markdown(resultat["contenu_final"])

```

Dernier bloc : le verdict du juge independant est affiche en couleur (vert si impactant, orange sinon) avec sa justification, un bouton permet de telecharger les resultats bruts en CSV, et le rapport complet genere par le Team est affiche en dessous.

Projet 10 — 10-Workflow_Builder

Workflows d'automatisation no-code (n8n) pour la gestion clients

LE BUT DE CE PROJET

Le but est de montrer qu'on peut automatiser des processus métier complets (qualifier un lead entrant, relancer un client après impayé, envoyer une enquête de satisfaction) sans écrire une ligne de backend, en assemblant visuellement des blocs préconstruits dans un outil "no-code" (n8n).

Le problème que ça résout : dans une petite entreprise, ces automatisations (scorer un lead, programmer une séquence d'emails, relancer une facture selon son retard) sont souvent faites à la main ou pas faites du tout, faute de développeur disponible pour écrire et maintenir un script. Un outil no-code comme n8n permet à quelqu'un qui ne code pas de brancher entre eux des "nœuds" (recevoir un événement, exécuter un petit bout de logique, appeler une API, attendre une date) et de faire tourner ça en continu, sans serveur à gérer soi-même. Le véritable "code" de la logique métier (comment calculer le score d'un lead, quand déclencher une relance) reste écrit en JavaScript dans des nœuds spécifiques, mais tout l'orchestrage (quel nœud déclenche quel autre) est visuel plutôt que programme.

10-Workflow_Builder/gestionclient.json

Workflow n8n complet : qualification de leads, onboarding, enquêtes de satisfaction, relance de factures.

Etape 1 — Recevoir un nouveau lead via un webhook

Lignes 4-18

```
{
  "parameters": {
    "path": "lead-qualification",
    "options": {}
  },
  "id": "webhook-lead-qualification",
  "name": "Nouveau Lead (Formulaire)",
  "type": "n8n-nodes-base.webhook",
  "typeVersion": 1,
  "position": [240, 300],
  "webhookId": "lead-qualification-webhook"
}
```

Point de départ du premier sous-flux : un "webhook" est une URL exposée par n8n que le formulaire de capture de leads appelle dès qu'un visiteur le remplit. C'est ce déclencheur qui démarre automatiquement tout le reste de la chaîne, sans que n8n ait besoin d'interroger le formulaire en boucle pour vérifier s'il y a du nouveau.

Etape 2 — Calculer un score de qualification (JavaScript embarqué)

Lignes 19-31 (jsCode reformate)

```
const data = $input.first().json;
let score = 0;
const weights = {
  company_size: { 'startup': 2, 'pme': 5, 'enterprise': 10 },
  budget: { 'low': 1, 'medium': 5, 'high': 10 },
  urgency: { 'low': 1, 'medium': 3, 'high': 7 },
  decision_maker: { 'yes': 8, 'influence': 4, 'no': 1 },
```

```

    industry_match: { 'perfect': 10, 'good': 6, 'average': 3 }
  };
Object.keys(weights).forEach(criteria => {
  if (data[criteria] && weights[criteria][data[criteria]]) {
    score += weights[criteria][data[criteria]];
  }
});
let qualification = score >= 35 ? 'hot' : score >= 20 ? 'warm' : 'cold';
let priority = score >= 40 ? 'high' : score >= 25 ? 'medium' : 'low';
return { json: { ...data, lead_score: score, qualification_status: qualification, priority } };

```

C'est ici que se trouve la vraie logique métier, sous forme de code JavaScript exécuté par un nœud "code" : chaque critère du lead (taille d'entreprise, budget, urgence...) rapporte un nombre de points selon une grille de pondération, le score total est ensuite converti en catégorie ("hot"/"warm"/"cold") et en priorité commerciale. C'est ce score qui va déterminer le chemin suivi par le lead dans le reste du workflow.

Etape 3 — Aiguiller le lead selon sa qualification

[Lignes 32-72](#)

```

{
  "parameters": {
    "conditions": {
      "conditions": [
        {
          "leftValue": "={{ $json.qualification_status }}",
          "rightValue": "hot",
          "operator": { "type": "string", "operation": "equals" },
          "leftValue": "={{ $json.qualification_status }}",
          "rightValue": "warm",
          "operator": { "type": "string", "operation": "equals" }
        ],
        "combinator": "or"
      ]
    },
    "id": "qualification-switch",
    "name": "Routage par Qualification",
    "type": "n8n-nodes-base.switch"
  }
}

```

Un nœud "switch" (l'équivalent visuel d'un if/elif) route le lead vers une branche différente du workflow selon son score : un lead "hot" ou "warm" continue vers la création dans le CRM et la notification au commercial, tandis qu'un lead "cold" s'arrête là, sans mobiliser de temps commercial humain sur un prospect peu prometteur.

Etape 4 — Créer le lead dans le CRM et attribuer un commercial

[Lignes 73-160](#)

```

{
  "parameters": {
    "method": "POST",
    "url": "={{ $vars.CRM_API_URL }}/leads",
    "authentication": "genericCredentialType",
    "bodyParameters": {
      "parameters": [
        { "name": "name", "value": "={{ $json.full_name }}" },
        { "name": "lead_score", "value": "={{ $json.lead_score }}" }
      ]
    },
    "name": "Creation Lead CRM",
    "type": "n8n-nodes-base.httpRequest"
  }
}

```

Un nœud "httpRequest" appelle l'API du CRM de l'entreprise pour y créer une fiche lead avec toutes les données calculées jusqu'ici, puis un second nœud de code JavaScript attribue automatiquement un commercial (senior ou junior, selon le secteur et la priorité) et calcule une échéance de réponse garantie (SLA) -- 2h pour un lead prioritaire, jusqu'à 72h pour un lead de faible priorité.

Etape 5 — Récupérer les nouveaux clients toutes les 6 heures

Lignes 189-262

```
{ "name": "Check Nouveaux Clients", "type": "n8n-nodes-base.cron", "parameters": { "rule": { "interval": [ { "field": "cronExpression", "value": "0 */6 * * *" } ] } } }
{ "name": "Recup Nouveaux Clients", "type": "n8n-nodes-base.httpRequest", "parameters": { "method": "GET", "url": "{ { $vars.CRM_API_URL } } /clients" } }
{ "name": "Split Clients", "type": "n8n-nodes-base.splitInBatches" }
```

Deuxieme sous-flux, independant du premier : un declencheur "cron" (planificateur temporel, ici toutes les 6 heures) reveille periodiquement le workflow pour verifier s'il existe de nouveaux clients a onboarder, recupere leur liste via l'API du CRM, puis un noeud "splitInBatches" les traite un par un plutot que tous en meme temps.

Etape 6 — Generer et programmer une sequence d'emails d'onboarding

Lignes 263-341 (jsCode reformate en partie)

```
const emailSequence = [
  { delay_hours: 0, template: 'welcome_email', subject: `Bienvenue ${client.first_name} !` },
  { delay_hours: 24, template: 'setup_guide', subject: 'Etape 1 : Configuration de votre compte' },
  { delay_hours: 72, template: 'training_invitation', subject: 'Votre formation personnalisee est prete !' },
  { delay_hours: 120, template: 'progress_check', subject: 'Comment se passe votre prise en main ?' }
];
return emailSequence.map(email => ( { json: { client_id: client.id, ...email,
  scheduled_for: new Date(Date.now() + email.delay_hours * 60 * 60 * 1000).toISOString() } } ));
```

Pour chaque nouveau client, ce noeud de code genere 4 emails echelonnees dans le temps (immmediat, +1 jour, +3 jours, +5 jours), chacun avec son propre gabarit et sa propre date d'envoi calculee. Un noeud "merge" recombine ensuite ces emails generes individuellement, avant qu'un dernier noeud httpRequest les transmette a un service d'emailing externe charge de les envoyer aux dates prevues.

Etape 7 — Envoyer des enquetes de satisfaction apres un service termine

Lignes 342-488

```
{ "name": "Surveys Satisfaction (10h)", "type": "n8n-nodes-base.cron" }
{ "name": "Services Terminees Hier", "type": "n8n-nodes-base.httpRequest" }
{ "name": "Creation Survey", "type": "n8n-nodes-base.httpRequest" }
{ "name": "Notif Survey Slack", "type": "n8n-nodes-base.slack" }
```

Troisieme sous-flux : chaque jour a 10h, le workflow recupere les prestations terminees la veille sans enquete deja envoyee, cree un questionnaire de satisfaction (note globale, professionnalisme, respect des delais, recommandation) via une plateforme d'enquetes externe, puis notifie l'equipe sur Slack -- ici via un noeud Slack natif authentifie par OAuth2, different du simple appel a une URL de webhook utilise plus haut pour la notification commerciale.

Etape 8 — Relancer progressivement les factures en retard

Lignes 489-571 (jsCode reformate en partie)

```
const daysOverdue = Math.floor((today - dueDate) / (1000 * 60 * 60 * 24));
let reminderLevel = 'none';
if (daysOverdue >= 1 && daysOverdue <= 7) { reminderLevel = 'gentle'; }
else if (daysOverdue >= 8 && daysOverdue <= 15) { reminderLevel = 'firm'; }
else if (daysOverdue >= 16 && daysOverdue <= 30) { reminderLevel = 'urgent'; escalation = true; }
else if (daysOverdue > 30) { reminderLevel = 'final'; escalation = true; accountAction = 'suspend'; }
let penalty = daysOverdue > 15 ? invoice.amount * 0.02 : 0;
```

Quatrieme et dernier sous-flux : chaque jour a 9h, les factures en retard sont recuperees, puis ce noeud de code determine un niveau de relance progressif selon le nombre de jours de retard (douce, ferme, urgente, jusqu'a la suspension du compte au-dela de 30 jours), avec une penalite financiere de 2% au-dela de 15 jours et une regle anti-spam qui empeche d'envoyer plus d'une relance par 24 heures pour la meme facture.

Projet 11 — 11-Notion_RAG

Chat sur une base de connaissances Notion d'entreprise

LE BUT DE CE PROJET

Le but est de permettre a des employes de poser, en francais ou en anglais courant, des questions a une base de connaissances d'entreprise (FAQ RH, procedures internes...) stockee dans Notion, et d'obtenir une reponse redigee a partir des entrees existantes -- plutot que de chercher manuellement dans les pages Notion.

Le probleme que ca resout : une base de connaissances Notion contient des paires question/reponse ecrites par des humains, dans un langage parfois different de la facon dont un employe formulerait sa propre question. Trouver la bonne entree par une recherche par mots-cles exacts echoue souvent. La solution ici est volontairement simple (pas de base vectorielle, pas d'embeddings) : a chaque question, le script interroge Notion en direct, compare l'ensemble des mots de la question a l'ensemble des mots de chaque entree, et ne retient que celles qui partagent au moins 60% des mots -- une approche legere, sans infrastructure a maintenir, adaptee a une petite FAQ d'entreprise qui change souvent.

11-Notion_RAG/noton_rag.py

Classe NotionRAG : agent agno avec une fonction de recherche personnalisee interrogeant l'API Notion.

Etape 1 — Initialiser le client Notion et l'agent

Lignes 12-28

```
class NotionRAG:
    def __init__(self):
        self.notion = Client(auth=os.getenv("NOTION_TOKEN"))
        self.database_id = os.getenv("NOTION_DATABASE_ID")
        self.agent = self._create_agent()

    def _create_agent(self):
        return Agent(
            model=Groq(id="openai/gpt-oss-120b", api_key=GROQ_API_KEY),
            knowledge=None,
            search_knowledge=True,
            knowledge_retriever=self._notion_retriever
        )
```

Contrairement a d'autres projets qui utilisent une base vectorielle classique (FAISS, PgVector), cet agent n'a pas de base pre-indexee : knowledge=None et search_knowledge=True avec un knowledge_retriever personnalise suffisent a agno pour appeler la methode _notion_retriever a chaque question et lui fournir les documents trouves comme contexte -- la recherche se fait en temps reel directement sur Notion, sans etape d'indexation prealable a maintenir.

Etape 2 — Interroger Notion et recuperer toutes les entrees

Lignes 30-57

```
def _notion_retriever(self, agent, query, num_documents=None, **kwargs):
    try:
        results = self.notion.databases.query(database_id=self.database_id)
        matching_documents = []
        query_words = set(query.lower().split())
```

```

for page in results["results"]:
    question_data = None
    if "Question" in page["properties"]:
        question_data = page["properties"]["Question"]["title"]
    elif "question" in page["properties"]:
        question_data = page["properties"]["question"]["title"]
    question = question_data[0]["plain_text"] if question_data and question_data else ""

    answer_data = None
    if "Response" in page["properties"]:
        answer_data = page["properties"]["Response"]["rich_text"]
    elif "response" in page["properties"]:
        answer_data = page["properties"]["response"]["rich_text"]
    elif "answer" in page["properties"]:
        answer_data = page["properties"]["answer"]["rich_text"]
    answer = answer_data[0]["plain_text"] if answer_data and answer_data else ""

```

Cette fonction est appelee automatiquement par agno des que l'agent a besoin de consulter la base de connaissances. Elle recupere d'abord toutes les pages de la base Notion, puis extrait la question et la reponse de chaque page en testant plusieurs variantes de casse du nom des colonnes ("Question"/"question", "Response"/"response"/"answer") -- une precaution necessaire car, dans Notion, les colonnes sont nommees manuellement par un humain et leur orthographe exacte peut varier.

Etape 3 — Comparer la question de l'utilisateur a chaque entree

Lignes 59-68

```

question_words = set(question.lower().split())
answer_words = set(answer.lower().split())
all_words = question_words.union(answer_words)

matches = len(query_words.intersection(all_words))
match_ratio = matches / len(query_words) if query_words else 0

if match_ratio >= 0.6: # 60% match threshold

```

Pour chaque entree Notion, le code calcule la proportion des mots de la question de l'utilisateur qui se retrouvent dans la question OU la reponse stockee. Au-dela de 60% de recouvrement, l'entree est jugee pertinente. C'est une recherche lexicale simple (par mots communs), pas une recherche semantique par embeddings : elle rate les reformulations sans mots communs, mais ne necessite aucune infrastructure vectorielle.

Etape 4 — Construire les documents pertinents avec leurs metadonnees

Lignes 88-100

```

content = f"Q: {question}\nA: {answer}"
if tags:
    content += f"\nTags: {' ', ' '.join(tags)}"

matching_documents.append({
    "content": content,
    "meta_data": {
        "source": "Company Knowledge Base",
        "department": department,
        "tags": tags,
        "match_score": match_ratio
    }
})

```

Chaque entree jugee pertinente est transformee en un document "content" (question + reponse + tags) accompagne de metadonnees, dont la source "Company Knowledge Base" -- c'est ce qui permet a l'agent de citer explicitement d'ou vient l'information dans sa reponse finale, plutot que de laisser croire qu'il "sait" la reponse par lui-meme.

Etape 5 — Trier par pertinence et gerer le cas "aucun resultat"

Lignes 102-115

```
matching_documents.sort(key=lambda x: x["meta_data"]["match_score"], reverse=True)
print(f"Found {len(matching_documents)} matching documents")

if matching_documents:
    return matching_documents[:num_documents] if num_documents else matching_documents
else:
    return [{"content": "No matching entry found in the knowledge base.",
            "meta_data": {"source": "Company Knowledge Base"}}]
```

Les documents trouves sont tries du plus au moins pertinent avant d'etre retournes a l'agent. Si aucune entree ne depasse le seuil de 60%, un document de secours explicite est retourne plutot qu'une liste vide -- ce qui permet a l'agent de repondre honnetement "je n'ai pas trouve cette information" plutot que d'halluciner une reponse.

Etape 6 — Mode chat interactif en ligne de commande

Lignes 122-132

```
def chat(self):
    print("Notion Knowledge Base Chat")
    print("Type 'quit' to exit")

    while True:
        question = input("\nYour question: ")
        if question.lower() in ["quit", "exit", "q"]:
            break

    self.ask(question)
```

Boucle simple de conversation en console : chaque question saisie est transmise a l'agent (qui appellera automatiquement `_notion_retrieve` au besoin), jusqu'a ce que l'utilisateur tape un mot-cle de sortie.

Projet 14 — 14-Multi_AgentAGNO

Equipe d'agents finance (recherche + reasoning) servie via Playground

LE BUT DE CE PROJET

Le but de ce projet est de montrer comment plusieurs agents IA specialises peuvent collaborer sous la supervision d'un agent coordinateur pour produire une analyse financiere complete, plutot que de tout demander a un seul modele generaliste. Le probleme que ca resout : un LLM seul, interroge sur "faut-il investir dans telle action", n'a ni acces aux cours de bourse en temps reel, ni la capacite de chercher des actualites recentes, et il a tendance a melanger recherche d'information et raisonnement financier dans une seule reponse peu structuree. Ici, on decoupe le probleme : un agent se specialise dans la recherche web (actualites, contexte), un autre se specialise dans les donnees financieres chiffrees (cours, ratios, recommandations d'analystes), et une "equipe" (Team) pilotee par son propre modele repartit les questions entre eux puis assemble une reponse finale coherente, avec un veritable raisonnement outille (ReasoningTools) pour justifier ses conclusions.

L'autre concept cle demontre est la memoire persistante partagee : au lieu d'oublier tout a chaque nouvelle conversation, les agents et l'equipe enregistrent en base de donnees (MySQL/Postgres) des faits durables sur l'utilisateur -- ses preferences d'investissement, sa tolerance au risque, ses recherches passees -- afin de personnaliser les reponses futures. Pourquoi ca compte : c'est le meme principe que des outils comme un conseiller financier humain qui se souvient de vos discussions precedentes, applique a un systeme multi-agents automatise, avec une interface web (Playground) permettant de tester tout cela sans construire de frontend.

14-Multi_AgentAGNO/reasoning_finance_team.py

Ce script configure deux agents IA specialises (recherche web et finance), les assemble en une equipe coordonnee avec memoire persistante en base de donnees, puis lance une interface web (Playground) pour interagir avec eux.

Etape 1 — Chargement de la configuration et verification de la cle API

Lignes 1-20

```
import os
from agno.agent import Agent
from agno.memory.v2.db.postgres import PostgresMemoryDb
from agno.memory.v2.memory import Memory
from agno.models.openai import OpenAIChat
from agno.storage.agent.postgres import PostgresAgentStorage
from agno.team.team import Team
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.reasoning import ReasoningTools
from agno.tools.yfinance import YFinanceTools
from agno.playground import Playground, serve_playground_app
from dotenv import load_dotenv

load_dotenv()
# SÉCURITÉ: Utiliser les variables d'environnement
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY") or os.getenv("OPENAI_KEY_API")
```

```

if not OPENAI_API_KEY:
    print("ERREUR: Clé OpenAI manquante dans .env")
    exit(1)

```

On importe le framework Agno (agents, memoire, stockage, outils, equipe) ainsi que Playground pour l'interface web, puis on charge le fichier .env pour recuperer la cle API OpenAI sans jamais l'ecrire en dur dans le code. Le script verifie immediatement que la cle existe et s'arrete avec un message clair si ce n'est pas le cas : c'est le principe du "fail fast", qui evite de laisser le programme planter plus loin, de facon confuse, au moment du premier appel au modele.

Etape 2 — Construction de l'URL de connexion a la base de donnees et test de connexion

Lignes 22-50

```

MYSQL_HOST = os.getenv("MYSQL_HOST", "localhost")
MYSQL_PORT = os.getenv("MYSQL_PORT", "3306")
MYSQL_USER = os.getenv("MYSQL_USER", "root")
MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "")
MYSQL_DATABASE = os.getenv("MYSQL_DATABASE", "agno_finance_db")

if MYSQL_PASSWORD:
    db_url =
f"mysql+mysqlconnector://{MYSQL_USER}:{MYSQL_PASSWORD}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}"
else:
    db_url = f"mysql+mysqlconnector://{MYSQL_USER}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}"

print(f"MySQL: {MYSQL_USER}@{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}")

def test_mysql_connection():
    try:
        from sqlalchemy import create_engine, text
        engine = create_engine(db_url)
        with engine.connect() as connection:
            connection.execute(text("SELECT 1"))
            result = connection.execute(text("SELECT DATABASE()"))
            current_db = result.fetchone()[0]
            print(f"MySQL OK - Base active: {current_db}")
        return True
    except Exception as e:
        print(f"Erreur MySQL: {e}")
        return False

if not test_mysql_connection():
    exit(1)

```

Les parametres de connexion MySQL sont lus depuis l'environnement avec des valeurs par default pratiques pour du developpement local, puis assembles en une URL de connexion SQLAlchemy. Avant de creer le moindre agent, le script effectue un veritable "health check" : il ouvre une connexion et execute une requete triviale pour confirmer que la base est joignable. Cette verification precoce evite de demarrer des agents qui echoueraient silencieusement plus tard, au moment ou ils tenteraient de sauvegarder leur memoire ou leur historique de session.

Etape 3 — Creation de la memoire partagee

Lignes 52-57

```

memory = Memory(
    model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable
    db=PostgresMemoryDb(table_name="user_memories", db_url=db_url),
    delete_memories=False,
    clear_memories=False,
)

```

Cet objet Memory est le coeur du mecanisme de personnalisation : il utilise lui-meme un modele GPT-4o pour lire les conversations et en extraire des faits durables sur l'utilisateur (preferences, contexte recurrent), qu'il stocke dans la table "user_memories". Cette instance unique est ensuite partagee entre les deux agents et l'equipe, ce qui garantit qu'ils accedent tous a la meme base de souvenirs au lieu d'avoir chacun une memoire cloisonnee.

Desactiver la suppression et l'effacement automatiques garantit que les souvenirs survivent d'une execution du script a l'autre.

Etape 4 — Definition de l'agent de recherche web

Lignes 59-78

```
web_agent = Agent(
    name="Web Search Agent",
    role="Handle web search requests and general research",
    agent_id="web_agent",
    model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable
    tools=[DuckDuckGoTools()],
    storage=PostgresAgentStorage(db_url=db_url, table_name="web_agent_sessions"),
    memory=memory,
    instructions=[
        "Search for current and relevant information on financial topics",
        "Always include sources and publication dates",
        "Focus on reputable financial news sources",
        "Provide context and background information",
        "Remember user preferences and previous searches",
    ],
    markdown=True,
    enable_agentic_memory=True,
    add_history_to_messages=True,
    add_datetime_to_instructions=True,
)
```

Ce premier agent specialise recoit l'outil DuckDuckGoTools, qui lui donne la capacite reelle de chercher de l'information a jour sur internet -- une capacite qu'un LLM seul n'a pas puisque ses connaissances s'arretent a la date de son entrainement. Son "role" textuel n'est pas decoratif : il sert au coordinateur de l'equipe, plus loin dans le fichier, a decider quand lui deleguer une tache. Les instructions cadrent son comportement (privilegier des sources fiables, citer dates et sources) et les options enable_agentic_memory / add_history_to_messages activent respectivement l'usage actif de la memoire long-terme et l'inclusion de l'historique de conversation dans le contexte envoye au modele.

Etape 5 — Definition de l'agent financier

Lignes 80-107

```
finance_agent = Agent(
    name="Finance Agent",
    role="Handle financial data requests and market analysis",
    agent_id="finance_agent",
    model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable
    tools=[YFinanceTools(
        stock_price=True,
        company_info=True,
        stock_fundamentals=True,
        key_financial_ratios=True,
        analyst_recommendations=True,
    )],
    storage=PostgresAgentStorage(db_url=db_url, table_name="finance_agent_sessions"),
    memory=memory,
    instructions=[
        "You are a financial data specialist and your goal is to generate comprehensive and accurate financial reports.",
        "Use tables to display stock prices, fundamentals (P/E, Market Cap, Revenue), and recommendations.",
        "Clearly state the company name and ticker symbol.",
        "Include key financial ratios and metrics in your analysis.",
        "Focus on delivering actionable financial insights.",
        "Delegate tasks and run tools in parallel if needed.",
        "Remember user's investment preferences and risk tolerance.",
    ],
    markdown=True,
    enable_agentic_memory=True,
    add_history_to_messages=True,
```

```

    add_datetime_to_instructions=True,
)

```

Ce deuxième agent est le pendant chiffre du premier : YFinanceTools lui donne accès à des données financières réelles (cours de bourse, fondamentaux, ratios clés, recommandations d'analystes) directement depuis Yahoo Finance. Il possède sa propre table de sessions ("finance_agent_sessions") mais partage la même instance de mémoire que l'agent web, ce qui permet à l'équipe de croiser des informations qualitatives (actualités) et quantitatives (chiffres) sur un même utilisateur. Les instructions le poussent explicitement à présenter les données sous forme de tableaux et à se souvenir de la tolérance au risque de l'utilisateur, deux exigences typiques d'un rapport financier professionnel.

Etape 6 — Assemblage de l'équipe coordinatrice

Lignes 109-135

```

reasoning_finance_team = Team(
    name="Reasoning Finance Team",
    mode="coordinate",
    team_id="reasoning_finance_team",
    model=OpenAIChat(api_key=OPENAI_API_KEY, id="gpt-4o"), # CORRECTION: Utiliser la variable
    members=[web_agent, finance_agent],
    tools=[ReasoningTools(add_instructions=True)],
    instructions=[
        "Collaborate to provide comprehensive financial and investment insights",
        "Consider both fundamental analysis and market sentiment",
        "Provide actionable investment recommendations with clear rationale",
        "Use tables and charts to display data clearly and professionally",
        "Ensure all claims are supported by data and sources",
        "Present findings in a structured format",
        "Only output the final consolidated analysis",
        "Don't use emojis in formal financial reports",
        "Build on previous team analyses and preferences stored in memory",
    ],
    storage=PostgresAgentStorage(db_url=db_url, table_name="reasoning_finance_team_sessions"),
    memory=memory,
    markdown=True,
    enable_agentic_memory=True,
    enable_agentic_context=True,
    add_datetime_to_instructions=True,
    enable_team_history=True,
    success_criteria="The team has provided a complete financial analysis with data, visualizations, risk
    assessment, and actionable recommendations.",
)

```

C'est ici que le pattern multi-agents prend tout son sens : le Team regroupe les deux agents spécialisés sous la responsabilité d'un modèle coordinateur, en mode "coordinate" -- il répartit une question complexe entre les membres pertinents, récupère leurs contributions, puis ne restitue que l'analyse finale consolidée ("Only output the final consolidated analysis"), sans exposer à l'utilisateur les échanges internes. Le ReasoningTools ajoute une couche de raisonnement explicite et trace avant la conclusion, ce qui est précieux sur une tâche où il faut pondérer l'information qualitative et données chiffrées. La mémoire et le success_criteria définissent respectivement ce dont l'équipe doit se souvenir d'une session à l'autre et ce que signifie une réponse "complète" et réussie.

Etape 7 — Exposition via l'interface web Playground

Lignes 137-142

```

app = Playground(
    app_id="multi-agent-reasoning-app-mysql",
    name="Multi Agent Reasoning App (MySQL)",
    agents=[web_agent, finance_agent],
    teams=[reasoning_finance_team],
).get_app()

```

Playground fournit clé en main une interface de chat web pour dialoguer avec les agents individuels ou avec l'équipe complète, sans avoir à développer un frontend. get_app() renvoie une application web (compatible FastAPI) prête à

etre servie, ce qui permet de tester et demontrer le systeme multi-agents de facon interactive.

Etape 8 — Diagnostic de la base et lancement du serveur

Lignes 144-166

```
def show_database_info():
    try:
        from sqlalchemy import create_engine, text
        engine = create_engine(db_url)
        print("État DB MySQL:")
        with engine.connect() as connection:
            tables = connection.execute(text("SHOW TABLES")).fetchall()
            if tables:
                for table in tables:
                    table_name = table[0]
                    count = connection.execute(text(f"SELECT COUNT(*) FROM
`{table_name}`")).fetchone()[0]
                    print(f"{table_name}: {count} enregistrements")
            else:
                print("Aucune table trouvée")
    except Exception as e:
        print(f"Impossible d'afficher la base: {e}")

if __name__ == "__main__":
    print("Démarrage Application Multi-Agents d'Analyse Financière")
    print(f"Interface: http://localhost:7777")
    print(f"Base: MySQL - {MYSQL_DATABASE}")
    show_database_info()
    serve_playground_app("__main__:app", port=7777)
```

Avant de demarrer le serveur, le script affiche un etat des lieux de la base (tables existantes et nombre d'enregistrements), ce qui aide a verifier en un coup d'oeil que la memoire et les sessions des executions precedentes sont bien presentes. Le bloc final ne s'execute que si le fichier est lance directement (garde `\_\_main\_\_`) et démarre le serveur web sur le port 7777, rendant l'equipe d'agents accessible et testable via un navigateur.

Projet 16 — 16-Whatsapp

Bot WhatsApp gestionnaire de todo-list (Todoist)

LE BUT DE CE PROJET

Le but de ce projet est de transformer WhatsApp en interface pour un assistant personnel de gestion de tâches, capable de comprendre des demandes en langage naturel ("j'ai combien de tâches demain ?", "ajoute-moi un rendez-vous demain à 15h") et de les traduire en actions concrètes dans une vraie application de todo-list (Todoist). Le problème que ça résout : la plupart des gens n'ouvrent pas une application dédiée pour noter une tâche rapide, mais ils ont presque toujours WhatsApp ouvert sous la main. En branchant un agent IA derrière le webhook WhatsApp, on évite à l'utilisateur d'apprendre une syntaxe de commande rigide -- il écrit comme il parlerait à un assistant humain, et c'est le LLM qui détermine quel outil appeler (lire les tâches, en ajouter une, calculer "demain" par rapport à la date du jour) et comment formuler la réponse.

Le concept technique central démontre ici est l'intégration d'un webhook de messagerie (WhatsApp Business API via Meta/Facebook) avec un agent LLM outille : un serveur web reçoit les messages entrants, les valide, les transmet à un agent qui possède des fonctions Python comme "outils" (récupérer la date du jour, lister les tâches, en créer une), puis renvoie la réponse générée vers l'utilisateur via l'API WhatsApp. Pourquoi ça compte : c'est le schéma général de tout chatbot d'assistance connecté à une messagerie grand public -- comprendre ce flux (vérification du webhook, validation du message, traitement en arrière-plan, appel à l'agent, envoi de la réponse) est transférable à n'importe quel autre cas d'usage (support client, prise de rendez-vous, etc.) sur WhatsApp, Telegram ou autre.

16-Whatsapp/AI_TODO_List/main.py

Ce fichier est le serveur web FastAPI qui expose le webhook WhatsApp : il vérifie l'abonnement au webhook demandé par Meta, reçoit les messages entrants et lance leur traitement en arrière-plan, et fournit des routes de test pour envoyer des messages manuellement.

Etape 1 — Configuration et initialisation de l'application FastAPI

Lignes 1-24

```
import os
import json
import uvicorn
import logging
import requests
from threading import Thread
from dotenv import load_dotenv
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import PlainTextResponse, JSONResponse
from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message

load_dotenv()

VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')
WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')
```

```

VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to the portal!"}

```

On met en place le serveur web (FastAPI) et on charge depuis le .env tous les identifiants nécessaires pour parler à l'API WhatsApp de Meta : l'identifiant de version d'API, le numéro de téléphone associé au compte WhatsApp Business, le jeton d'accès, et un jeton de vérification propre au webhook. On importe aussi les fonctions de traitement des messages depuis whatsapp_utils, ce qui sépare la logique réseau (ce fichier) de la logique métier de traitement des messages. La route racine sert juste de point de contrôle simple ("l'app est en ligne").

Etape 2 — Vérification du webhook exigée par WhatsApp

[Lignes 26-42](#)

```

# Required webhook verification for WhatsApp
async def verify(request: Request):
    params = dict(request.query_params)
    mode = params.get("hub.mode")
    token = params.get("hub.verify_token")
    challenge = params.get("hub.challenge")

    if mode and token:
        if mode == "subscribe" and token == VERIFY_TOKEN:
            logging.info("WEBHOOK_VERIFIED")
            return PlainTextResponse(content=challenge, status_code=200)
        else:
            logging.warning("VERIFICATION_FAILED")
            raise HTTPException(status_code=403, detail="Verification failed")
    else:
        logging.warning("MISSING_PARAMETER")
        raise HTTPException(status_code=400, detail="Missing parameters")

```

Avant d'accepter d'envoyer des messages à un serveur, Meta exige de "prouver" que ce serveur est bien le propriétaire légitime du webhook : Meta envoie une requête GET avec un jeton secret (VERIFY_TOKEN) et un "challenge" à renvoyer tel quel. Cette fonction compare le jeton reçu à celui configuré côté serveur et, s'ils correspondent, renvoie le challenge pour valider l'abonnement ; sinon elle rejette la requête avec une erreur 403. C'est une étape obligatoire, à faire une seule fois, avant que WhatsApp n'accepte de transmettre de vrais messages entrants.

Etape 3 — Réception et traitement asynchrone des messages entrants

[Lignes 44-68](#)

```

async def handle_message(request: Request):
    try:
        body = await request.json()
    except json.JSONDecodeError:
        logging.error("Failed to decode JSON")
        return JSONResponse(status_code=400, content={"status": "error", "message": "Invalid JSON provided"})

    if (
        body.get("entry", [{}])[0]
        .get("changes", [{}])[0]
        .get("value", {})
        .get("statuses")
    ):
        logging.info("Received a WhatsApp status update.")
        return JSONResponse(content={"status": "ok"})

    if is_valid_whatsapp_message(body):
        try:

```

```

        # Run message processing in a background thread
        Thread(target=process_whatsapp_message, args=(body,)).start()
    except Exception as e:
        logging.error(f"Error starting background message thread: {e}")
        return JsonResponse(content={"status": "ok"})
    else:
        return JsonResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})

```

Cette fonction est le cœur du webhook : elle reçoit chaque événement envoyé par WhatsApp. Elle filtre d'abord les événements qui ne sont pas de vrais messages (par exemple des accusés de statut "livre/lu"), puis valide la structure du message via `is_valid_whatsapp_message` avant de le traiter. Le traitement réel (appel à l'agent IA, qui peut prendre plusieurs secondes) est lancé dans un Thread séparé plutôt qu'attendu directement : c'est essentiel car WhatsApp attend une réponse HTTP rapide au webhook et coupe la connexion si elle tarde trop -- on répond donc immédiatement "ok" pendant que le traitement se poursuit en arrière-plan.

Etape 4 — Routes de test pour l'envoi manuel de messages

[Lignes 72-109](#)

```

@app.get("/test")
def test():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'template',
        'template': {
            'name': 'hello_world',
            'language': {
                'code': 'en_US'
            }
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Test successful!", "response": response.json()}

@app.get("/send_message")
def send_message():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'text',
        'text': {
            'body': 'Hello, how are you?'
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Message sent successfully!", "response": response.json()}

```

Ces deux routes ne servent pas au fonctionnement normal du bot, mais à vérifier manuellement que la connexion sortante vers l'API WhatsApp fonctionne : `/test` envoie le message-modèle "hello_world" (obligatoire pour initier une conversation WhatsApp Business avec un nouveau contact), et `/send_message` envoie un texte libre simple. Elles sont utiles en phase de mise au point pour isoler un problème de configuration (jeton, numéro de téléphone) indépendamment de la logique de l'agent IA.

Etape 5 — Branchement des routes GET/POST du webhook et lancement du serveur

Lignes 111-122

```
@app.get("/webhook")
async def webhook_get(request: Request):
    print("GET webhook called!")
    return await verify(request)

@app.post("/webhook")
async def webhook_post(request: Request):
    print("POST webhook called!")
    return await handle_message(request)

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

On expose la meme URL /webhook sous deux methodes HTTP differentes, conformément au protocole attendu par Meta : la requete GET sert a la verification initiale de l'abonnement (verify), la requete POST sert a recevoir chaque nouveau message (handle_message). Le bloc final lance le serveur Uvicorn sur toutes les interfaces reseau (0.0.0.0), necessaire pour qu'un service externe comme WhatsApp puisse atteindre ce serveur.

16-Whatsapp/AI_TODO_List/agent_service.py

Ce fichier definit l'agent IA et ses outils : il expose des fonctions pour lire/creer des taches Todoist et calculer des dates relatives ("aujourd'hui", "demain"), puis assemble un agent Agno capable de les utiliser pour repondre en langage naturel.

Etape 1 — Configuration et acces a l'API Todoist

Lignes 1-22

```
import os
import requests
from datetime import datetime, timedelta
from textwrap import dedent
from agno.agent import Agent
from dotenv import load_dotenv
from agno.models.openai import OpenAIChat

load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
TODOIST_API_KEY = os.getenv("TODOIST_API_KEY")
HEADERS = {
    "Authorization": f"Bearer {TODOIST_API_KEY}",
    "Content-Type": "application/json"
}

def get_tasks():
    response = requests.get("https://api.todoist.com/rest/v2/tasks", headers=HEADERS)
    if response.ok:
        return response.json()
    return []
```

On charge les cles API (OpenAI pour le LLM, Todoist pour la vraie application de taches) et on prepare l'en-tete d'authentification Todoist une seule fois, reutilise par toutes les fonctions suivantes. get_tasks() est la premiere fonction-outil : un simple appel GET a l'API REST de Todoist qui recupere toutes les taches existantes, avec un repli sur une liste vide en cas d'echec plutot que de lever une exception qui casserait le flux de l'agent.

Etape 2 — Fonction pour creer une tache

Lignes 24-29

```
def add_task(content, due_datetime=None):
    payload = {"content": content}
    if due_datetime:
```

```

    payload["due_datetime"] = due_datetime
    response = requests.post("https://api.todoist.com/rest/v2/tasks", headers=HEADERS, json=payload)
    return response.ok

```

Cette fonction est l'outil qui permet effectivement à l'agent d'agir, pas seulement de lire : elle envoie une requête POST à Todoist pour créer une nouvelle tâche, avec une date/heure d'échéance optionnelle. C'est cette fonction que le LLM appellera quand l'utilisateur demandera d'ajouter quelque chose à faire, en générant lui-même le contenu et la date à partir du message en langage naturel.

Etape 3 — Calcul de la date/heure actuelle via une API externe

[Lignes 31-38](#)

```

def get_current_datetime():
    url = 'https://timeapi.io/api/Time/current/zone?timeZone=Africa/Casablanca'
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        return data["dateTime"]
    else:
        return None

```

Un LLM ne connaît pas la date et l'heure actuelles avec certitude (son entraînement s'arrête à une date fixe, et il n'a pas d'horloge interne fiable) : cette fonction résout ce problème en interrogeant une API de temps réel, fixée sur le fuseau horaire Africa/Casablanca pertinent pour l'utilisateur. C'est la fonction de base sur laquelle s'appuient toutes les autres fonctions de date ci-dessous.

Etape 4 — Fonctions d'aide pour "aujourd'hui" et "demain"

[Lignes 40-71](#)

```

# NOUVELLES FONCTIONS D'AIDE POUR L'AGENT
def get_tasks_for_date(date_str):
    """Récupère les tâches pour une date spécifique (format: 2025-08-30)"""
    all_tasks = get_tasks()
    tasks_for_date = []

    for task in all_tasks:
        if task.get("due") and task["due"].get("date") == date_str:
            tasks_for_date.append({
                "content": task["content"],
                "time": task["due"].get("datetime", "Pas d'heure spécifique")
            })

    return tasks_for_date

def get_today_date():
    """Retourne la date d'aujourd'hui au format 2025-08-29"""
    current_dt = get_current_datetime()
    if current_dt:
        # Extraire juste la date (YYYY-MM-DD)
        return current_dt.split("T")[0]
    return None

def get_tomorrow_date():
    """Retourne la date de demain au format 2025-08-30"""
    today = get_today_date()
    if today:
        # Convertir en date et ajouter 1 jour
        today_dt = datetime.strptime(today, "%Y-%m-%d")
        tomorrow_dt = today_dt + timedelta(days=1)
        return tomorrow_dt.strftime("%Y-%m-%d")
    return None

```

Ces trois fonctions transforment la date brute renvoyée par l'API en concepts directement utiles pour un assistant conversationnel : filtrer les tâches d'un jour précis, obtenir "aujourd'hui" sous forme simple (YYYY-MM-DD), et calculer "demain" en ajoutant un jour avec le module datetime. C'est un découpage délibérément fait pour que le

LLM n'ait jamais à calculer une date lui-même (ce qu'il fait mal de façon fiable) : il se contente d'appeler la bonne fonction-util, qui fait le calcul exact en code Python classique.

Etape 5 — Definition de l'agent et de ses instructions

Lignes 73-112

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o", api_key=OPENAI_API_KEY),
    instructions = dedent("""
    Tu es un assistant personnel qui gère les tâches via WhatsApp.

    TES OUTILS DISPONIBLES:
    - get_current_datetime() : heure actuelle complète
    - get_today_date() : date d'aujourd'hui (format: 2025-08-29)
    - get_tomorrow_date() : date de demain (format: 2025-08-30)
    - get_tasks() : toutes les tâches
    - get_tasks_for_date(date) : tâches pour une date précise
    - add_task(content, due_datetime) : ajouter une tâche

    COMMENT RÉPONDRE:

    Pour "J'ai combien de tâches demain?":
    1. Appeler get_tomorrow_date()
    2. Appeler get_tasks_for_date(date_de_demain)
    3. Compter et lister les tâches

    Pour "Mes tâches aujourd'hui":
    1. Appeler get_today_date()
    2. Appeler get_tasks_for_date(date_aujourd'hui)
    3. Lister les tâches

    Pour ajouter une tâche demain à 15h:
    1. Appeler get_tomorrow_date() "2025-08-30"
    2. Créer datetime: "2025-08-30T15:00:00Z"
    3. Appeler add_task("contenu", "2025-08-30T15:00:00Z")

    EXEMPLES DE RÉPONSES:
    - " Vous avez 2 tâches demain : Courses (pas d'heure), Appel (15h)"
    - " Tâche ajoutée pour demain !"
    - " Aucune tâche prévue demain"

    Sois simple, direct et utilise des emojis appropriés.
    """),
    markdown=True,
    tools=[get_current_datetime, get_today_date, get_tomorrow_date, get_tasks, get_tasks_for_date,
    add_task]
)
```

C'est ici que tout s'assemble : l'agent Agno reçoit un modèle GPT-4o, un long prompt d'instructions qui explique explicitement quelle séquence d'appels d'outils suivre pour chaque type de demande fréquente (combien de tâches demain, tâches du jour, ajout avec heure précise), et surtout la liste des six fonctions Python définies plus haut passées telles quelles comme "tools". Cette technique de prompt très directif (avec des scénarios d'exemple pas-à-pas) compense le fait que le modèle n'a aucune connaissance a priori de Todoist ou du fuseau horaire visé ; elle guide le raisonnement du LLM pour qu'il enchaîne correctement plusieurs appels d'outils avant de répondre, plutôt que de deviner.

Etape 6 — Fonction d'entrée publique et test local

Lignes 114-119

```
def get_response(message):
    response = agent.run(message)
    return response.content

if __name__ == "__main__":
    print(get_response("J'ai combien de tâches demain?"))
```

`get_response` est le point d'entree simple utilise par le reste de l'application (notamment `whatsapp_utils.py`) : elle envoie le message utilisateur brut a l'agent et renvoie uniquement le texte de la reponse generee, en cachant tous les details d'appels d'outils internes. Le bloc de test permet de verifier rapidement, en local, que l'agent repond correctement sans passer par tout le circuit WhatsApp.

16-Whatsapp/AI_TODO_List/whatsapp_utils.py

Ce fichier fait le lien entre le format de message propre a l'API WhatsApp et l'agent IA : il valide la structure des messages entrants, extrait le texte utile, appelle l'agent, puis reconstruit et envoie la reponse au bon format WhatsApp.

Etape 1 — Configuration et fonctions utilitaires de base

Lignes 1-34

```
import requests
import json
import logging
import re
from fastapi.responses import JSONResponse
from dotenv import load_dotenv
from agent_service import get_response
import os

load_dotenv()

WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

def log_http_response(response):
    logging.info(f"Status: {response.status_code}")
    logging.info(f"Content-type: {response.headers.get('content-type')}")
    logging.info(f"Body: {response.text}")

def get_text_message_input(recipient, text):
    return {
        "messaging_product": "whatsapp",
        "recipient_type": "individual",
        "to": recipient,
        "type": "text",
        "text": {"preview_url": False, "body": text},
    }
```

On importe `get_response` depuis `agent_service.py`, ce qui montre le decouplage clair entre "comprendre et repondre" (`agent_service`) et "parler le protocole WhatsApp" (ce fichier). `get_text_message_input` construit le dictionnaire JSON exact attendu par l'API WhatsApp Business pour envoyer un message texte simple a un destinataire donne ; `log_http_response` est un utilitaire de journalisation pour deboguer les reponses de l'API en cas de probleme.

Etape 2 — Envoi effectif d'un message vers l'API WhatsApp

Lignes 37-54

```
def send_message(data):
    headers = {
        "Content-type": "application/json",
        "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    }
    url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"

    try:
        response = requests.post(url, json=data, headers=headers, timeout=10)
        response.raise_for_status()
        log_http_response(response)
```

```

        return response
    except requests.Timeout:
        logging.error("Timeout occurred while sending message")
        return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
    except requests.RequestException as e:
        logging.error(f"Request failed due to: {e}")
        return JSONResponse({"status": "error", "message": "Failed to send message"}, status_code=500)

```

Cette fonction realise l'appel sortant reel vers l'API Graph de Facebook/Meta pour livrer un message a l'utilisateur, avec un timeout de 10 secondes pour ne pas bloquer indefiniment si le service est lent. Les deux blocs except distinguent explicitement une erreur de timeout d'une erreur de requete plus generale, ce qui permet de journaliser un message de diagnostic precis plutot qu'un plantage generique -- important puisque cette fonction s'execute dans un thread en arriere-plan ou une erreur silencieuse serait difficile a detecter.

Etape 3 — Validation de la structure d'un message WhatsApp entrant

Lignes 57-65

```

def is_valid_whatsapp_message(body: dict):
    return (
        body.get("object")
        and body.get("entry")
        and body["entry"][0].get("changes")
        and body["entry"][0]["changes"][0].get("value")
        and body["entry"][0]["changes"][0]["value"].get("messages")
        and body["entry"][0]["changes"][0]["value"]["messages"][0]
    )

```

Le webhook WhatsApp peut recevoir des payloads JSON de formes tres variees (messages, accuses de lecture, changements de statut, evenements divers), tous imbriques dans la meme structure generique "entry/changes/value". Cette fonction verifie, niveau par niveau, que le chemin necessaire pour atteindre un vrai message texte existe bien avant de tenter d'y acceder, ce qui evite un plantage par KeyError si un evenement d'un autre type arrive sur le meme webhook.

Etape 4 — Extraction du message, appel a l'agent et envoi de la reponse

Lignes 68-77

```

def process_whatsapp_message(body: dict):
    value = body["entry"][0]["changes"][0]["value"]
    wa_id = value["contacts"][0]["wa_id"]
    name = value["contacts"][0]["profile"]["name"]
    message_body = value["messages"][0]["text"]["body"]

    response_text = get_response(message_body)
    data = get_text_message_input(wa_id, response_text)
    send_message(data)

```

C'est la fonction orchestratrice de tout le flux cote reception : elle extrait l'identifiant WhatsApp de l'expediteur (wa_id) et le texte brut du message, transmet ce texte a l'agent IA via get_response (qui gere en interne l'appel a Todoist et la logique de date), puis reconstruit une reponse au format WhatsApp attendu et l'envoie via send_message. C'est cette fonction que main.py lance dans un thread separe des reception d'un message valide, fermant ainsi la boucle complete : message WhatsApp entrant vers analyse par l'agent vers action Todoist vers reponse WhatsApp sortante.

Projet 18 — 18-Memory_agent

Assistants sante / immobilier avec memoire persistante

LE BUT DE CE PROJET

Le but de ce projet est de montrer comment donner a un agent conversationnel une veritable memoire persistante entre sessions, pour qu'il se souvienne d'un utilisateur d'une conversation a l'autre au lieu de repartir de zero a chaque fois. Le probleme que ca resout : par default, un agent LLM ne retient que ce qui se trouve dans la fenetre de contexte de la conversation en cours -- des qu'on ferme et rouvre la discussion, il a tout oublie. Or dans des cas d'usage comme un assistant de sante ou un assistant immobilier, il est essentiel de se souvenir d'informations donnees precedemment (antecedents medicaux mentionnes, budget et quartier recherches) pour eviter de faire repeter l'utilisateur et pour personnaliser vraiment l'accompagnement. Ce projet utilise le service externe Mem0 comme "cerveau memoire" : une base specialisee qui stocke des informations associees a un identifiant utilisateur et permet de les retrouver plus tard par recherche semantique (chercher par sens, pas seulement par mot-cle exact).

Deux variantes illustrent ce meme concept sur deux domaines differents et a des niveaux de sophistication differents. La version "Health-care" est la plus simple : un agent enregistre et retrouve des informations patient, avec repli local si la cle Mem0 est absente, sans jamais poser de diagnostic medical. La version "Real-Estate" va plus loin en combinant cette memoire conversationnelle avec une recherche vectorielle FAISS sur un catalogue de biens immobiliers : l'agent peut ainsi non seulement se souvenir des preferences du client, mais aussi retrouver les annonces les plus pertinentes semantiquement par rapport a une demande en langage naturel, sans que le client ait besoin de connaitre les criteres de filtrage exacts (prix, surface, quartier) a l'avance.

18-Memory_agent/Health-care/agent.py

Ce fichier definit un agent assistant sante qui utilise Mem0 (avec repli en memoire locale) pour enregistrer et retrouver des informations patient, et propose un outil de prise de rendez-vous.

Etape 1 — Configuration et initialisation du service memoire

Lignes 1-28

```
import os
from mem0 import MemoryClient
from google.adk.agents import LlmAgent
from google.adk.models.lite_llm import LiteLlm
from dotenv import load_dotenv

# Charger les variables d'environnement
load_dotenv()

# Configuration
USER_ID = "Isma"
APP_NAME = "agent"
SESSION_ID_SCHEMA_AGENT = "session_agent"
MEM0_API_KEY = os.getenv("MEM0_API_KEY")

class PatientService:
```

```

"""Service pour gérer les informations des patients avec mémoire persistante"""

def __init__(self, user_id) -> None:
    """Initialise le service avec l'ID utilisateur"""
    self.user_id = user_id
    if MEMO_API_KEY:
        self.memo_client = MemoryClient(api_key=MEMO_API_KEY)
        self.use_mem0 = True
    else:
        self.memo_client = None
        self.use_mem0 = False
        self.local_memory = {}

```

On importe Google ADK (Agent Development Kit) pour construire l'agent et sa configuration de modèle, ainsi que le client Mem0 pour la mémoire externe. La classe PatientService encapsule toute la logique mémoire : son constructeur vérifie si une cle Mem0 est configurée et, si oui, initialise un vrai client Mem0 relié à cet identifiant utilisateur ; sinon, il bascule sur un dictionnaire Python local en mémoire vive comme solution de repli. Ce mécanisme de fallback rend le projet démontrable même sans compte Mem0 configuré, tout en gardant la même interface de fonctions pour l'agent.

Etape 2 — Sauvegarde des informations patient

Lignes 30-48

```

def save_patient_info(self, information: str) -> dict:
    """Sauvegarde les informations importantes du patient en mémoire."""
    try:
        if self.use_mem0 and self.memo_client:
            response = self.memo_client.add(
                [{"role": "user", "content": information}],
                user_id=self.user_id,
                run_id="healthcare_session1",
                metadata={"type": "patient_info"}
            )
            return {"status": "success", "message": "Information sauvegardée"}
        else:
            # Fallback local
            if self.user_id not in self.local_memory:
                self.local_memory[self.user_id] = []
            self.local_memory[self.user_id].append(information)
            return {"status": "success", "message": "Information sauvegardée localement"}
    except Exception as e:
        return {"status": "error", "message": f"Erreur: {str(e)}"}

```

C'est l'outil que l'agent appelle dès qu'une information jugée importante apparaît dans la conversation (un antécédent, une allergie mentionnée, etc.). Selon le mode actif, l'information est soit envoyée à Mem0 avec des métadonnées qui la catégorisent ("patient_info") et l'associent à une session précise, soit simplement ajoutée à une liste locale associée à l'utilisateur. Le bloc try/except garantit que l'agent reçoit toujours une réponse structurée (succès ou erreur) plutôt qu'une exception qui interromprait la conversation.

Etape 3 — Récupération des informations patient

Lignes 50-75

```

def retrieve_patient_info(self, query: str) -> dict:
    """Récupère les informations pertinentes du patient depuis la mémoire."""
    try:
        if self.use_mem0 and self.memo_client:
            results = self.memo_client.search(
                query,
                user_id=self.user_id,
                limit=5,
                threshold=0.7,
                output_format="v1.1"
            )

```

```

        if results and len(results) > 0:
            memories = [memory["memory"] for memory in results.get("results", [])]
            return {"status": "success", "memories": memories, "count": len(memories)}
        else:
            return {"status": "no_results", "memories": [], "count": 0}
    else:
        # Fallback local
        if self.user_id in self.local_memory and self.local_memory[self.user_id]:
            memories = self.local_memory[self.user_id]
            return {"status": "success", "memories": memories, "count": len(memories)}
        else:
            return {"status": "no_results", "memories": [], "count": 0}
except Exception as e:
    return {"status": "error", "message": f"Erreur: {str(e)}", "memories": [], "count": 0}

```

Cette fonction permet à l'agent de retrouver, avant de répondre, ce qu'il sait déjà sur ce patient. Avec Mem0, la recherche est sémantique : elle ne cherche pas un mot exact mais des souvenirs dont le sens se rapproche de la requête (paramètre `threshold=0.7` pour ne garder que les correspondances suffisamment pertinentes), limitée aux 5 résultats les plus proches. C'est ce mécanisme qui permet à l'agent d'éviter de poser une question déjà répondue lors d'une session précédente, même si l'utilisateur ne reformule pas exactement les mêmes mots.

Etape 4 — Prise de rendez-vous et instanciation du service

[Lignes 77-91](#)

```

def schedule_appointment(self, date: str, time: str, reason: str) -> dict:
    """Programme un rendez-vous médical."""
    try:
        appointment_id = f"APPT-{hash(date + time) % 10000}"
        return {
            "status": "success",
            "appointment_id": appointment_id,
            "confirmation": f"Rendez-vous programmé le {date} à {time} pour {reason}",
            "message": "Merci d'arriver 15 minutes en avance."
        }
    except Exception as e:
        return {"status": "error", "message": f"Erreur: {str(e)}"}

# Initialiser le service
patient_service = PatientService(USER_ID)

```

Ce troisième outil simule la prise d'un rendez-vous médical en générant un identifiant pseudo-unique à partir d'un hash de la date et de l'heure (une simplification volontaire : dans une vraie application, ceci se connecterait à un vrai système de planification). L'instance `patient_service` est créée une seule fois au niveau du module, ce qui garantit qu'un seul objet gère la mémoire de l'utilisateur pendant toute la durée de vie de l'application, plutôt que d'en recréer un à chaque appel.

Etape 5 — Définition de l'agent Google ADK et de ses instructions

[Lignes 93-123](#)

```

# IMPORTANT: Créer l'agent au niveau du module pour ADK
root_agent = LlmAgent(
    name="healthcare_assistant",
    model=Litellm(model="openai/gpt-4o-mini"),
    description="Assistant IA Santé pour accompagner les patients",

    instruction="""
    Vous êtes un assistant IA de santé professionnel et bienveillant.

    Quand on vous salue, répondez chaleureusement en vous présentant.

    Vos responsabilités:
    1. Enregistrer les informations de santé avec save_patient_info
    2. Récupérer l'historique avec retrieve_patient_info
    3. Programmer des rendez-vous avec schedule_appointment
    """
)

```

```

Comportement:
- Ton empathique et professionnel
- Ne donnez JAMAIS de diagnostic médical
- Encouragez à consulter un professionnel pour les symptômes sérieux
- Respectez la confidentialité

Exemple de salutation: "Bonjour ! Je suis votre assistant santé IA. Comment puis-je vous aider
aujourd'hui ?"
"""

tools=[
    patient_service.save_patient_info,
    patient_service.retrieve_patient_info,
    patient_service.schedule_appointment,
],
)

# Code d'initialisation optionnel
if __name__ == "__main__":
    print("Assistant IA Santé prêt")
    print(f"Utilisateur: {USER_ID}")

```

L'agent est construit avec LiteLlm, une couche d'abstraction qui permet à Google ADK d'utiliser un modèle OpenAI (ici gpt-4o-mini, un modèle plus léger et économique) comme s'il s'agissait d'un modèle natif. Les trois méthodes de PatientService lui sont fournies directement comme outils, ce qui lui permet de décider seul, selon la conversation, quand enregistrer une information, quand consulter la mémoire ou quand programmer un rendez-vous. Les instructions posent un garde-fou éthique et légal essentiel dans ce domaine sensible : ne jamais poser de diagnostic médical et orienter systématiquement vers un professionnel de santé pour tout symptôme sérieux. La création de root_agent directement au niveau du module (et non dans une fonction) est signalée comme importante car c'est la convention attendue par le framework ADK pour découvrir automatiquement l'agent.

18-Memory_agent/Real-Estate/agent.py.py

Ce fichier définit un agent assistant immobilier combinant mémoire Mem0 (préférences client) et recherche vectorielle FAISS sur un catalogue de biens, pour répondre de façon personnalisée et retrouver des annonces pertinentes sémantiquement.

Etape 1 — Configuration et définition de la classe d'outils mémoire

Lignes 1-29

```

import os
import json
import faiss
import warnings
from mem0 import MemoryClient
from dotenv import load_dotenv
from google.adk.agents import LlmAgent
from google.adk.models.lite_llm import LiteLlm
from sentence_transformers import SentenceTransformer

# Supprimer les avertissements de dépréciation
warnings.filterwarnings("ignore", category=DeprecationWarning)

# Charger les variables d'environnement
load_dotenv()

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
MEM0_API_KEY = os.getenv("MEM0_API_KEY")
USER_ID = "Ismo"

class MemoryTools:
    """Classe pour gérer la mémoire persistante et la recherche vectorielle"""

```

```
def __init__(self):
    """Initialisation du client mémoire et des outils de recherche"""
    self.mem0_client = MemoryClient(MEM0_API_KEY)

    # Initialisation du modèle d'embedding et de l'index FAISS
    self._initialize_search_system()
```

En plus de Mem0 pour la mémoire conversationnelle, ce fichier importe faiss (bibliothèque de recherche vectorielle développée par Meta) et sentence_transformers (pour convertir du texte en vecteurs numériques, ou "embeddings"). La classe MemoryTools regroupe deux capacités distinctes dans un seul objet : se souvenir du client (via Mem0) et chercher intelligemment dans un catalogue de propriétés (via FAISS), toutes deux initialisées dès la construction de l'objet.

Etape 2 — Chargement du catalogue et construction de l'index de recherche FAISS

[Lignes 31-81](#)

```
def _initialize_search_system(self):
    """Initialise le système de recherche vectorielle FAISS"""
    # Chargement des données JSON des propriétés
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))

    try:
        with open(os.path.join(BASE_DIR, "agency_property_data.json"), "r") as f:
            self.data = json.load(f)
    except FileNotFoundError:
        print("Fichier de données immobilières non trouvé")
        self.data = []

    # Initialisation du modèle d'embedding
    self.embed_model = SentenceTransformer("all-MiniLM-L6-v2")

    # Extraction des textes pour l'embedding
    self.texts = self._extract_property_texts()

    # Calcul des embeddings
    if self.texts:
        embeddings = self.embed_model.encode(self.texts, convert_to_numpy=True)

        # Création de l'index FAISS
        dim = embeddings.shape[1]
        self.faiss_index = faiss.IndexFlatL2(dim)
        self.faiss_index.add(embeddings)
    else:
        self.faiss_index = None

def _extract_property_texts(self) -> list:
    """Extrait et formate les textes des propriétés pour l'embedding"""
    texts = []

    for item in self.data:
        # Création d'un texte descriptif complet pour chaque propriété
        property_text = (
            f"{item.get('title', '')} "
            f"{item.get('description', '')} "
            f"{item.get('location', '')} "
            f"{str(item.get('price', ''))} "
            f"{item.get('currency', '')} "
            f"{str(item.get('surface_area', ''))} "
            f"{str(item.get('rooms', ''))} "
            f"{str(item.get('bedrooms', ''))} "
            f"{str(item.get('bathrooms', ''))} "
            f"{' '.join(item.get('features', []))} "
            f"{item.get('status', '')}"
        )
        texts.append(property_text)
```

```
return texts
```

Cette étape prépare, une seule fois au démarrage, tout ce qu'il faut pour une recherche sémantique rapide : elle charge le catalogue de propriétés depuis un fichier JSON local (avec repli sur une liste vide si le fichier manque), transforme chaque annonce en un texte descriptif unique combinant titre, description, prix, surface et caractéristiques, puis convertit tous ces textes en vecteurs numériques (embeddings) via le modèle SentenceTransformer. Ces vecteurs sont ensuite indexés dans une structure FAISS (IndexFlatL2, une recherche par distance euclidienne) qui permettra plus tard de retrouver, en quelques millisecondes, les annonces dont le sens se rapproche le plus d'une requête en langage naturel, sans avoir à comparer mot à mot.

Etape 3 — Sauvegarde et récupération des préférences client via Mem0

[Lignes 83-139](#)

```
def save_info(self, information: str) -> dict:
    """Sauvegarde les préférences importantes du client en mémoire"""
    try:
        # Stockage dans Mem0
        response = self.mem0_client.add(
            [{"role": "user", "content": information}],
            user_id=USER_ID,
            run_id="real_estate_session",
            metadata={"type": "property_info"}
        )

        return {
            "status": "success",
            "message": "Informations client sauvegardées avec succès",
            "response": response
        }

    except Exception as e:
        return {
            "status": "error",
            "message": f"Erreur lors de la sauvegarde: {str(e)}"
        }

def retrieve_info(self, query: str) -> dict:
    """Récupère les informations pertinentes du client depuis la mémoire"""
    try:
        # Recherche dans Mem0
        results = self.mem0_client.search(
            query,
            user_id=USER_ID,
            limit=5,
            threshold=0.7, # Seuil plus élevé pour des résultats plus pertinents
            output_format="v1.1"
        )

        # Formatage et retour des résultats
        if results and len(results) > 0:
            memories = [memory["memory"] for memory in results.get("results", [])]
            return {
                "status": "success",
                "memories": memories,
                "count": len(memories)
            }
        else:
            return {
                "status": "no_results",
                "memories": [],
                "count": 0
            }

    except Exception as e:
        return {
            "status": "error",
```

```

        "message": f"Erreur lors de la récupération: {str(e)}",
        "memories": [],
        "count": 0
    }

```

Ces deux outils fonctionnent en miroir de la version Health-care : `save_info` enregistre dans `Mem0` une préférence client (budget, quartier recherche, critères) associée à l'utilisateur courant et à la session `"real_estate_session"`, tandis que `retrieve_info` effectue une recherche sémantique dans ces souvenirs, limitée aux résultats dont la pertinence dépasse un seuil de 0.7. C'est ce qui permet à l'agent de rester cohérent d'un échange à l'autre sans faire répéter au client ses critères déjà donnés.

Etape 4 — Recherche vectorielle FAISS dans le catalogue de propriétés

[Lignes 141-176](#)

```

def search_faiss(self, query: str, top_k: int = 5) -> dict:
    """Recherche FAISS et retourne les top_k résultats"""
    try:
        if not self.faiss_index or not self.texts:
            return {
                "status": "error",
                "message": "Index FAISS non initialisé",
                "results": [],
                "count": 0
            }

        # Encodage de la requête
        query_vec = self.embed_model.encode([query], convert_to_numpy=True)

        # Recherche dans l'index FAISS
        D, I = self.faiss_index.search(query_vec, top_k)

        # Récupération des résultats originaux
        results = []
        for idx in I[0]:
            if idx < len(self.data): # Vérification des limites
                results.append(self.data[idx])

        return {
            "status": "success",
            "results": results,
            "count": len(results)
        }

    except Exception as e:
        return {
            "status": "error",
            "message": f"Erreur lors de la recherche: {str(e)}",
            "results": [],
            "count": 0
        }

```

C'est l'outil qui met en œuvre concrètement la recherche sémantique préparée plus haut : la requête du client (par exemple "appartement lumineux près du centre avec balcon") est encodée dans le même espace vectoriel que les annonces, puis l'index FAISS renvoie les indices des top_k biens les plus proches. Ce mécanisme permet de trouver des annonces pertinentes même si le vocabulaire exact du client ne correspond pas mot pour mot au texte des annonces, ce qu'une simple recherche par mots-clés ne saurait pas faire.

Etape 5 — Prise de rendez-vous et coordonnées de l'agence

[Lignes 178-201](#)

```

def schedule_appointment(self, date: str, time: str, reason: str) -> dict:
    """Programme un rendez-vous"""
    try:
        # Dans une vraie application, ceci se connecterait à un système de planification
        appointment_id = f"APPT-{hash(date + time) % 10000}"

```

```

        return {
            "status": "success",
            "appointment_id": appointment_id,
            "confirmation": f"Appointment scheduled for {date} at {time} for {reason}"
        }

    except Exception as e:
        return {
            "status": "error",
            "message": f"Erreur lors de la planification: {str(e)}"
        }

def get_contact_info(self) -> dict:
    """Retourne les informations de contact de l'agence"""
    return {
        "status": "success",
        "contact_info": "You can contact the agency at +33 6 12 34 56 78 or by email at
realestateagency@mail.com"
    }

```

Ces deux outils complètent le parcours client typique d'une agence immobilière : `schedule_appointment` simule la prise d'un rendez-vous de visite (même simplification par hash que dans la version Health-care), et `get_contact_info` fournit une réponse fixe avec les coordonnées de l'agence, pour que l'agent puisse orienter le client vers un contact humain quand c'est pertinent, sans avoir à inventer cette information.

Etape 6 — Construction de l'agent immobilier avec instructions détaillées

[Lignes 204-261](#)

```

# =====
# CRÉATION DE L'AGENT IMMOBILIER
# =====
def create_real_estate_agent():
    """Crée et configure l'agent IA immobilier professionnel"""

    # Initialisation des outils mémoire
    memory_tools = MemoryTools()

    # Création de l'agent avec LiteLlm pour OpenAI
    root_agent = LlmAgent(
        name="real_estate_assistant",
        model=LiteLlm(model="openai/gpt-4o-mini"),

        description="""
        AI-powered Real Estate Assistant for a reputable agency, supporting clients by recording
        preferences,
        retrieving past inquiries, searching property listings, scheduling viewings, and providing
        professional guidance.
        """,

        instruction="""
        You are a professional Real Estate Assistant representing a top-tier agency in France. Always
        speak in a courteous, confident, and expert manner—just like a senior agent at our firm.
        ...
        """,

        # Outils disponibles pour l'agent
        tools=[
            memory_tools.save_info,
            memory_tools.retrieve_info,
            memory_tools.search_faiss,
            memory_tools.schedule_appointment,
            memory_tools.get_contact_info,
        ],
    )

    return root_agent

```

Contrairement à la version Health-care où l'agent est créé directement au niveau du module, ici la construction est encapsulée dans une fonction `create_real_estate_agent` qui instancie d'abord ses propres `MemoryTools` puis assemble l'agent avec ses cinq outils (mémoire et recherche confondues). Les instructions, très détaillées, définissent un persona professionnel précis ("agent immobilier haut de gamme"), décrivent quand utiliser chaque outil (vérifier la mémoire avant de redemander une information, ne pas donner de conseil juridique ou financier, proposer les coordonnées de l'agence), ce qui montre comment un prompt système long et structuré peut cadrer un comportement commercial et déontologique complet à partir des mêmes cinq fonctions Python.

Étape 7 — Point d'entrée principal et vérification du chargement des données

[Lignes 264-287](#)

```
# =====
# POINT D'ENTRÉE PRINCIPAL
# =====
if __name__ == "__main__":
    try:
        # Initialisation des outils mémoire
        memory_tools = MemoryTools()

        # Créer l'agent immobilier
        real_estate_agent = create_real_estate_agent()

        print(" Assistant IA Immobilier initialisé avec succès!")
        print(f" Utilisateur: {USER_ID}")
        print(" Prêt à accompagner vos clients...")

        # Informations sur les données chargées
        if hasattr(memory_tools, 'data') and memory_tools.data:
            print(f" {len(memory_tools.data)} propriétés chargées dans la base")
        else:
            print(" Aucune donnée de propriété chargée")

    except Exception as e:
        print(f" Erreur lors de l'initialisation: {e}")
        print("Vérifiez vos clés API et dépendances")
```

Ce bloc, exécute uniquement en lancement direct du script, sert de test de fumée ("smoke test") : il crée une instance de `MemoryTools` et l'agent, puis affiche des messages de statut incluant le nombre de propriétés effectivement chargées depuis le fichier JSON -- un indicateur rapide pour vérifier que l'index de recherche FAISS a bien quelque chose à chercher. Le `try/except` global capture toute erreur d'initialisation (cle API manquante, dépendance non installée) et invite explicitement à vérifier la configuration plutôt que de laisser une trace d'erreur brute et peu exploitable.

Projet 20 — 20-Agno-Team

Analyse de contrats par 3 agents specialises coordonnees

LE BUT DE CE PROJET

Le but de ce projet est de montrer comment automatiser la relecture d'un contrat juridique en confiant le travail non pas à une seule IA généraliste, mais à une équipe de plusieurs agents IA spécialisés qui collaborent, chacun avec son propre angle d'analyse.

Le problème que ça resout : relire un contrat pour repérer les clauses ambiguës, les risques juridiques et les points négociables est un travail long, qui demande des compétences différentes (organisation du document, droit, stratégie de négociation). Demander tout ça d'un seul coup à un seul LLM produit souvent une réponse superficielle qui mélange les angles et oublie des points. La solution démontrée ici consiste à découper le problème en trois métiers distincts confiés à trois agents IA différents (un agent "structure" qui vérifie que le contrat est bien organisé, un agent "juridique" qui traque les clauses à risque en les citant précisément, et un agent "négociation" qui repère les clauses à renégocier), puis à faire relire et fusionner leurs trois rapports par un quatrième agent "manager" qui produit une synthèse unique et exploitable.

Pourquoi ça compte : c'est une illustration concrète du concept d'"équipe d'agents" (multi-agent team), une approche de plus en plus utilisée pour des tâches complexes où un seul prompt ne suffit pas. Elle montre aussi une technique importante contre les hallucinations des LLM : chaque agent est obligé de citer le texte exact du contrat pour justifier chaque remarque, plutôt que d'affirmer des choses sans preuve. Enfin, le projet est packagé avec une interface web (upload d'un fichier, bouton d'analyse, téléchargement du rapport), ce qui montre comment rendre ce genre de système utilisable par une personne non technique.

20-Agno-Team/contract_analysis.py

Définit les trois agents IA spécialisés (structure, juridique, négociation) et l'équipe (Team) qui les coordonne pour produire un rapport d'analyse de contrat unifié.

Etape 1 — Configuration du modèle et connexion au document

Lignes 1-21

```
import os
from dotenv import load_dotenv
from agno.agent import Agent
from agno.team.team import Team
from agno.models.groq import Groq
from textwrap import dedent
from datetime import datetime
import utils

load_dotenv()

# Le script original utilisait OpenAI (gpt-4o) ; remplacé par Groq pour
# réutiliser la clé GROQ_API_KEY déjà disponible.
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
```

```
GROQ_MODEL_ID = "openai/gpt-oss-120b"
```

```
def get_document():  
    """  
    Récupère le document à analyser depuis le module utils  
    """  
    return utils.get_document()
```

Cette étape prépare le terrain avant de créer le moindre agent. `load_dotenv()` charge la clé d'API Groq depuis un fichier `.env` local plutôt que de l'écrire en dur dans le code (bonne pratique de sécurité). `GROQ_MODEL_ID` fixe le modèle de langage que les agents spécialisés utiliseront par défaut. La fonction `get_document()` est le pont vers `utils.py` : elle sera donnée aux agents comme "outil" (tool use / function calling), c'est-à-dire une fonction Python que le LLM peut décider d'appeler lui-même pendant son raisonnement pour aller chercher le texte du contrat, plutôt que d'inventer un contenu.

Etape 2 — Agent Structure — vérifier l'organisation du contrat

[Lignes 23-54](#)

```
def create_structure_agent():  
    """  
    Crée l'agent spécialisé dans l'analyse structurelle des contrats  
    """  
    return Agent(  
        model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),  
        name='Agent Structure',  
        role='Expert en Structuration de Contrats',  
        instructions=dedent("""  
            Vous êtes un Expert en Structuration de Contrats. Votre rôle est d'évaluer la structure  
            d'un contrat et de suggérer des améliorations ou de construire une structure appropriée  
            à partir de zéro.  
  
            Vous utiliserez l'outil `get_document` pour récupérer le texte complet du contrat.  
            Votre tâche est d'analyser le contrat et de déterminer s'il est structuré de manière  
            claire, complète et juridiquement appropriée.  
  
            Vous devez identifier les sections manquantes ou peu claires :  
            - S'il manque de structure, suggérez une structure complète en utilisant des en-têtes  
              de section standard (ex: Définitions, Conditions, Obligations, Résiliation, Loi  
              applicable).  
            - Évitez l'interprétation juridique - concentrez-vous uniquement sur l'organisation,  
              la clarté et le flux logique.  
            - Soyez concis mais clair dans votre analyse.  
  
            Produisez une structure au format markdown si vous créez une nouvelle structure,  
            ou des commentaires sous forme de puces si vous évaluez une structure existante.  
  
            """),  
        tools=[get_document],  
        markdown=True,  
    )
```

Cette fonction "usine" construit le premier agent spécialisé. Son `role` et ses `instructions` (qui jouent le rôle de "system prompt") le cantonnent volontairement à un seul angle : l'organisation du document (sections, en-têtes, clarté), en lui interdisant explicitement de faire de l'interprétation juridique. Cette séparation stricte des rôles évite que les trois agents se marchent dessus et produisent des rapports redondants. L'outil `get_document` lui est fourni pour qu'il aille chercher lui-même le texte réel du contrat plutôt que de deviner son contenu.

Etape 3 — Agent Juridique — détecter les risques avec citations obligatoires

[Lignes 56-99](#)

```
def create_legal_agent():  
    """  
    Crée l'agent spécialisé dans l'analyse juridique des contrats
```

```

"""
return Agent(
    model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
    name='Agent Juridique',
    role='Analyste des Questions Juridiques',
    instructions=dedent("""
        Vous êtes un Analyste du Cadre Juridique chargé d'identifier les problèmes juridiques,
        les risques et les principes juridiques clés dans le contrat téléchargé.

        Utilisez l'outil `get_document` pour accéder au texte complet du contrat.
        Pour chaque problème juridique ou observation, vous DEVEZ :

        - Citer la clause, phrase ou paragraphe exact du contrat sur lequel votre point est basé.
        - Commencer une nouvelle ligne avec 'Problème :' suivi d'une explication courte
          et claire de la préoccupation ou du principe juridique.
        - Faire clairement référence au titre de section, en-tête ou numéro de paragraphe
          si disponible. Sinon, décrire son emplacement (ex: "section commençant par [texte]...").
        - NE PAS faire d'évaluation ou commentaire juridique à moins qu'il ne soit directement
          soutenu par une citation du contrat.

        Votre tâche :
        - Identifier le domaine juridique du contrat (ex: droit commercial, emploi, NDA, etc.)
        - Déterminer la juridiction probable ou la loi applicable
        - Souligner tout problème juridique potentiel ou clause problématique
        - Vérifier la conformité RGPD si applicable
        - Identifier les déséquilibres de responsabilité et de responsabilité

        Formatez chaque constatation comme suit :
        Clause :
        "Texte cité du contrat ici."

        Section : (Titre de section ou emplacement)

        Problème :
        Votre brève analyse de pourquoi cette clause peut présenter une préoccupation juridique.

        """),
    tools=[get_document],
    debug_mode=False,
    markdown=True,
)

```

Cet agent est le cœur de la démonstration anti-hallucination : ses instructions lui interdisent explicitement toute évaluation juridique qui ne serait pas "directement soutenue par une citation du contrat". Chaque constat doit suivre un gabarit fixe (Clause citée / Section / Problème), ce qui force le modèle à ancrer chaque affirmation dans le texte réel plutôt que dans des connaissances générales, et rend en plus le rapport final facile à parcourir et à vérifier par un humain.

Etape 4 — Agent Négociation — repérer les clauses à renégocier

[Lignes 101-132](#)

```

def create_negotiation_agent():
    """
    Crée l'agent spécialisé dans les stratégies de négociation contractuelle
    """
    return Agent(
        model=Groq(id=GROQ_MODEL_ID, api_key=GROQ_API_KEY),
        name='Agent Négociation',
        role='Stratège en Négociation de Contrats',
        instructions=dedent("""
            Vous êtes un Stratège en Négociation de Contrats.

            Votre travail est d'identifier les parties d'un contrat qui sont couramment négociables
            ou potentiellement déséquilibrées. Vous DEVEZ :

            - Toujours citer le paragraphe ou la clause exacte à laquelle vous faites référence.

```

- Expliquer clairement pourquoi elle peut être négociable ou nécessite un ajustement.
- Suggérer une contre-offre ou une formulation alternative.

Structurez votre analyse comme ceci :

1. **Clause Citée** (texte exact du contrat)
2. **Pourquoi elle est négociable ou problématique**
3. **Exemple de stratégie ou contre-suggestion**

NE FAITES PAS de commentaires généraux. Chaque point que vous faites doit être soutenu par une citation directe du contrat, et votre sortie doit clairement montrer quelle partie du contrat vous analysez.

```
"""),
tools=[get_document],
debug_mode=False,
markdown=True,
)
```

Troisième agent spécialisé, sur le même moule que les deux précédents mais orienté "stratégie" : il repère les clauses déséquilibrées et propose une contre-offre concrète. La même contrainte de citation exacte est réutilisée, montrant que le pattern "grounding par citation" est appliqué de façon cohérente à travers toute l'équipe, pas seulement à l'agent juridique.

Etape 5 — Équipe manager — délégation forcée et synthèse finale

[Lignes 134-197](#)

```
def create_manager_team():
    """
    Crée l'équipe de management qui coordonne tous les agents spécialisés

    Returns:
        Team: Équipe coordonnée d'agents pour l'analyse complète de contrats
    """

    # Création des agents spécialisés
    structure_agent = create_structure_agent()
    legal_agent = create_legal_agent()
    negotiate_agent = create_negotiation_agent()

    # Équipe coordonnée par un agent manager
    manager_agent = Team(
        members=[structure_agent, legal_agent, negotiate_agent],
        # Le leader doit synthétiser en un seul appel les 3 rapports complets
        # (Structure + Légal + Négociation), ce qui dépasse la limite Groq du
        # modèle 120B (8000 tokens/minute) ; le modèle 20B a un quota plus élevé.
        model=Groq(id="openai/gpt-oss-20b", api_key=GROQ_API_KEY),
        mode='coordinate',
        # Force la délégation systématique aux 3 agents (le rapport a besoin des
        # 3 apports) plutôt que de laisser le leader décider s'il délègue ou non.
        delegate_to_all_members=True,
        # success_criteria n'existe plus comme paramètre séparé de Team : son contenu
        # est fusionné dans instructions ci-dessous.
        instructions=dedent("""
            Critère de réussite : un résumé bien organisé et traçable du contrat qui inclut
            un contexte juridique soulignant les problèmes juridiques potentiels avec le texte
            du contrat cité comme preuve, une revue structurelle avec des suggestions de clarté
            et de formatage, des stratégies de négociation directement liées à des paragraphes
            ou clauses spécifiques, une évaluation des risques consolidée avec des recommandations
            actionnables, et un classement de priorité clair de tous les problèmes identifiés.

            Vous êtes le Coordinateur principal et résumeur. Vous devez combiner les apports de :
            1. Agent Structure
            2. Agent Juridique
            3. Agent Négociation

            IMPORTANT : Ne demandez JAMAIS à l'utilisateur de fournir le texte du contrat.
            Déléguez immédiatement la tâche aux trois agents ci-dessus : chacun a accès à
```

```
l'outil get_document() et récupérera lui-même le contrat déjà chargé en mémoire.
```

Exigences clés :

- Pour tous les points juridiques et de négociation, préservez les clauses citées du contrat comme preuve.
- L'Agent Juridique devrait souligner des problèmes juridiques spécifiques, suivis d'une courte explication 'Problème : '.
- Chaque extrait cité doit être suivi de l'explication ou recommandation de l'agent.
- Consolidez tous les scores en une évaluation globale des risques
- Fournissez un résumé exécutif clair avec des étapes actionnables suivantes
- Assurez-vous de la traçabilité complète de toutes les contributions des agents

Format final :

1. Résumé Exécutif
2. Contexte Juridique (Avec clauses citées et explications des problèmes)
3. Retours sur la Structure du Contrat
4. Recommandations de Négociation

```
"""),  
debug_mode=True,  
markdown=True,  
)  
  
return manager_agent
```

C'est ici que les trois agents deviennent une véritable équipe. `mode='coordinate'` fait du `Team` un leader qui délègue le travail puis fusionne les réponses (plutôt que de simplement router vers un seul agent). `delegate\_to\_all\_members=True` force ce leader à toujours consulter les trois agents, sans lui laisser le choix : sans cette contrainte, un LLM pourrait décider par lui-même de ne solliciter qu'un ou deux agents et casser le format de rapport attendu. Le leader utilise volontairement un modèle différent (20B au lieu de 120B) car il doit ingérer d'un coup les trois rapports complets, ce qui dépasse le quota de débit (tokens/minute) du modèle plus gros sur l'infrastructure Groq — une contrainte pratique typique du travail avec des API de LLM à quota limité. Ses instructions imposent enfin un format de sortie fixe en quatre sections, pour que le rapport final soit exploitable directement par un humain.

20-Agno-Team/utils.py

Gère le chargement d'un fichier de contrat et l'extraction de son texte selon son format (PDF, Word, texte), et sert d'état partagé entre l'interface et les agents.

Etape 1 — État global du document actif

Lignes 1-10

```
from pathlib import Path  
import fitz # PyMuPDF  
  
# Variable globale  
current_file_path = None  
  
def set_document_file(file_path):  
    """Définit le fichier à analyser"""  
    global current_file_path  
    current_file_path = file_path
```

Une variable globale `current\_file\_path` sert de mémoire partagée entre l'interface (`app.py`, qui appelle `set\_document\_file` après l'upload) et les agents (`contract\_analysis.py`, qui appellent `get\_document` plus loin). Cette approche simple évite d'avoir à faire transiter le chemin du fichier à travers tous les appels d'agents : n'importe quelle fonction du module peut savoir "quel est le document actif en ce moment".

Etape 2 — Détection du format et extraction avec gestion d'erreurs

Lignes 12-35

```
def get_document():
    """Retourne le contenu du fichier"""
    global current_file_path

    if not current_file_path:
        return {"error": "Aucun fichier", "content": ""}

    file_path = Path(current_file_path)
    extension = file_path.suffix.lower()

    try:
        if extension == '.pdf':
            content = extract_pdf(file_path)
        elif extension in ['.docx', '.doc']:
            content = extract_word_simple(file_path)
        elif extension == '.txt':
            content = extract_text(file_path)
        else:
            return {"error": "Format non supporté", "content": ""}

        return {"content": content, "error": None}

    except Exception as e:
        return {"error": str(e), "content": ""}
```

C'est la fonction que les agents appellent réellement comme "outil". Elle regarde l'extension du fichier pour choisir la bonne méthode d'extraction, et surtout, elle ne laisse jamais planter le programme : toute erreur (aucun fichier défini, format non supporté, exception pendant la lecture) est transformée en un dictionnaire `{"error": ..., "content": ...}` que l'agent peut lire et comprendre, plutôt qu'une exception qui casserait tout le workflow d'analyse.

Etape 3 — Extraction du texte selon le format de fichier

[Lignes 37-62](#)

```
def extract_pdf(file_path):
    """Extrait le texte d'un PDF"""
    doc = fitz.open(file_path)
    text = ""
    for page in doc:
        text += page.get_text()
    doc.close()
    return text

def extract_word_simple(file_path):
    """Extrait le texte d'un fichier Word avec PyMuPDF"""
    try:
        # PyMuPDF peut aussi lire les fichiers Word !
        doc = fitz.open(file_path)
        text = ""
        for page in doc:
            text += page.get_text()
        doc.close()
        return text
    except:
        return "Erreur: Impossible de lire ce fichier Word. Convertissez-le en PDF."

def extract_text(file_path):
    """Extrait le texte d'un fichier texte"""
    with open(file_path, 'r', encoding='utf-8') as file:
        return file.read()
```

Trois fonctions dédiées, une par format supporté. Fait notable : la même bibliothèque PyMuPDF (`fitz`), pourtant pensée pour le PDF, est réutilisée pour lire aussi les fichiers Word, ce qui évite d'ajouter une dépendance supplémentaire juste pour ce cas ; si ça échoue, un message clair invite à convertir le fichier en PDF. Le fichier texte brut est lu simplement en UTF-8 pour gérer correctement les accents.

20 - Agno - Team/app.py

Interface web Streamlit permettant d'uploader un contrat, de lancer son analyse par l'équipe d'agents et d'afficher/télécharger le rapport résultant.

Etape 1 — Upload et sauvegarde temporaire du fichier

Lignes 1-20

```
import streamlit as st
import tempfile
from pathlib import Path
from utils import get_document, set_document_file
from contract_analysis import create_manager_team

def main():
    st.title("Analyseur de Contrats")

    # Upload du fichier
    uploaded_file = st.file_uploader(
        "Choisir un contrat",
        type=['pdf', 'docx', 'txt']
    )

    if uploaded_file:
        # Sauvegarder temporairement
        with tempfile.NamedTemporaryFile(delete=False, suffix=Path(uploaded_file.name).suffix) as
tmp_file:
            tmp_file.write(uploaded_file.getvalue())
            tmp_file_path = tmp_file.name
```

Streamlit fournit un widget d'upload qui restreint les formats acceptés aux trois formats gérés par `utils.py`. Le fichier téléversé n'existe qu'en mémoire (bytes) côté Streamlit, alors que les fonctions d'extraction attendent un vrai chemin de fichier sur disque : il faut donc l'écrire dans un fichier temporaire (`delete=False` pour qu'il survive après le bloc `with`, puisqu'il doit rester lisible plus tard).

Etape 2 — Lancement de l'analyse par l'équipe d'agents

Lignes 22-34

```
# Bouton d'analyse
if st.button("Analyser le contrat", type="primary"):

    with st.spinner("Analyse en cours..."):
        try:
            # Charger le document
            set_document_file(tmp_file_path)

            # Créer l'équipe d'agents
            manager_team = create_manager_team()

            # Lancer l'analyse
            result = manager_team.run("Analysez ce contrat complètement.")
```

Au clic sur le bouton, le fichier temporaire est enregistré comme document actif (`set\_document\_file`), puis une équipe fraîche d'agents est recrée à chaque analyse. `manager\_team.run(...)` déclenche tout le pipeline vu dans `contract\_analysis.py` : délégation aux trois agents spécialisés puis synthèse par le leader. Le `st.spinner` donne un retour visuel pendant cette opération potentiellement longue (plusieurs appels LLM enchaînés).

Etape 3 — Affichage du rapport et téléchargement

Lignes 36-56

```
# Afficher les résultats
st.success("Analyse terminée!")

if hasattr(result, 'content'):
```

```

        analysis_text = result.content
    else:
        analysis_text = str(result)

    st.markdown(analysis_text)

    # Bouton de téléchargement
    st.download_button(
        " Télécharger le rapport",
        analysis_text,
        file_name=f"analyse_{uploaded_file.name}.md",
        mime="text/markdown"
    )

except Exception as e:
    st.error(f" Erreur: {str(e)}")

if __name__ == "__main__":
    main()

```

Le résultat est récupéré de façon défensive (`hasattr(result, 'content')`) pour rester robuste si la forme exacte de l'objet retourné par agno change. Le rapport est ensuite affiché en markdown (cohérent avec `markdown=True` côté agents) et proposé au téléchargement en `.md`. Un `try/except` englobe toute l'analyse pour afficher une erreur lisible plutôt que de faire planter l'application en cas de problème d'API ou d'extraction.

Projet 21 — 21-SDV-synthetise

Generation de donnees synthetiques (SDV + GPT-2)

LE BUT DE CE PROJET

Le but de ce projet est de montrer comment générer des données synthétiques réalistes à partir d'un jeu de données réel, pour pouvoir travailler ou partager des données sans exposer les vraies informations sensibles.

Le problème que ça résout : dans beaucoup de contextes (entreprise, recherche, formation, tests de logiciels), on a besoin d'un jeu de données pour développer ou démontrer quelque chose, mais les vraies données contiennent des informations personnelles ou confidentielles (clients, commandes, avis...) qu'on n'a pas le droit de diffuser librement. Recréer un faux jeu de données à la main serait long et ne reproduirait pas les mêmes tendances statistiques que les vraies données (moyennes, corrélations entre colonnes, etc.), ce qui le rendrait peu utile pour des tests réalistes. Ce projet montre comment un modèle statistique peut apprendre les "règles" cachées d'un dataset réel (distributions, corrélations) puis générer de nouvelles lignes fictives qui suivent ces mêmes règles, sans jamais copier une vraie ligne existante.

Pourquoi ça compte : cette technique s'appelle la génération de données synthétiques, et elle est de plus en plus utilisée pour concilier besoin de données réalistes et protection de la vie privée. Le projet va même plus loin en montrant deux approches complémentaires : une approche statistique classique (une "copule gaussienne", qui capture les distributions et corrélations de données tabulaires comme des commandes ou des montants) pour les données chiffrées/catégorielles, et une approche par génération de texte (un petit modèle de langage, GPT-2) pour produire des données "narratives" comme de faux messages de support client. Le script propose aussi des outils pour vérifier que les données générées sont valides et statistiquement fidèles à l'original, ce qui est essentiel : des données synthétiques de mauvaise qualité seraient trompeuses.

21-SDV-synthetise/data_synthesizer.py

Script en ligne de commande qui génère des données synthétiques (tabulaires et narratives) à partir d'un dataset réel avec la bibliothèque SDV, puis propose des rapports et visualisations pour en évaluer la qualité.

Etape 1 — Préparation de l'environnement et chargement des données réelles

Lignes 1-36

```
import sys

# La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas
# les emojis utilisés dans les messages de ce script (ex: ) – ça faisait
# planter le script juste après avoir sauvegardé les résultats.
for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import pandas as pd
import warnings
```

```

import os

# transformers essaie de charger TensorFlow en plus de PyTorch, et la version de
# TensorFlow installée sur cette machine est incompatible avec la version de
# protobuf présente. On n'a besoin que du backend PyTorch pour le pipeline GPT-2.
os.environ.setdefault("USE_TF", "0")

from sdv.metadata import Metadata
from sdv.single_table import GaussianCopulaSynthesizer
from sdv.evaluation.single_table import run_diagnostic, evaluate_quality, get_column_plot
from transformers import pipeline

# Ignorer les messages d'avertissement pour garder une sortie propre
warnings.filterwarnings("ignore")

# Dossier du script : les chemins sont résolus par rapport à lui, pas au
# répertoire courant, pour que le script fonctionne peu importe d'où il est lancé.
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "Data")

# Créer un dossier nommé "Data" s'il n'existe pas déjà
os.makedirs(DATA_DIR, exist_ok=True)

# Charger le dataset original (premières 500 lignes pour la démo)
real_data = pd.read_csv(os.path.join(DATA_DIR, "E-Commerce Dataset.csv"), nrows=500)

```

Avant même de générer quoi que ce soit, cette étape règle deux problèmes d'environnement concrets (encodage de la console Windows incompatible avec les emojis, et conflit de versions entre TensorFlow et protobuf pour la bibliothèque `transformers`) puis charge les briques nécessaires : SDV pour la synthèse tabulaire, `transformers` pour la génération de texte. Les chemins sont calculés par rapport à l'emplacement du script (pas au répertoire courant), pour que le script fonctionne peu importe d'où il est lancé. Seules les 500 premières lignes du CSV réel sont chargées, un compromis volontaire pour accélérer la démonstration.

Etape 2 — Génération de données synthétiques tabulaires

Lignes 38-66

```

# Initialiser les variables pour stocker les données et résultats
metadata = None
synthetic_data = None
quality_report_result = None

def synthesize_data(realism=False):
    """
    Générer des données synthétiques basées sur le dataset original

    Args:
        realism (bool): Si True, active le mode réalisme pour plus de variété
    """
    global metadata, synthetic_data

    # Détecter automatiquement la structure et les types depuis le dataset original
    metadata = Metadata.detect_from_dataframe(real_data)

    # Initialiser le synthétiseur avec les options sélectionnées
    synthesizer = GaussianCopulaSynthesizer(metadata)

    # Entraîner le synthétiseur sur le dataset réel
    synthesizer.fit(real_data)

    # Générer des échantillons synthétiques
    synthetic_data = synthesizer.sample(num_rows=500)

    # Sauvegarder les données générées dans un fichier CSV
    synthetic_data.to_csv(os.path.join(DATA_DIR, "synthetic_data.csv"), index=False)
    print("\n Données synthétiques sauvegardées dans Data/synthetic_data.csv")

```

C'est le cœur technique du projet. `Metadata.detect_from_dataframe`` détecte automatiquement le type de chaque colonne (numérique, catégoriel...) sans que le développeur ait à les décrire à la main. Le `GaussianCopulaSynthesizer`` est ensuite entraîné (`fit``) sur les vraies données : il apprend la distribution statistique de chaque colonne et les corrélations entre elles. `sample(num_rows=500)`` génère ensuite 500 lignes entièrement nouvelles, tirées du modèle statistique appris — aucune ligne réelle n'est copiée, ce qui est précisément ce qui rend cette approche utile pour protéger des données sensibles tout en conservant leurs tendances statistiques.

Etape 3 — Génération de données narratives avec GPT-2

Lignes 68-90

```
def generate_narrative_data():
    """
    Générer des données narratives synthétiques en utilisant GPT-2
    """
    global narrative_data

    # Créer un pipeline de génération de texte
    generator = pipeline("text-generation", model="gpt2")

    # Générer des messages synthétiques
    prompts = ["Message support client:" for _ in range(10)]
    messages = [
        generator(prompt, max_new_tokens=40, truncation=True)[0]["generated_text"].replace(prompt,
        "").strip()
        for prompt in prompts
    ]

    # Créer et sauvegarder le DataFrame
    narrative_data = pd.DataFrame({
        "id": [f"MSG-{i+1}" for i in range(len(messages))],
        "message": messages
    })
    narrative_data.to_csv(os.path.join(DATA_DIR, "narrative_data.csv"), index=False)
    print("\n Données narratives synthétiques sauvegardées dans Data/narrative_data.csv")
```

Cette fonction illustre l'autre grande famille de données synthétiques : le texte libre. Contrairement à SDV qui modélise des colonnes structurées, on utilise ici GPT-2 (un petit modèle de langage) via un pipeline `transformers`` pour générer 10 messages fictifs de type "support client" à partir de la même amorce de texte répétée 10 fois. Le résultat est ensuite mis en forme dans un DataFrame avec des identifiants générés (`MSG-1``, `MSG-2``...) et sauvegardé, montrant qu'un même projet peut combiner génération statistique (SDV) et génération de langage (transformers) selon le type de champ à synthétiser.

Etape 4 — Évaluation de la validité et de la qualité des données générées

Lignes 92-141

```
def diagnostic_report():
    """
    Exécuter un diagnostic pour s'assurer que les données sont valides
    """
    diagnostic = run_diagnostic(
        real_data=real_data,
        synthetic_data=synthetic_data,
        metadata=metadata
    )
    print("\n Rapport de diagnostic généré")
    return diagnostic

def quality_report():
    """
    Mesurer la qualité des données ou la similarité statistique entre les vraies et fausses données
    """
    global quality_report_result
    quality_report_result = evaluate_quality(
        real_data,
        synthetic_data,
```

```

        metadata
    )
    print("\n Rapport de qualité généré")
    print(f"Score de qualité global: {quality_report_result.get_score():.3f}")

def column_plot():
    """
    Afficher les détails de qualité par colonne
    """
    global quality_report_result
    if quality_report_result is None:
        print("Rapport de qualité pas encore généré. Génération en cours...")
        quality_report()
    print("Colonnes classées de la meilleure à la moins bonne qualité:")
    print("-----")
    print(quality_report_result.get_details("Column Shapes"))

def visualize_data():
    """
    Visualiser la comparaison d'une colonne entre les données réelles et synthétiques
    """
    # Visualiser en comparant une colonne (PreferredOrderCat) des vraies données aux synthétiques
    fig = get_column_plot(
        real_data=real_data,
        synthetic_data=synthetic_data,
        column_name="PreferredOrderCat",
        metadata=metadata
    )
    fig.show()
    print("\n Graphique de comparaison affiché")

```

Générer des données synthétiques ne suffit pas : encore faut-il vérifier qu'elles sont fiables. ``run_diagnostic`` contrôle la validité structurelle basique (formats, plages de valeurs respectées). ``evaluate_quality`` va plus loin en calculant un score global de similarité statistique entre données réelles et synthétiques, puis ``column_plot`` permet d'inspecter ce score colonne par colonne pour repérer celles qui sont le moins bien reproduites. ``visualize_data`` complète le tout par un graphique comparant visuellement la distribution d'une colonne précise entre données réelles et synthétiques. Ces quatre fonctions forment la boîte à outils de validation, indispensable puisque des données synthétiques de mauvaise qualité seraient trompeuses plutôt qu'utiles.

Etape 5 — Menus interactifs en ligne de commande

[Lignes 143-208](#)

```

def post_synthesis_menu():
    """
    Menu d'analyse post-synthèse permettant d'évaluer les résultats
    """
    print("\nDonnées synthétisées avec succès, consultez le fichier 'Data/synthetic_data.csv'")
    print("""
Que voulez-vous faire ensuite ?
1. Voir le rapport de diagnostic
2. Voir le rapport de qualité
3. Voir le graphique des colonnes
4. Visualiser les données
5. Quitter
""")

    while True:
        choice = input("Tapez votre choix : ").strip()
        if choice == "1":
            diagnostic_report()
        elif choice == "2":
            quality_report()
        elif choice == "3":
            column_plot()
        elif choice == "4":
            visualize_data()

```

```

        elif choice == "5":
            print(" Fermeture du programme.")
            exit()
        else:
            print(" Entrée invalide. Veuillez entrer un nombre de 1 à 5.")

# Menu principal
if __name__ == "__main__":
    print("""
    Quel est l'objectif des données que vous voulez synthétiser (tapez le numéro de l'objectif) :
    1. Réplication de schéma
    2. Réalisme + variété
    3. Champs narratifs (Ex: avis, tickets de support)
    Vous voulez connaître la différence entre les objectifs ? Tapez 'aide'
    """)

    while True:
        user_input = input("Tapez votre choix : ")
        if user_input == "aide":
            print("""
            1. Réplication de schéma : S'assure que les données synthétiques correspondent à la structure
            (colonnes, types de données, contraintes) du dataset original.

            2. Réalisme + variété : Se concentre sur la création de données synthétiques réalistes et
            diversifiées, capturant les motifs et la variabilité observés dans les vraies données.

            3. Champs narratifs : Implique la génération de texte cohérent et humain pour des champs
            comme les avis ou tickets de support, nécessitant compréhension et génération du langage naturel.
            """)
            elif user_input == "1":
                synthesize_data()
                post_synthesis_menu()
            elif user_input == "2":
                synthesize_data(realism=True)
                post_synthesis_menu()
            elif user_input == "3":
                generate_narrative_data()
                # Note: Pas de menu post-synthèse pour les données narratives dans le code original
                print("\nDonnées narratives générées. Tapez 'quit' pour quitter ou recommencez.")
            else:
                print("Entrée invalide. Veuillez entrer 1, 2, 3 ou 'aide'.")
                continue

```

Le script se termine par une couche d'interaction en ligne de commande à deux niveaux. Le menu principal (`if \_\_name\_\_ == "\_\_main\_\_"`) laisse choisir l'objectif de la synthèse (réplication de schéma, réalisme, ou champs narratifs) avec une aide contextuelle si demandé. Une fois des données tabulaires générées, `post\_synthesis\_menu()` propose un second menu pour explorer les résultats via les outils d'évaluation vus à l'étape précédente. Ces boucles `while True` avec vérification de saisie forment un pattern classique de menu interactif robuste aux erreurs de frappe de l'utilisateur.

Projet 22 — 22-Cyber_Workflow

Reconnaissance + tests de sécurité passifs sur un domaine

LE BUT DE CE PROJET

Le but de ce projet est de montrer comment automatiser les premières étapes d'un audit de sécurité offensif sur un site web, de façon totalement passive et légale, en combinant des outils de reconnaissance classiques avec des agents IA qui décident intelligemment quoi faire ensuite.

Le problème que ça résout : avant de tester la sécurité d'un site (pentest), un expert commence toujours par une phase de reconnaissance pour comprendre à quoi il a affaire (quel type de site, quelles technologies, quels risques probables) avant de choisir les vérifications pertinentes à mener. Faire cela manuellement pour chaque cible est répétitif : il faut interroger le WHOIS, le DNS, les en-têtes HTTP, puis réfléchir aux tests appropriés, puis les exécuter un par un. Ce projet automatise ce raisonnement en deux temps : un premier agent IA lit les données de reconnaissance collectées automatiquement et propose une stratégie de test adaptée à ce type de site, puis un second agent IA lit cette stratégie et choisit lui-même, parmi neuf outils disponibles, lesquels sont pertinents à exécuter — sans jamais tenter d'exploitation active ou destructrice, uniquement de la lecture d'informations publiques.

Pourquoi ça compte : c'est une bonne illustration de l'utilisation d'un LLM comme "décideur outillé" (agentic tool use) : au lieu de coder en dur la logique "si c'est un site e-commerce, alors lancer tel test", on laisse le modèle raisonner sur le contexte et choisir dynamiquement les outils appropriés parmi une liste, avec obligation de justifier ses choix. Le projet illustre aussi le pattern "Workflow" d'agno, qui permet d'enchaîner explicitement en code Python plusieurs agents dans un ordre précis (contrairement à un Team où le leader décide lui-même de la coordination), tout en gardant un cadre éthique strict imposé à la fois par le prompt et par la nature même des outils fournis (aucun d'eux ne peut faire de dégâts, ils ne font que lire des informations déjà publiques).

22-Cyber_Workflow/OffCyberTest.py

Workflow en deux agents (reconnaissance puis test outillé) qui orchestre l'audit de sécurité passif d'un domaine cible avec le framework agno.

Etape 1 — Configuration et collecte brute des données de reconnaissance

Lignes 1-54

```
import sys

# La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas
# certains caractères utilisés dans le rendu rich (icônes, box-drawing).
for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import os
import whois
```

```

import requests
import dns.resolver
from textwrap import dedent
from typing import Iterator
from dotenv import load_dotenv
from agno.utils.log import logger
from agno.workflow import Workflow
from agno.agent import Agent, RunOutput as RunResponse
from agno.models.groq import Groq
from agno.utils.pprint import pprint_run_response
from tools import check_security_headers, check_common_directories, find_js_urls, check_robots_txt,
check_sitemap, check_env_exposure, check_server_header, check_http_redirect, check_github_mentions

load_dotenv()

# Le script original utilisait OpenAI (gpt-4o) ; remplacé par Groq pour
# réutiliser la clé GROQ_API_KEY déjà disponible.
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

def extract_domain(url: str) -> str:
    """Extract domain from a full URL."""
    return url.split("//")[-1].split("/")[0]

def get_whois(domain: str) -> str:
    try:
        w = whois.whois(domain)
        return "\n".join(f"{k}: {v}" for k, v in w.items())
    except Exception as e:
        return f"WHOIS lookup failed: {e}"

def get_dns_records(domain: str) -> str:
    try:
        answers = dns.resolver.resolve(domain, 'A')
        return "\n".join(rdata.address for rdata in answers)
    except Exception as e:
        return f"DNS lookup failed: {e}"

def get_http_headers(domain: str) -> str:
    url = f"https://{domain}"
    try:
        response = requests.head(url, timeout=5)
        headers = response.headers
        return "\n".join(f"{k}: {v}" for k, v in headers.items())
    except Exception as e:
        return f"HTTP header fetch failed: {e}"

```

Après les mêmes correctifs d'encodage console que dans le projet précédent, cette étape définit trois petites fonctions de collecte de données publiques : WHOIS (qui possède le domaine, depuis quand), DNS (adresses IP associées) et en-têtes HTTP (via une requête HEAD légère). Chacune est protégée par un `try/except` qui transforme toute erreur en simple message texte, pour que l'échec d'une seule source de données n'interrompe pas tout le workflow. Ces fonctions sont appelées directement en code Python classique (pas comme outils d'agent), car cette collecte initiale est volontairement déterministe et non laissée au jugement du LLM.

Etape 2 — Recon Agent — catégoriser la cible et recommander des tests

Lignes 57-83

```

class OffensiveCybersecurityTester(Workflow):
    recon_agent: Agent = Agent(
        name = 'Recon Agent',
        model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),
        description=dedent("""
            Gather detailed information about the target URL to understand its environment and potential
            vulnerabilities.
            This agent performs passive reconnaissance by collecting public data without interacting
            aggressively with the target.
            """),

```

```

instructions = dedent("""
    You are a Recon Agent. You will receive reconnaissance data about a target domain.

    Your tasks:
    1. Analyze the WHOIS, DNS, and any available metadata.
    2. Based on patterns (e.g. tech stack, DNS structure, WHOIS info), category the target:
       - Is it an e-commerce site, blog, SaaS app, API backend, login portal, etc.?
    3. Identify notable or suspicious findings (e.g., short registration, strange DNS, tech
used).
    4. Based on the category, suggest specific types of tests that could be safely conducted
later.

    Example: If it's an e-commerce site, suggest testing cart, search, login, etc.

    Your output should include:
    - Target category
    - Key observations
    - List of recommended test types (with short reasons)

    Do not make up data. Only use what's in the input.
    """),
)

```

Le `Workflow` d'agno commence par déclarer ses agents comme attributs de classe (instanciés une seule fois, contrairement au projet 20 où des fonctions "usine" créaient un agent à chaque appel). Cet agent n'a aucun outil : il reçoit uniquement le texte de contexte (WHOIS/DNS/en-têtes) collecté à l'étape précédente et doit raisonner dessus pour catégoriser le site (e-commerce, blog, API...) et suggérer des types de tests pertinents. La consigne finale "Do not make up data" est une garde-fou anti-hallucination essentiel dans un contexte de sécurité, où une fausse déduction orienterait la suite du workflow vers des tests inutiles.

Etape 3 — Test Agent — sélectionner et exécuter les outils pertinents

[Lignes 85-124](#)

```

test_agent: Agent = Agent(
    name = 'Test Agent',
    model=Groq(id='openai/gpt-oss-120b', api_key=GROQ_API_KEY),
    description=dedent("""
        You are a Test Agent. You will receive an Agent Summary output of recommended tests to conduct
on the target domain.
        """),
    instructions = dedent("""
        You are the Tester Agent in an offensive cybersecurity workflow.

        You are equipped with the following safe and legal tools:

        1. check_security_headers - Inspect CSP, HSTS, and X-Frame headers.
        2. check_common_directories - Scan for /admin, /login, /config, etc.
        3. find_js_urls - List JS file URLs from the homepage.
        4. check_robots_txt - Look for robots.txt and list contents.
        5. check_sitemap - Look for sitemap.xml and print summary.
        6. check_env_exposure - Check for exposed .env file.
        7. check_server_header - Read the 'Server' HTTP header.
        8. check_http_redirect - Check if HTTP redirects to HTTPS.
        9. check_github_mentions - Provide Google search link for GitHub leaks.

        Your tasks:

        1. Read the Recon Agent's output (which includes recon analysis and test
recommendations).
        2. Based on those recommendations, select only the relevant tools to run.
        3. Perform safe, non-intrusive tests using only those selected tools.
        4. Return a structured response including:

        - Which tools were used and why (brief reasoning).
        - Which tools were skipped and why (e.g. "not relevant" or "not recommended").
        - A brief summary of findings from each test used.
        - A security risk score: Low / Medium / High (based on number and severity of issues found).
    """)
)

```

- (Optional) Suggest other safe checks to run later if helpful.

Rules:

- Never run tools unless explicitly recommended by the recon phase.
- Never test destructively. Only public, passive, and educational analysis is allowed.

```
"""),
tools=[check_security_headers, check_common_directories, find_js_urls, check_robots_txt,
check_sitemap, check_env_exposure, check_server_header, check_http_redirect, check_github_mentions],
)
```

Contrairement au Recon Agent, celui-ci reçoit les neuf fonctions de `tools.py` comme véritables outils exécutables (tool use). Ses instructions documentent chaque outil en langage naturel pour que le modèle sache quand l'utiliser, puis lui demandent de choisir lui-même, en fonction des recommandations reçues du Recon Agent, quels outils exécuter et lesquels ignorer — avec justification obligatoire des deux côtés. Les règles finales imposent un cadre éthique strict : ne jamais agir sans recommandation de la phase de reconnaissance, et ne jamais tester de façon destructrice, ce qui est cohérent avec le fait que tous les outils fournis ne font que lire des informations déjà publiques.

Etape 4 — Enchaînement du workflow : collecte, reconnaissance, puis test

[Lignes 126-149](#)

```
def run(self, domain: str) -> Iterator[RunResponse]:
    domain = extract_domain(domain)
    whois_info = get_whois(domain)
    dns_records = get_dns_records(domain)
    http_headers = get_http_headers(domain)
    full_context = f"""
Domain: {domain}
WHOIS info:
{whois_info}
DNS records:
{dns_records}
HTTP headers:
{http_headers}
"""
    logger.info(f'Running Recon for domain: {domain}')
    recon_response: Iterator[RunResponse] = self.recon_agent.run(full_context)
    if recon_response is None or not recon_response.content:
        yield RunResponse(
            run_id=self.run_id,
            content="Sorry, could not get the Recon analysis report.",
        )
    return
    logger.info('Testing the target domain')
    yield from self.test_agent.run(recon_response.content, stream=True)
```

C'est ici que le workflow prend tout son sens : contrairement à un `Team` où un leader IA déciderait lui-même de la coordination, ici l'enchaînement est écrit explicitement en code Python, étape par étape. Le domaine est d'abord nettoyé, puis les trois sources de données publiques sont collectées et assemblées en un texte de contexte unique envoyé au `recon\_agent`. Si la reconnaissance échoue (pas de réponse), le workflow s'arrête proprement avec un message d'erreur plutôt que de planter. Sinon, la sortie du `recon\_agent` est transmise telle quelle au `test\_agent`, qui va alors choisir et exécuter ses outils — le mot-clé `yield` (la méthode est un générateur) permet de streamer la réponse finale au fur et à mesure qu'elle est produite, plutôt que d'attendre que tout soit fini.

Etape 5 — Point d'entrée en ligne de commande

[Lignes 151-157](#)

```
if __name__ == '__main__':
    from rich.prompt import Prompt

    workflow = OffensiveCybersecurityTester()
    domain = Prompt.ask("Enter the domain to test")
    response: Iterator[RunResponse] = workflow.run(domain=domain)
```

```
pprint_run_response(response, markdown=True)
```

Le point d'entrée du script instancie le workflow, demande interactivement le domaine cible à l'utilisateur via la bibliothèque ``rich`` (pour un prompt terminal plus soigné), lance ``workflow.run(...)`` puis affiche la réponse streamée avec mise en forme markdown grâce à ``pprint_run_response``, l'utilitaire d'affichage d'agno prévu justement pour ce genre de flux de réponses progressives.

22-Cyber_Workflow/tools.py

Bibliothèque de fonctions de vérification de sécurité passives (headers, répertoires, fichiers sensibles...) fournies comme outils au Test Agent du workflow.

Etape 1 — Vérifications réseau de base : redirection HTTPS et mentions GitHub

[Lignes 1-24](#)

```
import requests
from bs4 import BeautifulSoup
import ssl
import socket
from datetime import datetime

def check_http_redirect(domain: str) -> str:
    """Vérifie si HTTP redirige vers HTTPS"""
    try:
        r = requests.get(f"http://{domain}", timeout=5, allow_redirects=True)
        if r.url.startswith("https://"):
            return " HTTP redirects to HTTPS."
        else:
            return " HTTP does not redirect securely. Final URL: {r.url}"
    except Exception as e:
        return f"Redirect check failed: {e}"

def check_github_mentions(domain: str) -> str:
    """Recherche les mentions GitHub du domaine"""
    try:
        query = f'"{domain}" site:github.com'
        return f" Search manually for GitHub mentions:\\nhttps://www.google.com/search?q={query.replace(' ', '+')}"
    except Exception as e:
        return f"GitHub mention check failed: {e}"
```

Ces deux fonctions ouvrent le catalogue d'outils. ``check_http_redirect`` vérifie une bonne pratique de sécurité de base : que le trafic non chiffré (HTTP) redirige bien vers la version chiffrée (HTTPS). ``check_github_mentions`` ne fait pas de requête directe (ce qui serait un scraping potentiellement fragile) mais génère simplement un lien de recherche Google prêt à l'emploi pour qu'un humain vérifie manuellement si des identifiants ou secrets du domaine ont fuité sur GitHub — un choix pragmatique qui reste dans l'esprit "passif" du projet.

Etape 2 — Fonctions non branchées au Test Agent (SSL, endpoints API, cookies)

[Lignes 26-75](#)

```
def check_ssl_certificate(domain: str) -> str:
    """Vérifie le certificat SSL"""
    try:
        context = ssl.create_default_context()
        with socket.create_connection((domain, 443), timeout=5) as sock:
            with context.wrap_socket(sock, server_hostname=domain) as ssock:
                cert = ssock.getpeercert()
                expiry = datetime.strptime(cert['notAfter'], "%b %d %H:%M:%S %Y %Z")
                days_left = (expiry - datetime.utcnow()).days
                return f" SSL Certificate is valid. Expires in {days_left} days."
    except Exception as e:
        return f"SSL certificate check failed: {e}"
```

```

def check_api_endpoints(domain: str) -> str:
    """Vérifie les endpoints API communs"""
    common_paths = ["/api", "/api/v1", "/graphql", "/swagger", "/admin", "/api-docs"]
    found = []

    for path in common_paths:
        url = f"https://{domain}{path}"
        try:
            r = requests.get(url, timeout=3)
            if r.status_code in [200, 403, 401]:
                found.append(f"{url} - Status {r.status_code}")
        except:
            continue

    if found:
        return " Possible API endpoints:\n" + "\n".join(found)
    else:
        return " No common API endpoints found."

def check_cookie_flags(domain: str) -> str:
    """Vérifie les flags de sécurité des cookies"""
    try:
        r = requests.get(f"https://{domain}", timeout=5)
        cookies = r.headers.get("Set-Cookie", "")
        issues = []

        if cookies:
            if "Secure" not in cookies:
                issues.append("Missing 'Secure'")
            if "HttpOnly" not in cookies:
                issues.append("Missing 'HttpOnly'")

        if issues:
            return " Cookie flags issue: " + ", ".join(issues)
        return " Cookies have Secure and HttpOnly flags."
    except Exception as e:
        return f"Cookie security check failed: {e}"

```

Ces trois fonctions existent dans le module mais ne font pas partie de la liste d'outils passée au `test\_agent` dans `OffCyberTest.py` : elles constituent une boîte à outils plus large que ce qui est réellement exposé à l'agent. `check\_ssl\_certificate` vérifie la validité et la date d'expiration du certificat HTTPS via une connexion socket bas niveau. `check\_api\_endpoints` sonde une liste de chemins d'API courants pour voir s'ils répondent. `check\_cookie\_flags` inspecte les cookies renvoyés par le serveur pour vérifier la présence des attributs de sécurité `Secure` et `HttpOnly`. Elles suivent toutes le même style défensif (try/except, retour texte préfixé d'un emoji indicateur) que le reste du fichier.

Etape 3 — Outils actifs du Test Agent : headers, répertoires, fichiers JS

Lignes 77-131

```

def check_security_headers(domain: str) -> str:
    """Vérifie les en-têtes de sécurité"""
    try:
        url = f"https://{domain}"
        response = requests.get(url, timeout=5)
        headers = response.headers

        results = []
        required = ['Content-Security-Policy', 'X-Frame-Options', 'Strict-Transport-Security']

        for h in required:
            if h in headers:
                results.append(f" {h} found")
            else:
                results.append(f" {h} missing")

        return "\n".join(results)

```

```

except Exception as e:
    return f"Header check failed: {e}"

def check_common_directories(domain: str) -> str:
    """Vérifie les répertoires communs"""
    url_base = f"https://{domain}"
    wordlist = ["/admin", "/login", "/config", "/.git", "/backup"]
    found = []

    for path in wordlist:
        try:
            full_url = url_base + path
            r = requests.get(full_url, timeout=3)
            if r.status_code in [200, 301, 403]:
                found.append(f" Found: {full_url} [{r.status_code}]")
        except:
            continue

    return "\\n".join(found) if found else "No common directories found."

def find_js_urls(domain: str) -> str:
    """Trouve les fichiers JavaScript"""
    try:
        url = f"https://{domain}"
        r = requests.get(url, timeout=5)
        soup = BeautifulSoup(r.text, "html.parser")
        js_urls = []

        for script in soup.find_all("script", src=True):
            src = script["src"]
            if src.startswith("http"):
                js_urls.append(src)
            else:
                js_urls.append(f"{url.rstrip('/')}/{src.lstrip('/')}")

        return "\\n".join(js_urls) if js_urls else "No JS files found."
    except Exception as e:
        return f"JS scan failed: {e}"

```

Trois des neuf outils réellement fournis au Test Agent. `check_security_headers`` vérifie la présence de trois en-têtes de sécurité HTTP standards (CSP, X-Frame-Options, HSTS). `check_common_directories`` teste une petite liste de chemins sensibles classiques (`/admin``, `/.git``...) en observant simplement les codes de statut HTTP retournés — c'est une reconnaissance passive au sens où elle ne fait qu'observer la réponse d'une requête GET normale, sans tenter d'exploiter quoi que ce soit. `find_js_urls`` télécharge la page d'accueil et utilise BeautifulSoup pour parser le HTML et lister les fichiers JavaScript chargés, une information utile pour deviner la stack technique du site.

Etape 4 — Outils actifs du Test Agent : robots.txt, sitemap, .env, header Server

Lignes 133-179

```

def check_robots_txt(domain: str) -> str:
    """Vérifie le fichier robots.txt"""
    try:
        url = f"https://{domain}/robots.txt"
        r = requests.get(url, timeout=3)
        if r.status_code == 200:
            return f" robots.txt found:\\n{r.text[:500]}"
        else:
            return f" robots.txt not found (status {r.status_code})"
    except Exception as e:
        return f"robots.txt check failed: {e}"

def check_sitemap(domain: str) -> str:
    """Vérifie le fichier sitemap.xml"""
    try:
        url = f"https://{domain}/sitemap.xml"

```

```

        r = requests.get(url, timeout=3)
        if r.status_code == 200:
            return f" sitemap.xml found:\\n{r.text[:500]}"
        else:
            return f" sitemap.xml not found (status {r.status_code})"
    except Exception as e:
        return f"sitemap check failed: {e}"

def check_env_exposure(domain: str) -> str:
    """Vérifie l'exposition du fichier .env"""
    try:
        url = f"https://{domain}/.env"
        r = requests.get(url, timeout=3)
        if "APP_KEY" in r.text or "DB_PASSWORD" in r.text:
            return f" Exposed .env file found:\\n{r.text[:300]}"
        elif r.status_code == 200:
            return " .env file exists but no sensitive data detected."
        else:
            return " .env file not accessible."
    except Exception as e:
        return f".env check failed: {e}"

def check_server_header(domain: str) -> str:
    """Vérifie l'en-tête Server"""
    try:
        url = f"https://{domain}"
        r = requests.get(url, timeout=5)
        server = r.headers.get("Server", "Not found")
        return f" Server header: {server}"
    except Exception as e:
        return f"Server header check failed: {e}"

```

Quatre autres outils fournis au Test Agent. `check\_robots\_txt` et `check\_sitemap` récupèrent et tronquent (500 premiers caractères) le contenu de ces deux fichiers standards, qui révèlent souvent la structure du site. `check\_env\_exposure` est le test le plus sensible du lot : il vérifie qu'un fichier `.env` (qui contient normalement des secrets comme des mots de passe de base de données) n'est pas accidentellement exposé publiquement, en cherchant des motifs caractéristiques (`APP\_KEY`, `DB\_PASSWORD`) dans la réponse. `check\_server\_header` lit simplement l'en-tête `Server`, qui peut révéler la technologie serveur utilisée.

Etape 5 — Scan séquentiel autonome et point d'entrée du module

Lignes 181-210

```

# Fonction principale pour exécuter tous les tests
def run_security_scan(domain: str):
    """Exécute un scan de sécurité complet"""
    print(f"=== Security Scan for {domain} ===\\n")

    checks = [
        ("HTTP to HTTPS Redirect", check_http_redirect),
        ("SSL Certificate", check_ssl_certificate),
        ("Security Headers", check_security_headers),
        ("Cookie Security", check_cookie_flags),
        ("API Endpoints", check_api_endpoints),
        ("Common Directories", check_common_directories),
        ("JavaScript Files", find_js_urls),
        ("robots.txt", check_robots_txt),
        ("sitemap.xml", check_sitemap),
        ("Environment File", check_env_exposure),
        ("Server Header", check_server_header),
        ("GitHub Mentions", check_github_mentions)
    ]

    for check_name, check_function in checks:
        print(f"--- {check_name} ---")
        result = check_function(domain)
        print(result)

```

```
print()

# Exemple d'utilisation
if __name__ == "__main__":
    domain = input("Enter domain to scan: ")
    run_security_scan(domain)
```

Cette dernière fonction offre un mode d'utilisation alternatif du module, indépendant du workflow IA : `run\_security\_scan` exécute systématiquement les douze fonctions de vérification (y compris celles non exposées à l'agent, comme SSL ou cookies) dans un ordre fixe, sans aucune sélection intelligente. C'est une façon simple et déterministe d'utiliser la même boîte à outils "à la main", en ligne de commande, sans passer par le raisonnement du Test Agent — utile pour tester rapidement `tools.py` isolément ou pour un usage où l'on veut systématiquement tout vérifier plutôt que de laisser un LLM choisir.

Projet 23 — 23-Whatsapp-lette

Bot WhatsApp relie a un agent Letta

LE BUT DE CE PROJET

Le but est de montrer comment brancher un agent conversationnel IA (qui a une mémoire et une personnalité) directement sur WhatsApp, pour qu'un utilisateur puisse lui parler comme à un contact normal, en texte ou en envoyant une photo. Le problème concret que ça résout : WhatsApp ne permet pas de "brancher" un chatbot n'importe comment, il faut passer par un webhook officiel de Meta qui impose des règles strictes (vérification d'URL par un jeton secret, réponse HTTP rapide sous peine que Meta considère l'appel en échec et le renvoie en double).

Le projet illustre donc l'architecture typique d'une intégration de bot messagerie : un serveur web (FastAPI) qui reçoit les événements de Meta, un traitement en arrière-plan (thread séparé) pour ne pas bloquer la réponse au webhook pendant que l'IA réfléchit, et un agent Letta, une plateforme d'agents "avec état" qui se souvient de la conversation d'un utilisateur d'une session à l'autre, contrairement à un simple appel ponctuel à un modèle de langage sans mémoire. Ça montre aussi comment gérer les images : WhatsApp ne transmet jamais le fichier directement, il faut aller le télécharger via un lien temporaire, l'encoder en base64, puis le transmettre à l'agent avec la légende associée.

23-Whatsapp-lette/CompanionAgent/main.py

Serveur FastAPI qui expose les routes webhook WhatsApp (vérification et réception de messages) et des endpoints de test pour l'envoi de messages via l'API Cloud WhatsApp de Meta.

Etape 1 — Imports et configuration

Lignes 1-20

```
import os
import json
import uvicorn
import logging
import requests
from threading import Thread
from dotenv import load_dotenv
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import PlainTextResponse, JSONResponse
from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message

load_dotenv()

VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')
WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')
VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')

app = FastAPI()
```

Ce bloc pose les fondations du serveur. os et dotenv permettent de charger les secrets (jetons, identifiants) depuis un fichier .env plutôt que de les écrire en dur dans le code, une pratique de sécurité de base. uvicorn est le serveur ASGI qui fera réellement tourner l'application FastAPI, capable de gérer les routes asynchrones (async def) utilisées

plus loin. Thread est importé en prévision du traitement en arrière-plan des messages, un point clé pour respecter les contraintes de Meta. Enfin, les fonctions de validation et de traitement sont importées depuis `whatsapp_utils.py` : séparer cette logique métier du fichier de routes garde chaque fichier focalisé sur une responsabilité claire. Les constantes lues via `os.getenv` sont les identifiants nécessaires à tout appel vers l'API WhatsApp Cloud de Meta.

Etape 2 — Route racine de contrôle

Lignes 22-24

```
@app.get("/")
def read_root():
    return {"message": "Welcome to the portal!"}
```

Cette route GET simple sert de "ping" : elle permet de vérifier rapidement, en visitant l'URL, que le serveur est bien démarré et répond, sans dépendre d'aucune configuration WhatsApp.

Etape 3 — Vérification du webhook (handshake Meta)

Lignes 26-42

```
# Required webhook verification for WhatsApp
async def verify(request: Request):
    params = dict(request.query_params)
    mode = params.get("hub.mode")
    token = params.get("hub.verify_token")
    challenge = params.get("hub.challenge")

    if mode and token:
        if mode == "subscribe" and token == VERIFY_TOKEN:
            logging.info("WEBHOOK_VERIFIED")
            return PlainTextResponse(content=challenge, status_code=200)
        else:
            logging.warning("VERIFICATION_FAILED")
            raise HTTPException(status_code=403, detail="Verification failed")
    else:
        logging.warning("MISSING_PARAMETER")
        raise HTTPException(status_code=400, detail="Missing parameters")
```

Avant d'envoyer le moindre événement réel, Meta appelle une fois cette route avec trois paramètres (`hub.mode`, `hub.verify_token`, `hub.challenge`) pour vérifier que le serveur qui répond est bien celui du développeur. La fonction compare le jeton reçu au secret `VERIFY_TOKEN` stocké côté serveur ; s'il correspond, elle renvoie la valeur `challenge` telle quelle en texte brut (obligatoire pour que Meta valide l'URL), sinon elle renvoie une erreur 403 ou 400. C'est un mécanisme de sécurité classique pour prouver la propriété d'une URL avant de commencer à y recevoir des données sensibles.

Etape 4 — Réception et traitement asynchrone des messages

Lignes 44-68

```
async def handle_message(request: Request):
    try:
        body = await request.json()
    except json.JSONDecodeError:
        logging.error("Failed to decode JSON")
        return JSONResponse(status_code=400, content={"status": "error", "message": "Invalid JSON provided"})

    if (
        body.get("entry", [{}])[0]
        .get("changes", [{}])[0]
        .get("value", {})
        .get("statuses")
    ):
        logging.info("Received a WhatsApp status update.")
        return JSONResponse(content={"status": "ok"})

    if is_valid_whatsapp_message(body):
```

```

    try:
        # Run message processing in a background thread
        Thread(target=process_whatsapp_message, args=(body,)).start()
    except Exception as e:
        logging.error(f"Error starting background message thread: {e}")
        return JsonResponse(content={"status": "ok"})
    else:
        return JsonResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})

```

Cette fonction gère chaque notification réelle envoyée par Meta une fois le webhook validé. Elle commence par sécuriser le décodage JSON, puis filtre les événements qui ne sont que des mises à jour de statut (accusé d'envoi/lecture) car ils ne nécessitent pas de réponse de l'agent. Pour un vrai message entrant valide, le point clé est de lancer `process_whatsapp_message` dans un Thread séparé plutôt que de l'appeler directement : Meta exige une réponse HTTP rapide sous peine de considérer l'appel en échec et de le renvoyer en double, alors que l'appel à l'agent IA peut prendre plusieurs secondes. Le serveur répond donc "ok" immédiatement pendant que le vrai travail continue en tâche de fond.

Etape 5 — Routes de test d'envoi de message

[Lignes 72-109](#)

```

@app.get("/test")
def test():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'template',
        'template': {
            'name': 'hello_world',
            'language': {
                'code': 'en_US'
            }
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Test successful!", "response": response.json()}

@app.get("/send_message")
def send_message():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'text',
        'text': {
            'body': 'Hello, how are you?'
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Message sent successfully!", "response": response.json()}

```

Ces deux routes GET (`/test` et `/send_message`) permettent de vérifier manuellement, pendant le développement, que la connexion à l'API WhatsApp fonctionne, sans passer par tout le cycle webhook. `/test` envoie un message "template" pré-approuvé par Meta (obligatoire pour initier une conversation vers un utilisateur qui n'a pas encore écrit), tandis que `/send_message` envoie un message texte libre. Les deux construisent la même URL d'API Graph et les mêmes en-têtes d'autorisation, ce qui illustre le format standard attendu par l'API WhatsApp Cloud pour tout

envoi de message.

Etape 6 — Déclaration des routes webhook et démarrage du serveur

Lignes 111-120

```
@app.get("/webhook")
async def webhook_get(request: Request):
    return await verify(request)

@app.post("/webhook")
async def webhook_post(request: Request):
    return await handle_message(request)

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Ce bloc relie les fonctions `verify()` et `handle_message()` aux vraies routes HTTP que Meta appellera : un GET pour la vérification initiale, un POST pour chaque événement suivant. Séparer la déclaration de route de la logique métier garde le code lisible. La dernière partie démarre le serveur uvicorn avec `host="0.0.0.0"`, ce qui le rend accessible depuis l'extérieur (par exemple via un tunnel comme ngrok), condition indispensable pour que les serveurs de Meta puissent effectivement atteindre ce webhook.

23-Whatsapp-lette/CompanionAgent/agent_service.py

Module qui envoie les messages (texte ou image) de l'utilisateur à l'agent Letta et récupère sa réponse.

Etape 1 — Imports, configuration et client Letta

Lignes 1-15

```
import os
import base64
from letta_client import Letta
from dotenv import load_dotenv

load_dotenv()

LETTA_TOKEN = os.getenv('LETTA_API_KEY')
AGENT_ID = os.getenv('AGENT_ID')

client = Letta(
    token=LETTA_TOKEN,
    # timeout=15,
)
```

Ce bloc initialise la connexion à Letta, une plateforme d'agents conversationnels "avec état" : contrairement à un simple appel à un modèle de langage sans mémoire, un agent Letta conserve son historique et sa mémoire côté serveur entre les appels. `base64` est importé en prévision de l'encodage d'images en texte, nécessaire car le JSON ne transporte que du texte. Le client est construit une seule fois avec le jeton d'authentification et sera réutilisé à chaque appel.

Etape 2 — Construction du message multimodal pour une image

Lignes 24-43

```
def get_response(message):
    message = message
    print('-----Agent triggered-----')
    if isinstance(message, dict):
        with open(message.get("image_path"), "rb") as image_file:
            encoded_string = base64.b64encode(image_file.read()).decode('utf-8')
        message = [
            {
                "type": "image",
```

```

        "source": {
            "type": "base64",
            "media_type": "image/jpeg",
            "data": encoded_string,
        },
    },
    {
        "type": "text",
        "text": message.get("caption")
    }
]

```

Quand le message reçu est un dictionnaire (convention utilisée pour signaler une image accompagnée d'une légende), la fonction ouvre le fichier en lecture binaire, l'encode en base64 (la seule façon de faire transiter des données binaires dans un payload JSON), puis reconstruit message sous forme de liste de blocs typés (image + texte). C'est le format de "contenu multimodal" attendu par l'API de l'agent, qui permet à un même message d'associer image et texte pour donner du contexte au modèle.

Etape 3 — Appel de l'agent et extraction de la réponse

Lignes 44-55

```

try:
    answer = client.agents.messages.create(
        agent_id=AGENT_ID,
        messages=[{"role": "user", "content": message}],
    )
    for msg in answer.messages:
        if hasattr(msg, "content") and msg.content:
            return msg.content
    return "Agent responded with no content."
except Exception as e:
    print("ERROR:", e)
    return "Sorry, I ran into an issue. Please try again! "

```

Le message (texte simple ou liste multimodale) est envoyé à l'agent Letta identifié par AGENT_ID, avec le rôle "user" conforme aux conventions des API de chat. Comme un agent peut renvoyer plusieurs messages internes, la boucle cherche le premier message porteur d'un contenu réel à retourner. Le bloc try/except protège tout l'appel réseau : en cas d'échec (quota, réseau, mauvaise configuration), un message d'erreur convivial est renvoyé à l'utilisateur plutôt que de laisser planter le flux WhatsApp.

Etape 4 — Point d'entrée de test manuel

Lignes 58-59

```

if __name__ == '__main__':
    print(get_response('hello'))

```

Ce bloc ne s'exécute que si le fichier est lancé directement (pas importé), et sert de test rapide en ligne de commande pour vérifier que la connexion à l'agent fonctionne, indépendamment de toute l'infrastructure WhatsApp.

23-Whatsapp-lette/CompanionAgent/whatsapp_utils.py

Fonctions utilitaires de validation, d'envoi et de traitement des messages WhatsApp entrants, y compris les images.

Etape 1 — Imports et configuration

Lignes 1-15

```

import requests
import json
import logging
import re
from fastapi.responses import JSONResponse

```

```

from dotenv import load_dotenv
from agent_service import get_response
import os
from time import sleep

```

```
load_dotenv()
```

```

WHATSAPP_ACCESS_TOKEN = os.getenv("WHATSAPP_ACCESS_TOKEN")
VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv("PHONE_NUMBER_ID")

```

Ce bloc importe les outils nécessaires pour appeler l'API WhatsApp (requests), manipuler du JSON, journaliser les erreurs, et surtout la fonction `get_response` depuis `agent_service.py` : c'est le pont entre la logique WhatsApp de ce fichier et la logique IA. Les constantes (token, version d'API, identifiant du numéro) sont relues depuis l'environnement, comme dans les autres fichiers du projet.

Etape 2 — Journalisation des réponses HTTP

[Lignes 17-20](#)

```

def log_http_response(response):
    logging.info(f"Status: {response.status_code}")
    logging.info(f"Content-type: {response.headers.get('content-type')}")
    logging.info(f"Body: {response.text}")

```

Cette petite fonction utilitaire centralise le format des logs de débogage réseau (code de statut, type de contenu, corps de la réponse), pour éviter de dupliquer ce code à chaque endroit où l'on appelle l'API WhatsApp.

Etape 3 — Construction du payload de message texte

[Lignes 23-35](#)

```

def get_text_message_input(recipient, text):
    return {
        "messaging_product": "whatsapp",
        "recipient_type": "individual",
        "to": recipient,
        "type": "text",
        "text": {"preview_url": False, "body": text},
    }

def generate_response(message):
    # Dummy response for now
    return message.upper()

```

`get_text_message_input` construit le dictionnaire exact attendu par l'API WhatsApp Cloud pour un message texte, en désactivant l'aperçu d'URL. Centraliser cette structure évite de la recopier à chaque endroit où un message est envoyé. `generate_response` est une fonction de réponse factice (retourne le texte en majuscules), vestige probable des tests avant l'intégration de l'agent Letta réel.

Etape 4 — Envoi du message avec gestion des erreurs réseau

[Lignes 38-55](#)

```

def send_message(data):
    headers = {
        "Content-type": "application/json",
        "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    }
    url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"

    try:
        response = requests.post(url, json=data, headers=headers, timeout=10)
        response.raise_for_status()
        log_http_response(response)
        return response

```

```
except requests.Timeout:
    logging.error("Timeout occurred while sending message")
    return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
except requests.RequestException as e:
    logging.error(f"Request failed due to: {e}")
    return JSONResponse({"status": "error", "message": "Failed to send message"}, status_code=500)
```

send_message centralise l'appel HTTP POST vers l'API WhatsApp avec un timeout explicite de 10 secondes, pour éviter qu'une requête ne reste bloquée indéfiniment si le serveur distant ne répond jamais. raise_for_status() transforme un code d'erreur HTTP en exception Python gérable, et deux except distincts traitent séparément le cas d'un simple timeout (408) et celui de toute autre erreur réseau (500), renvoyant à chaque fois une réponse JSON d'erreur structurée plutôt que de laisser planter le programme.

Etape 5 — Validation défensive du message entrant

Lignes 58-66

```
def is_valid_whatsapp_message(body: dict):
    return (
        body.get("object")
        and body.get("entry")
        and body["entry"][0].get("changes")
        and body["entry"][0]["changes"][0].get("value")
        and body["entry"][0]["changes"][0]["value"].get("messages")
        and body["entry"][0]["changes"][0]["value"]["messages"][0]
    )
```

Cette fonction vérifie, étape par étape via une chaîne de "and" qui s'arrête dès qu'une condition échoue, que le corps reçu correspond bien à la structure imbriquée d'un vrai message WhatsApp entrant. Elle sert de filtre avant tout traitement, pour écarter les payloads incomplets ou d'un autre type (comme les mises à jour de statut) sans faire planter le serveur avec une KeyError.

Etape 6 — Extraction du message et téléchargement d'une éventuelle image

Lignes 69-91

```
def process_whatsapp_message(body: dict):
    value = body["entry"][0]["changes"][0]["value"]
    wa_id = value["contacts"][0]["wa_id"]
    name = value["contacts"][0]["profile"]["name"]
    message = value["messages"][0]
    if message["type"] == "text":
        message_body = message["text"]["body"]
    elif message["type"] == "image":
        message_body = message["image"]
        id = message_body.get("id")
        headers = {
            "Content-type": "application/json",
            "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
        }
        metadata_res = requests.get(f"https://graph.facebook.com/{VERSION}/{id}", headers=headers)
        image_url = metadata_res.json().get("url")
        image_res = requests.get(image_url, headers={"Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}"})
        with open(f"images/{id}.jpg", "wb") as f:
            f.write(image_res.content)
        if message_body.get("caption"):
            message_body = {'image_path': f'images/{id}.jpg', 'caption': message_body.get("caption")}
        else:
            message_body = {'image_path': f'images/{id}.jpg', 'caption': ''}
```

process_whatsapp_message extrait d'abord l'identifiant et le nom de l'expéditeur, puis distingue le type de message. Pour un texte, le corps est extrait directement. Pour une image, WhatsApp ne transmet jamais le fichier binaire dans le webhook : il faut d'abord récupérer ses métadonnées (dont une URL temporaire) via un premier appel GET, puis télécharger réellement les octets de l'image via un second appel, avant de la sauvegarder sur le disque local. Le message est ensuite reformaté en dictionnaire {image_path, caption}, exactement le format attendu

par `get_response` dans `agent_service.py` pour reconnaître un message image.

Etape 7 — Appel de l'agent et envoi de la réponse

Lignes 93-95

```
response_text = get_response(message_body)
data = get_text_message_input(wa_id, response_text)
send_message(data)
```

Ces trois lignes bouclent le cycle complet du webhook : le contenu extrait (texte ou image+légende) est envoyé à l'agent Letta via `get_response`, sa réponse est reformatée au format attendu par l'API WhatsApp via `get_text_message_input`, puis envoyée au contact via `send_message`. C'est le point de sortie de tout le pipeline déclenché depuis `main.py`.

Projet 24 — 24-ResearchAnalyst

Reproduit un rapport pour un autre pays à partir d'un PDF source

LE BUT DE CE PROJET

Le but est d'automatiser une tâche de recherche répétitive : produire, à partir d'un rapport PDF existant sur un pays, un rapport équivalent sur un autre pays en gardant exactement la même structure. Le problème réel que ça résout : un analyste qui doit dupliquer un rapport pour plusieurs pays différents passerait des heures à relire chaque section pour comprendre le plan du document original avant de refaire toute la recherche pour chaque nouveau pays.

Le projet montre comment enchaîner plusieurs étapes automatiquement (un "workflow") avec deux agents IA spécialisés qui se passent le relais : le premier lit le texte brut extrait par OCR (reconnaissance optique de caractères, qui convertit une image de PDF scanné en texte exploitable) et en dégage la structure et les points clés ; le second reprend cette structure telle quelle, mais va chercher sur le web des informations réelles et à jour sur le nouveau pays demandé, pour remplir chaque section avec des données pertinentes. L'ensemble tourne dans une interface web simple (Streamlit) qui affiche le rapport final au fur et à mesure qu'il est généré, comme dans un chat IA.

24-ResearchAnalyst/WorkflowAgent.py

Définit le workflow à deux agents qui effectue l'OCR d'un PDF, en analyse la structure, puis génère un rapport équivalent pour un autre pays.

Etape 1 — Imports

Lignes 1-10

```
import os
from textwrap import dedent
from typing import Iterator
from mistralai.client import Mistral
from agno.workflow import Workflow
from agno.models.groq import Groq
from agno.agent import Agent, RunOutput as RunResponse
from agno.utils.pprint import pprint_run_response
from agno.tools.duckduckgo import DuckDuckGoTools
from dotenv import load_dotenv
```

Ce bloc importe les briques nécessaires au pipeline : Mistral pour l'OCR (reconnaissance optique de caractères, qui convertit un PDF scanné en texte exploitable), et le framework agno pour construire un Workflow (orchestration de plusieurs agents en code Python classique) composé d'Agents utilisant un modèle hébergé par Groq, un fournisseur connu pour la rapidité d'inférence. DuckDuckGoTools est également importé : c'est un "tool" que l'agent pourra invoquer lui-même pendant son raisonnement pour faire de vraies recherches web.

Etape 2 — Configuration et client Mistral

Lignes 12-18

```
load_dotenv()

mistral_key = os.getenv("MISTRAL_API_KEY")
# Le script original utilisait OpenAI (gpt-4o-mini) ; remplacé par Groq pour
# réutiliser la clé GROQ_API_KEY déjà disponible.
```

```
groq_api_key = os.getenv("GROQ_API_KEY")
client = Mistral(api_key=mistral_key)
```

Charge les clés API (Mistral pour l'OCR, Groq pour les modèles de langage) depuis l'environnement, et crée le client Mistral une seule fois. Le commentaire précise que le modèle OpenAI original a été remplacé par Groq pour réutiliser une clé déjà disponible dans le projet.

Etape 3 — Fonction ocr_pdf : extraction du texte du PDF

Lignes 20-48

```
# OCR the pdf and return the text
def ocr_pdf(pdf_path):
    # check if the file exists
    if not os.path.exists(pdf_path):
        return "PDF file not found"

    uploaded_pdf = client.files.upload(
        file={
            "file_name": pdf_path,
            "content": open(pdf_path, "rb"),
        },
        purpose="ocr"
    )

    signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = client.ocr.process(
        model="mistral-ocr-latest",
        document={
            "type": "document_url",
            "document_url": signed_url.url,
        },
        include_image_base64=True
    )

    with open("ocr_response.md", "w") as f:
        f.write("\n".join([page.markdown for page in ocr_response.pages]))

    return ocr_response.pages
```

Cette fonction convertit un PDF (potentiellement scanné, donc composé d'images de texte) en texte réellement exploitable. Le processus se fait en trois temps propres à l'API Mistral : téléverser le fichier, obtenir une URL signée temporaire qui le référence, puis lancer le traitement OCR sur cette URL avec le modèle dédié mistral-ocr-latest. Le résultat, fourni page par page déjà converti en markdown structuré, est à la fois sauvegardé dans un fichier local (ocr_response.md, relu par app.py) et retourné à l'appelant.

Etape 4 — Premier agent : analyse de structure du document

Lignes 50-85

```
# Main Agents Workflow
class ResearchAnalystWorkflow(Workflow):
    # Targeted report structure (Golden structure)
    structure_and_analysis_agent: Agent = Agent(
        name = 'Structure and Analysis Agent',
        model=Groq(id="openai/gpt-oss-120b", api_key=groq_api_key),
        instructions = dedent("""
            You are an intelligent document analysis agent. You are given the raw text of a PDF (OCR
            processed). Your task is to analyze the content and produce a structured summary of the document's
            subject, key points, and organization. The report discusses a specific country, which you must identify.
            Follow these steps:

            1. Detect and clearly state the COUNTRY being discussed (call this "Country X").
            2. Identify the MAIN TOPIC of the document (e.g. energy policy, health care reforms, education
            quality).
            3. Define the DOCUMENT STRUCTURE as a clean numbered outline with sections and subsections based
```

on the logical flow of the text.

4. Extract the KEY ENTITIES, KEY FIGURES (if any), and any important ORGANIZATIONS or AGENCIES mentioned.

5. Summarize the KEY POINTS and INSIGHTS under each major section, focusing on the data, trends, problems, or conclusions made about Country X.

Output format:

COUNTRY: <Country X>

TOPIC: <main subject of the document>

STRUCTURE:

1. <Section Title>

- <Summary or bullet points>

2. <Section Title>

- <Summary or bullet points>

...

KEY POINTS AND INSIGHTS:

- Bullet points summarizing major findings, arguments, statistics, etc.

- Must reflect what is emphasized in the document

- Keep this detailed but concise

This output will be used by a second agent to replicate a similar report for a different country. Be clear, complete, and well-structured.

"""),

)

La classe `ResearchAnalystWorkflow` hérite de `Workflow`, ce qui permet d'orchestrer plusieurs agents dans une méthode `run()` en Python classique. Le premier agent déclaré, `structure_and_analysis_agent`, a pour rôle de lire le texte OCR brut et d'en dégager une fiche structurée : le pays traité, le sujet principal, le plan du document, les entités clés et les points importants. Le prompt impose un format de sortie très précis (COUNTRY, TOPIC, STRUCTURE...) car cette sortie servira directement d'entrée au second agent : imposer un format strict est essentiel pour qu'une sortie de modèle de langage reste exploitable par un autre traitement automatique en aval.

Etape 5 — Méthode `run()` : exécution du premier agent et gestion de l'échec

Lignes 87-94

```
def run(self, ocr_text: str, user_country: str) -> Iterator[RunResponse]:
    analysis_response: Iterator[RunResponse] = self.structure_and_analysis_agent.run(ocr_text)
    if analysis_response is None or not analysis_response.content:
        yield RunResponse(
            run_id=self.run_id,
            content="Sorry, could not get the analysis report.",
        )
    return
```

La méthode `run()` du workflow prend le texte OCR et le pays cible demandé par l'utilisateur, et retourne un flux de réponses (Iterator, via `yield`, qui fait de cette méthode un générateur). Elle exécute d'abord l'agent d'analyse ; si celui-ci échoue à produire un contenu exploitable, le workflow s'arrête immédiatement avec un message d'erreur plutôt que de lancer inutilement le second agent sur une entrée vide.

Etape 6 — Second agent : recherche et génération du rapport pour le nouveau pays

Lignes 96-142

```
research_agent: Agent = Agent(
    name = 'Research Agent',
    model=Groq(id="openai/gpt-oss-120b", api_key=groq_api_key),
    instructions = dedent(f"""
        You are a research agent. You are given a structured summary of a document about a topic in a
        specific country. Your task is to generate a new report on the same topic but focused on COUNTRY:
        {user_country}.

        You may use the GoogleSearchTools() tool to search for real-world information related to Country
        {user_country}.
```

Instructions:

1. Read the input summary carefully. It contains:

- COUNTRY: <Country X>
- TOPIC: <Main Topic>
- STRUCTURE: <Numbered outline of the report>
- KEY POINTS AND INSIGHTS from the original document

2. Understand the structure and content focus. Your output must use **the same section headings** and address **the same themes**, but for **Country {user_country}**.

3. For each section, use `GoogleSearchTools()` to find reliable and up-to-date information specific to Country {user_country}. For example:

- Local policies
- Trends
- Statistics
- Institutional efforts
- Challenges and opportunities
- Relevant events

4. Rebuild the report using the exact structure, now filled with data and insights from Country {user_country}. The tone, depth, and organization should match the original as closely as possible.

Output format:

TITLE: <Same topic> in <Country {user_country}>

1. <Section Title from original>
 - <Findings related to Country {user_country}>
2. <Next Section>
 - <Continue same structure>

Cite or refer to sources found via search where helpful.

IMPORTANT: You must always stay exactly within the subject defined in the original document. Do not go broader, do not narrow the scope, and do not add or remove themes. Reproduce the same structure and content areas, adapted only to Country {user_country}. Nothing more, nothing less.

```
"""),
tools=[DuckDuckGoTools()],
# Les résultats bruts de web_search font gonfler le contexte au-delà de la
# limite Groq (8000 tokens/minute en tier gratuit) : on les compresse.
compress_tool_results=True,
)
yield from reasearch_agent.run(analysis_response.content, stream=True)
```

Ce second agent reçoit la fiche structurée produite par le premier et doit reconstruire un rapport équivalent, avec exactement la même structure de sections, mais rempli d'informations réelles sur le pays demandé par l'utilisateur (injecté dynamiquement dans le prompt via une f-string). L'outil `DuckDuckGoTools` lui permet d'aller chercher des données réelles et à jour sur le web plutôt que de se limiter à ses connaissances figées à l'entraînement. `compress_tool_results=True` compresse automatiquement les résultats de recherche bruts, car ils feraient sinon dépasser la limite de tokens par minute imposée par Groq en version gratuite. Enfin, `yield from` propage en flux (stream=True) la génération du rapport final vers l'appelant, ce qui permettra un affichage progressif côté interface.

24-ResearchAnalyst/app.py

Interface Streamlit permettant de téléverser un PDF, lancer son OCR puis afficher en flux le rapport généré par le workflow de recherche.

Etape 1 — Imports, initialisation et formulaire de saisie

Lignes 1-15

```
import streamlit as st
from typing import Iterator
from agno.agent import RunOutput as RunResponse
```

```

from WorkflowAgent import ResearchAnalystWorkflow, ocr_pdf

# Main function
if __name__ == "__main__":
    ocr_text = ""
    user_country = ""
    pdf_path = ""
    st.title("Research Analyst Multi-Agent System")
    with st.form("input_form"):
        pdf_path = st.file_uploader("Upload a PDF file", type=["pdf"])
        user_country = st.text_input("Enter your desired research country")
        submitted = st.form_submit_button("Submit")

```

Ce bloc importe Streamlit (le framework qui construit l'interface web sans HTML/CSS/JS) ainsi que la classe de workflow et la fonction OCR définies dans WorkflowAgent.py : cette interface ne fait qu'orchestrer visuellement un pipeline déjà défini ailleurs. Le formulaire (st.form) regroupe le champ de téléversement du PDF et le champ de saisie du pays cible, de façon à ce que Streamlit ne relance le script qu'une seule fois, au clic sur "Submit", plutôt qu'à chaque interaction.

Etape 2 — Sauvegarde du PDF et lancement de l'OCR

[Lignes 16-26](#)

```

if submitted and pdf_path and user_country:
    with st.spinner("Processing PDF..."):
        # save the pdf locally
        with open(f'UploadedPDF/{pdf_path.name}', "wb") as f:
            f.write(pdf_path.read())
        st.success("PDF file uploaded successfully")
        ocr_pdf(f'UploadedPDF/{pdf_path.name}')
        # read the ocr_response.md file
        with open("ocr_response.md", "r") as f:
            ocr_text = f.read()
        st.success("OCR completed successfully")

```

Une fois le formulaire soumis avec un fichier et un pays renseignés, le PDF téléversé (reçu en mémoire par Streamlit) est réécrit sur le disque local, car la fonction ocr_pdf du workflow attend un chemin de fichier réel. ocr_pdf est ensuite appelée, ce qui déclenche l'appel OCR à Mistral et écrit le résultat dans ocr_response.md ; ce fichier est aussitôt relu pour récupérer le texte OCR en mémoire pour l'étape suivante.

Etape 3 — Exécution du workflow et affichage en flux du rapport

[Lignes 27-38](#)

```

with st.spinner("Running research workflow..."):
    # run the workflow
    workflow = ResearchAnalystWorkflow()
    # Display a placeholder for streaming
    output_placeholder = st.empty()
    full_output = ""
    # Run the workflow and stream the output
    response: Iterator[RunResponse] = workflow.run(ocr_text=ocr_text, user_country=user_country)
    for run_response in response:
        if run_response.content:
            full_output += run_response.content
            output_placeholder.markdown(full_output)

```

Le workflow est instancié puis lancé avec le texte OCR et le pays choisi. Comme run() est un générateur, l'appel ne bloque pas : la boucle for parcourt chaque fragment de texte produit au fur et à mesure par le pipeline à deux agents, l'accumule, et réécrit à chaque itération le même conteneur Streamlit (st.empty) avec le texte complet accumulé jusque-là. Cela donne visuellement l'effet d'un texte qui s'écrit progressivement à l'écran, comme dans une interface de chat IA.

Etape 4 — Cas où le formulaire n'a pas été complété

Lignes 39-40

```
else:  
    st.write("No PDF file uploaded")
```

Si l'utilisateur n'a pas encore soumis le formulaire avec un PDF et un pays, l'application affiche simplement un message d'attente plutôt que de tenter un traitement sur des données manquantes.

Projet 25 — 25-PriceAndPositioning

Analyse concurrentielle de prix Amazon

LE BUT DE CE PROJET

Le but est d'automatiser une étude de marché pour aider à fixer le prix d'un nouveau produit. Le problème concret que ça résout : avant de lancer un produit, il faut savoir combien coûtent les concurrents déjà en vente et quels sont leurs points faibles (avis négatifs) pour se positionner intelligemment, un travail normalement long et manuel (chercher sur Amazon, lire des dizaines d'avis, comparer les prix).

Le projet montre un pipeline en deux temps typique du RAG (Retrieval-Augmented Generation, "génération augmentée par récupération") : d'abord on va chercher de vraies pages web de produits concurrents avec un moteur de recherche sémantique, on récupère leur contenu textuel, puis on demande à un modèle de langage de synthétiser cette matière brute en fiches produits structurées (nom, prix, avis négatifs), plutôt que de laisser le modèle inventer une réponse uniquement à partir de ce qu'il a mémorisé pendant son entraînement. Un second agent prend ensuite ces fiches concurrentes et en tire une vraie recommandation stratégique : dans quelle fourchette de prix se positionner et pourquoi, avec des actions concrètes pour se différencier des défauts constatés chez la concurrence. Le projet illustre aussi une bonne pratique de robustesse : si la recherche web échoue (clé API absente, service indisponible), le programme bascule automatiquement sur un mode de secours qui continue à fournir un résultat utile plutôt que de planter.

25-PriceAndPositioning/price_and_positioning.py

Workflow qui recherche des produits concurrents sur Amazon via Exa/OpenAI puis génère une recommandation de prix et de positionnement.

Etape 1 — Imports, configuration et initialisation des clients

Lignes 1-19

```
import os
from exa_py import Exa
from textwrap import dedent
from typing import Iterator
from dotenv import load_dotenv
from agno.workflow import Workflow
from agno.agent import Agent, RunResponse
from agno.models.openai import OpenAIChat
from agno.utils.pprint import pprint_run_response
from openai import OpenAI

load_dotenv()

openai_api_key = os.getenv("OPENAI_API_KEY")
exa_api_key = os.getenv("EXA_API_KEY")

# Initialiser les clients
exa = Exa(api_key=exa_api_key) if exa_api_key else None
openai_client = OpenAI(api_key=openai_api_key) if openai_api_key else None
```

Ce bloc importe Exa (moteur de recherche "neuronal"/sémantique pensé pour les applications d'IA, capable de retrouver des pages par le sens plutôt que par simple mot-clé), le SDK OpenAI standard pour un appel direct hors du framework agno, et agno pour le second agent du pipeline. Les clients Exa et OpenAI ne sont créés que si leur clé API respective est disponible (sinon None) : une écriture défensive qui évite un plantage immédiat au démarrage si une clé manque, l'erreur étant gérée plus tard au moment de l'usage réel.

Etape 2 — exa_search : recherche de produits concurrents et récupération du contenu

Lignes 21-40

```
def exa_search(search_term: str):
    """Fonction de recherche Exa """
    try:
        if not exa:
            raise Exception("Clé API Exa non configurée")

        # Utilisation de l'API Exa standard pour la recherche
        search_results = exa.search(
            query=f"Amazon {search_term} product reviews price",
            type="neural", # Recherche sémantique
            num_results=10,
            include_domains=["amazon.com", "amazon.fr"],
            use_autoprompt=True
        )

        # Obtenir le contenu des pages
        content_results = exa.get_contents(
            urls=[result.url for result in search_results.results[:5]],
            text=True
        )
```

Cette fonction lance une recherche sémantique via Exa ciblée sur Amazon (via include_domains) en combinant le terme produit avec des mots-clés orientés avis et prix. Comme la recherche ne renvoie que des métadonnées (titre, URL, score), un second appel get_contents va chercher le texte réel des cinq premières pages trouvées. C'est la première étape d'un pipeline "recherche puis synthèse" (une variante du principe RAG, Retrieval-Augmented Generation) : aller chercher de l'information externe et fraîche avant de la faire traiter par un modèle de langage.

Etape 3 — Mise en forme des résultats de recherche

Lignes 42-56

```
# Formater les résultats pour l'agent
formatted_content = f"""
Résultats de recherche pour "{search_term}":

"""

for i, (result, content) in enumerate(zip(search_results.results[:5], content_results.results)):
    formatted_content += f"""
    ## Produit {i+1}
    **Titre**: {result.title}
    **URL**: {result.url}
    **Score**: {result.score}
    **Contenu**: {content.text[:1000] if content.text else "Contenu non disponible"}...

    """
```

Les résultats bruts (titre, URL, score, contenu tronqué à 1000 caractères) sont assemblés en un texte markdown lisible, formatted_content, qui servira de contexte factuel à fournir au modèle de langage à l'étape suivante. La troncature du contenu limite volontairement la taille totale du texte envoyé ensuite.

Etape 4 — Synthèse des résultats via OpenAI

Lignes 58-85

```
# Utiliser OpenAI pour analyser les résultats Exa
if not openai_client:
```

```

        raise Exception("Clé API OpenAI non configurée")

    completion = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": """Tu es un expert en recherche de produits Amazon.
            Analyse les résultats de recherche fournis et extrait 3 produits pertinents avec:
            - Nom du produit
            - Prix (si disponible)
            - Description courte
            - Points négatifs mentionnés (ou crée des exemples réalistes d'avis négatifs basés sur le
type de produit)

            Format en markdown propre."""},
            {"role": "user", "content": f"""
            Analyse ces résultats de recherche pour "{search_term}" et extrait 3 produits Amazon
pertinents:

            {formatted_content}

            Si les prix ne sont pas visibles, estime des prix réalistes pour ce type de produit.
            Pour les avis négatifs, soit utilise ceux trouvés, soit crée des exemples réalistes basés
sur les problèmes courants de ce type de produit.
            """}
        ],
        temperature=0.7
    )

    return completion.choices[0].message.content

```

Cette étape transforme les résultats de recherche bruts en fiches produits exploitables, via un appel direct à l'API de chat completions d'OpenAI (gpt-4o-mini). Le message système fixe le rôle d'expert et la structure attendue (nom, prix, description, points négatifs), tandis que le message utilisateur injecte le contexte de recherche formaté à l'étape précédente : c'est le principe même du RAG, donner au modèle des données récupérées en direct plutôt que de compter uniquement sur sa mémoire d'entraînement. La consigne autorise explicitement le modèle à estimer des prix ou créer des exemples d'avis réalistes si les données réelles manquent.

Etape 5 — Mode de secours en cas d'échec de la recherche

[Lignes 87-114](#)

```

except Exception as e:
    print(f"Erreur lors de la recherche Exa: {e}")
    print(" Utilisation du mode fallback (OpenAI seulement)...")

    # Fallback : utiliser uniquement OpenAI avec des connaissances générales
    if not openai_client:
        return f" Erreur: Clé API OpenAI requise. Veuillez configurer OPENAI_API_KEY dans votre
fichier .env"

    completion = openai_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "user", "content": f"""
            Crée une analyse de marché fictive mais réaliste pour le produit suivant: {search_term}

            Inclus 3 produits concurrents avec:
            - Nom du produit
            - Prix estimé réaliste
            - Description courte
            - 3 avis clients négatifs réalistes pour chaque produit

            Base-toi sur ta connaissance des produits similaires sur Amazon.
            Format en markdown propre et détaillé.
            """}
        ],
        temperature=0.7
    )

```

```
)

return completion.choices[0].message.content
```

Si la recherche Exa échoue (clé absente, erreur réseau, quota...), le bloc except bascule vers un mode de secours qui interroge OpenAI seul, sans aucune donnée web réelle, en lui demandant de générer une analyse de marché fictive mais réaliste à partir de ses seules connaissances générales. C'est un choix de conception qui privilégie la disponibilité du service : plutôt que de faire échouer toute la fonctionnalité, on dégrade la qualité de la réponse mais on continue à fournir un résultat utile.

Etape 6 — Agent stratégie : instructions et format de sortie attendu

[Lignes 116-189](#)

```
class ProductPriceAndPositioning(Workflow):
    pricing_and_positioning_strategist: Agent = Agent(
        name='Pricing Strategist',
        model=OpenAIChat(id='gpt-4o-mini'),
        instructions=dedent("""
You are a pricing and product strategy expert.

You will receive structured market research data about 3 competitor products, including for each:
- Competitor name
- Price
- Product description summary
- 3 negative customer reviews highlighting product complaints

Your tasks:

1. Analyze the competitor prices and product positioning (budget, midrange, premium).
2. Pay special attention to the competitors' negative reviews to identify common product issues
and customer pain points.
3. Based on the competitive pricing landscape and product differentiation, propose a recommended
price range for your product:
- Classify it as Budget (< $30), Midrange ($30-$50), or Premium (> $50).
- Justify why this price range is appropriate, referencing competitors and market conditions.
4. Under Key Competitors, list each competitor with:
- Name and price
- A brief note on how it compares to your product
- Include the 3 bad reviews associated with that competitor, quoting customer complaints
5. Based on the negative reviews, suggest 2-4 tactical recommendations to improve your product
and stand out, such as product improvements, bundles, discounts, or unique positioning strategies. Tie
each recommendation directly to the competitor complaints you analyzed.
"""),
        expected_output=dedent("""
Return your output exactly in this markdown format:

---

Recommended Price Range

Range: `XX - YY`
Tier: _Budget / Midrange / Premium_

---

Key Competitors

- [Competitor Name] - `XX.XX`: _Short comparison note_
- Bad Reviews:
    1. "..."
    2. "..."
    3. "..."

- [Competitor Name] - `XX.XX`: _Short comparison note_
- Bad Reviews:
    1. "..."
```

```

        2. "..."  

        3. "..."  

  

        - _(Add more competitors if applicable)_  

  

        ---  

  

        ## Rationale  

  

        _Explain why this price range fits your product and market, referencing competitor prices,  

        positioning, and demand._  

  

        ---  

  

        ## Tactical Recommendations  

  

        - **[Recommendation 1]**: _Based on competitor complaints, e.g. "Improve pump mechanism to fix  

        frequent failures."_  

        - **[Recommendation 2]**: _E.g. "Offer a cleaning brush bundle addressing cleaning difficulties  

        customers mention."_  

        - _(Add more as relevant)_  

  

        ---  

  

        Keep your analysis specific, actionable, and tied closely to the competitor data and customer  

        feedback.  

        """),  

        markdown=True  

    )

```

La classe `ProductPriceAndPositioning` déclare un agent agno complet (contrairement à `exa_search` qui appelait l'API OpenAI directement) chargé de transformer les données de marché en recommandation stratégique. Les instructions détaillent précisément les tâches attendues (analyser les prix et positionnements, étudier les avis négatifs, proposer une fourchette de prix classée en trois catégories avec des seuils chiffrés, lister les concurrents avec leurs mauvais avis, proposer des recommandations tactiques). Le paramètre `expected_output`, séparé des instructions, impose un gabarit markdown exact (titres, emojis, structure) à reproduire fidèlement, ce qui renforce la conformité du format produit par le modèle.

Etape 7 — Méthode `run()` : enchaînement recherche puis stratégie

[Lignes 191-197](#)

```

def run(self, product_details: str):
    print(f" Recherche de données de marché pour: {product_details}")
    research_response = exa_search(product_details)
    print('=== DONNÉES DE RECHERCHE ===')
    print(research_response)
    print('=====')
    yield from self.pricing_and_positioning_strategist.run(research_response, stream=True)

```

La méthode `run()` du workflow appelle d'abord `exa_search` pour collecter les données de marché sur le produit demandé, les affiche dans la console à des fins de débogage, puis délègue à l'agent stratège la génération en flux (`stream=True`) de l'analyse finale à partir de ces données. C'est l'enchaînement séquentiel des deux étapes du pipeline : collecte puis synthèse stratégique.

Etape 8 — Point d'entrée en ligne de commande

[Lignes 199-206](#)

```

if __name__ == '__main__':

    from rich.prompt import Prompt
    product = Prompt.ask('Enter your product details')
    if product:
        workflow = ProductPriceAndPositioning()
        response: Iterator[RunResponse] = workflow.run(product_details=product)

```

```
pprint_run_response(response, markdown=True)
```

Ce bloc ne s'exécute que si le script est lancé directement. Il demande à l'utilisateur de décrire son produit via une invite de saisie stylisée (rich.Prompt), lance le workflow complet, puis affiche joliment dans le terminal le flux de réponses au format markdown, permettant de tester tout le pipeline sans interface graphique.

Projet 26 — 26-PlateDetection

Detection de plaques d'immatriculation + alerte SMS

LE BUT DE CE PROJET

Le but est de montrer comment automatiser une surveillance vidéo en temps réel : détecter automatiquement les plaques d'immatriculation visibles dans un flux vidéo, vérifier si le véhicule correspond à un enregistrement d'infraction connu, et alerter automatiquement par SMS si c'est le cas. Le problème concret que ça résout : surveiller manuellement des heures de vidéo pour repérer un véhicule recherché est fastidieux et sujet à l'erreur humaine.

Ce projet délègue la lecture des plaques à un pipeline de vision par ordinateur (qui détecte les véhicules, localise la plaque, puis lit le texte dessus, image par image), pendant qu'un agent IA se charge de transformer une donnée brute d'infraction (JSON) en message d'alerte lisible et cohérent, envoyé par SMS via un service de messagerie tiers. Le projet illustre bien la complémentarité entre deux types d'IA différents dans un même système : un modèle de vision spécialisé pour "voir" et lire les plaques, et un agent de langage pour formuler et déclencher l'action (l'envoi du SMS), avec des garde-fous stricts dans les instructions de l'agent pour l'empêcher d'inventer des données dans une alerte censée refléter des faits réels.

26-PlateDetection/plate_detect.py

Application Streamlit qui analyse une vidéo pour détecter des plaques d'immatriculation, les confronte à une base d'infractions et envoie une alerte SMS via un agent.

Etape 1 — Imports, configuration et clients (Roboflow, Twilio)

Lignes 1-26

```
import streamlit as st
import json
import os
import warnings
from textwrap import dedent
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from inference import InferencePipeline
from twilio.rest import Client

warnings.filterwarnings("ignore")

# Load environment
from dotenv import load_dotenv
load_dotenv()

# Global setup
roboflow_key = os.getenv('ROBOFLOW_API_KEY')
account_sid = os.getenv('TWILIO_ACCOUNT_SID')
auth_token = os.getenv('TWILIO_AUTH_TOKEN')
client = Client(account_sid, auth_token)

MAX_FRAMES = 60
PLATES_FOUND = []
DONE_PLATES = []
data = []
```

Ce bloc importe Streamlit pour l'interface, InferencePipeline du SDK Roboflow (plateforme de vision par ordinateur permettant d'exécuter localement un "workflow" de plusieurs modèles enchaînés sur un flux vidéo, image par image), et le client Twilio (service tiers d'envoi de SMS par programmation). Les avertissements sont désactivés pour garder une sortie propre malgré les bibliothèques de vision sous-jacentes. Les identifiants (clé Roboflow, identifiants Twilio) sont chargés depuis l'environnement, et des constantes globales sont initialisées : une limite de frames à traiter (MAX_FRAMES) et des listes pour suivre les plaques déjà rencontrées.

Etape 2 — Chargement de la base de données d'infractions

Lignes 28-35

```
# Load the violations database
try:
    with open('vehicle_violations.json', 'r') as file:
        data = json.load(file)
except FileNotFoundError:
    st.error(" 'vehicle_violations.json' not found.")
except json.JSONDecodeError:
    st.error(" JSON format error in 'vehicle_violations.json'.")
```

Ce bloc charge en mémoire, depuis un fichier JSON local, la liste des véhicules déjà associés à une infraction connue : c'est la base de comparaison contre laquelle chaque plaque détectée dans la vidéo sera confrontée. Les deux except gèrent proprement l'absence du fichier ou un JSON corrompu, en affichant une erreur dans l'interface plutôt que de faire planter l'application au démarrage.

Etape 3 — send_alert : fonction outil d'envoi de SMS

Lignes 37-45

```
# Send alert via Twilio
def send_alert(plate_info: str):
    st.info(f" Sending alert: {plate_info}")
    message = client.messages.create(
        from_=os.getenv('TWILIO_FROM_NUMBER'),
        to=os.getenv('TWILIO_TO_NUMBER'),
        body=plate_info
    )
    return message.sid
```

Cette fonction envoie un SMS via l'API Twilio à partir d'un texte d'alerte. Elle est conçue pour être utilisée comme "tool" (outil) par l'agent défini juste après, c'est-à-dire une fonction Python que le modèle de langage pourra décider d'appeler lui-même pendant son raisonnement (function calling), plutôt qu'être appelée directement depuis le code métier.

Etape 4 — Déclaration de l'agent d'alerte

Lignes 47-85

```
# Agent setup
agent = Agent(
    model=OpenAIChat(id='gpt-4o'),
    instructions=dedent("""
        You are a license plate alert system. You receive JSON input with full violation data for a
        detected vehicle.

        Your task is:
        1. Extract the actual values from the JSON input:
        - license_plate
        - crime
        - date
        - location
        - severity
        - fine_amount

        2. Format a single alert message exactly like this:
        " ALERT: Vehicle [license_plate] detected on [location] | VIOLATION RECORD: [crime] | Original
        incident: [date] | Severity: [severity] | Fine: £[fine_amount] | Vehicle with known violation history
```

```
spotted"
```

```
3. Use the `send_alert` tool to send this message as a string.
```

```
Return a JSON response in this format:
```

```
{
  "status": "success",
  "message": "Alert sent for license plate [license_plate]",
  "alert_sent": "[full_alert_message]"
}
```

```
If anything fails, return:
```

```
{
  "status": "failed",
  "message": "Failed to send alert",
  "error": "[error_details]"
}
```

```
IMPORTANT:
```

- Do NOT invent or guess any data. Only use what's provided in the input.
- NEVER use placeholder values like ABC123 or Speeding.

```
"""),
tools=[send_alert]
```

```
)
```

L'agent reçoit en entrée les données JSON complètes d'une infraction détectée et doit en extraire les champs utiles (plaque, infraction, date, lieu, gravité, amende) pour composer un message d'alerte selon un gabarit texte précis, puis appeler l'outil `send_alert` pour l'envoyer réellement. Les instructions imposent aussi un format de réponse JSON strict (succès ou échec) et interdisent explicitement d'inventer ou de deviner des données, une contrainte de prompt engineering cruciale ici car une alerte SMS censée refléter une infraction réelle ne doit jamais contenir de données fabriquées.

Etape 5 — `plate_detection` : recherche d'une infraction correspondant à une plaque

[Lignes 87-92](#)

```
# Lookup violation
def plate_detection(plate: str):
    for d in data:
        if plate == d['license_plate'] and plate not in DONE_PLATES:
            return d
    return False
```

Cette fonction parcourt la base de données chargée en mémoire pour trouver un enregistrement correspondant à la plaque détectée, en vérifiant aussi qu'elle n'a pas déjà été traitée (via `DONE_PLATES`), afin d'éviter de redéclencher une alerte pour un véhicule déjà signalé.

Etape 6 — `my_sink` : callback de traitement de chaque frame vidéo

[Lignes 94-124](#)

```
# Display + process detected plates
def my_sink(result, video_frame):
    if video_frame.frame_id == MAX_FRAMES:
        pipeline.terminate()
        return

    frame_plates = set()
    for group in result['open_ai']:
        for res in group:
            if res.get('output'):
                frame_plates.add(res['output'].strip())

    for plate in frame_plates:
        plate_info = plate_detection(plate)
        if plate_info:
            if plate_info['license_plate'] not in DONE_PLATES:
                DONE_PLATES.append(plate_info['license_plate'])
                st.success(f" Match found: {plate}")
```

```

agent_response = agent.run(json.dumps(plate_info))
final_agent_output = None
for msg in reversed(agent_response.messages):
    if msg.role == 'assistant' and msg.content:
        final_agent_output = msg.content
        break
if final_agent_output:
    try:
        st.json(json.loads(final_agent_output))
    except json.JSONDecodeError as e:
        st.error(f" Agent returned non-JSON content:\n{final_agent_output}")
else:
    st.warning(" No assistant response received.")

```

Cette fonction est le callback appelé automatiquement par InferencePipeline à chaque frame traité (le patron de conception "callback" typique des pipelines de flux vidéo, qui évite d'écrire une boucle manuelle sur les frames). Elle arrête le pipeline une fois la limite de frames atteinte, extrait les textes de plaques détectés dans le résultat du modèle de vision, puis pour chaque plaque nouvellement identifiée dans la base d'infractions, marque la plaque comme traitée, déclenche l'agent d'alerte avec les données JSON de l'infraction, et affiche sa réponse finale (en repartant du dernier message de l'assistant) dans l'interface Streamlit, avec une gestion d'erreur si la sortie n'est pas un JSON valide.

Etape 7 — Interface Streamlit et lancement du pipeline vidéo

[Lignes 127-142](#)

```

# Streamlit UI
st.title(" License Plate Violation Detection")

if st.button("■ Start Detection from Video"):
    with st.spinner("Processing video..."):
        pipeline = InferencePipeline.init_with_workflow(
            api_key=roboflow_key,
            workspace_name="tp-qpwtk",
            workflow_id="custom-workflow",
            video_reference='video/traffic-3.mp4',
            max_fps=30,
            on_prediction=my_sink
        )
        pipeline.start()
        pipeline.join()
    st.success(" Finished processing video.")

```

Ce dernier bloc affiche le titre de l'application et un bouton de démarrage manuel. Au clic, il construit le pipeline d'inférence Roboflow configuré avec le workflow personnalisé (détection puis lecture de plaque), le fichier vidéo local à analyser, et la fonction my_sink comme point de connexion entre la bibliothèque de vision et la logique métier du fichier. pipeline.start() lance le traitement et pipeline.join() bloque le script jusqu'à sa fin, avant d'afficher un message de succès.

Projet 27 — 27-ImportExportSpecialist

Conformite import/export UK-Maroc, verification et analyse de risque

LE BUT DE CE PROJET

Le but de ce projet est de générer automatiquement un rapport de conformité douanière fiable pour une expédition de produits électroniques entre le Royaume-Uni et le Maroc, en s'appuyant sur le texte réel des lois d'exportation et d'importation plutôt que sur ce qu'un modèle de langage croit savoir de mémoire. Le problème que ça résout : les règles d'export-import (licences, taxes, quotas, produits interdits, formulaires douaniers) sont denses, changent régulièrement et sont réparties dans deux textes de loi distincts, dans deux pays différents ; les lire et les croiser à la main prend des heures, et demander directement la réponse à une IA généraliste est risqué, car elle peut inventer avec assurance un taux de TVA ou un code de licence qui n'existe pas (une hallucination), simplement parce que ça "sonne juste".

La technique démontrée est le RAG (Retrieval-Augmented Generation, ou génération augmentée par récupération) : au lieu de laisser le modèle répondre depuis sa mémoire d'entraînement, le système va d'abord chercher le texte réel des deux lois (converties de PDF en texte via un service d'OCR) puis injecte des extraits de ces textes directement dans le contexte envoyé au modèle avant qu'il ne rédige sa réponse. Le modèle ne "récite" donc plus, il "cite" une source qu'on lui a fournie. Le projet illustre aussi une décomposition en agents spécialisés qui se relaient : un agent traduit, un agent expert rédige le rapport de conformité (en s'appuyant sur le RAG et, si besoin, une recherche web), un agent vérificateur relit ce rapport pour repérer les incohérences ou les affirmations non sourcées, et un agent d'analyse de risques en extrait les dangers juridiques, opérationnels et financiers.

Pourquoi ça compte : cette architecture permet d'obtenir un premier rapport de conformité rapide et ancré dans des sources vérifiables, avec un mécanisme intégré de relecture critique, plutôt qu'une réponse unique et opaque d'un chatbot sur laquelle on n'aurait aucune garantie avant de s'en servir pour une vraie expédition commerciale.

27-ImportExportSpecialist/import_export_agent.py

Définit le pipeline d'extraction des textes de loi par OCR, le mécanisme RAG qui les injecte dans le contexte du modèle, et les quatre agents agno (traduction, spécialiste conformité, vérification, analyse de risques).

Etape 1 — Configuration et clés API

Lignes 1-17

```
import os
from textwrap import dedent
from dotenv import load_dotenv
from mistralai.client import Mistral
from agno.models.groq import Groq
from agno.agent import Agent
from agno.tools.duckduckgo import DuckDuckGoTools
```

```
# Load API keys from .env
load_dotenv()
# Le script original utilisait OpenAI (gpt-4o/gpt-4o-mini) ; remplacé par Groq
# pour réutiliser la clé GROQ_API_KEY déjà disponible.
groq_key = os.getenv("GROQ_API_KEY")
mistral_key = os.getenv("MISTRAL_API_KEY")

# Initialize Mistral client
client = Mistral(api_key=mistral_key)
```

Cette étape met en place les briques nécessaires au reste du fichier : le framework agno (agents, modèle Groq, outil de recherche web), le client Mistral (utilisé uniquement pour son service d'OCR) et le chargement des clés API depuis un fichier .env plutôt qu'écrites en dur dans le code, pour ne pas exposer de secrets si le projet est partagé. Le choix de Groq à la place d'OpenAI est un choix pragmatique de coût et de disponibilité : il permet de réutiliser une clé déjà existante pour interroger des modèles open-weight rapides plutôt que de payer un nouvel accès.

Etape 2 — Extraction du texte des lois par OCR

Lignes 19-40

```
# OCR PDF to Markdown
def ocr_pdf(pdf_path, pdf_name):
    if not os.path.exists(pdf_path):
        return False

    uploaded_pdf = client.files.upload(
        file={"file_name": pdf_path, "content": open(pdf_path, "rb")},
        purpose="ocr"
    )

    signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = client.ocr.process(
        model="mistral-ocr-latest",
        document={"type": "document_url", "document_url": signed_url.url},
        include_image_base64=True
    )

    with open(f"Documents/{pdf_name}.md", "w") as f:
        f.write("\n".join([page.markdown for page in ocr_response.pages]))

    return True
```

Les textes de loi sont fournis sous forme de PDF, un format image/pixels que le modèle de langage ne peut pas lire directement. Cette fonction envoie le PDF au service d'OCR (reconnaissance optique de caractères) de Mistral, qui le convertit en texte structuré Markdown page par page, puis écrit le résultat dans un fichier local. Ce fichier Markdown généré une seule fois sert ensuite de source de vérité réutilisable pour tous les agents, évitant de refaire cet appel coûteux à chaque exécution.

Etape 3 — Lecture des lois et retriever RAG combiné

Lignes 42-62

```
# Load OCR outputs
def export_law_document(agent, query, num_documents=None, **kwargs):
    with open("Documents/uk-export-law.md", "r") as f:
        return f.read()

def import_law_document(agent, query, num_documents=None, **kwargs):
    with open("Documents/morocco-import-law.md", "r") as f:
        return f.read()

# knowledge_retriever n'accepte qu'une seule fonction callable (pas une liste) :
# on combine les deux documents en un seul retriever. Les documents complets
# (~180 Ko à eux deux) dépassaient largement la limite Groq (8000 tokens/minute
# en tier gratuit) rien qu'en contexte, avant même la génération de la réponse ;
# on les tronque à un extrait représentatif.
```

```
MAX_LAW_EXCERPT_CHARS = 4000
```

```
def combined_law_documents(agent, query, num_documents=None, **kwargs):  
    return [  
        export_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],  
        import_law_document(agent, query)[:MAX_LAW_EXCERPT_CHARS],  
    ]
```

C'est le cœur du mécanisme RAG du projet : ces fonctions "retrieve" vont chercher le texte déjà extrait des deux lois et le renvoient pour qu'agno l'injecte automatiquement dans le contexte envoyé au modèle avant qu'il ne réponde. Comme agno n'accepte qu'un seul retrieveur (une fonction, pas une liste), les deux documents sont fusionnés en un seul, et tronqués à 4000 caractères chacun car les documents complets dépassaient la limite de débit de tokens de Groq sur le tier gratuit. Ce compromis (extrait plutôt que texte intégral) illustre une contrainte réelle du RAG en production : il faut équilibrer la richesse du contexte fourni au modèle avec les limites techniques et financières du fournisseur d'inférence.

Etape 4 — Agent de traduction

[Lignes 64-72](#)

```
# Translation Agent  
translation_agent = Agent(  
    name="Translation Agent",  
    model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),  
    instructions=dedent("""  
        You are an expert in translation. You are given a text in markdown format in French and you need to  
        translate it to English.  
        IMPORTANT: Keep the same structure and content of the original text.  
        """),  
)
```

Ce premier agent a un rôle unique et étroit : traduire un texte Markdown du français vers l'anglais en préservant sa structure, afin que la loi marocaine (rédigée en français) puisse être exploitée en anglais aux côtés de la loi britannique. Lui donner une responsabilité isolée plutôt que de fondre cette tâche dans l'agent spécialiste est un choix délibéré de conception multi-agent : un agent avec des instructions courtes et ciblées est plus fiable qu'un agent généraliste chargé de tout faire à la fois.

Etape 5 — Agent spécialiste de conformité export-import

[Lignes 74-148](#)

```
# Export-Import Compliance Expert  
specialist_agent = Agent(  
    name="Specialist Agent",  
    # Le retriever combiné injecte le texte intégral des deux lois dans le contexte,  
    # ce qui dépasse la limite Groq du modèle 120B (8000 tokens/minute) ; le modèle  
    # 20B a un quota plus élevé sur le tier gratuit.  
    model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),  
    instructions=dedent("""  
        You are an Export-Import Compliance Expert with access to two knowledge sources:  
        - The official UK export law document for electronics (including licences, restrictions, taxes,  
        fees).  
        - The official Moroccan import law document for electronics (including permits, quantity limits,  
        import duties, VAT, prohibited items, and customs procedures).  
  
        You also have access to GoogleSearchTools() to find up-to-date, authoritative data when it is missing  
        or unclear.  
  
        Your task is: given detailed product info (type, quantity, raw materials) and the shipment route (UK  
        to Morocco), provide a full, precise, step-by-step explanation of:  
  
        1. UK export process:  
        - Exact licence requirements by name and code,  
        - Clear restrictions on product types or materials,  
        - Specific export taxes, fees, or levies with exact amounts or percentages,  
        - Necessary documentation, including official form names or codes,
```

- Compliance rules and penalties with numbers and legal references.

2. **Moroccan import process**:

- Precise import permits and certifications required,
- Exact quantity restrictions or exemptions,
- Detailed import duties and VAT rates with percentages or fixed amounts,
- List of prohibited or restricted items by name and code,
- Required documents with official titles,
- Customs clearance steps with timelines and fees.

Use only **facts and figures** extracted from your knowledge base or from live, authoritative sources. Do not guess or be vague.

If any data is missing in your knowledge, immediately perform a `GoogleSearchTools()` lookup for official government or customs sites and incorporate exact data found.

Present the response in clear, professional language, with no filler text or disclaimers.

Provide the output as a flowing narrative, but include concrete numbers, names, codes, and references wherever applicable.

The goal is to give the user a complete, actionable export-import compliance report with real numbers and official requirements.

If data is unavailable, explicitly state that you researched and could not find updated official info.

```
"""),
expected_output=dedent("""
### UK Export Process
```

```
[Explain if an export licence is required for the product. Specify licence type and official code if available.]
```

```
[Mention any restrictions on product types, raw materials, or dual-use classifications found in the UK export law document.]
```

```
[Provide specific export taxes or fees applicable, with exact amounts or percentages, citing the document.]
```

```
[Describe required paperwork and compliance steps before shipment.]
```

```
### Moroccan Import Process
```

```
[Detail import permits, licences, or certifications required for the product according to the Moroccan import law document.]
```

```
[State any quantity limits or restrictions, quoting exact figures or thresholds.]
```

```
[Specify prohibited items or materials mentioned in the document.]
```

```
[Give import duties and VAT rates with percentages or fixed amounts, as documented.]
```

```
[List all required customs documents and procedures.]
```

```
### Taxes and Compliance Details
```

```
[Summarize how taxes and fees are calculated and at which stage (export or import) they apply, based on the findings.]
```

```
[Explain compliance obligations and potential penalties for non-compliance extracted from the documents.]
```

```
[Include any additional practical advice referenced in the documents for smooth customs clearance.]
```

```
### Summary and Recommendations
```

```
[Provide a concise summary of the entire export-import process highlighting key points and actionable next steps based solely on the document contents.]
```

```
"""),
knowledge=None,
search_knowledge=True,
knowledge_retriever=combined_law_documents,
tools=[DuckDuckGoTools()],
compress_tool_results=True,
```

```
)
```

C'est l'agent central du projet. Il combine trois mécanismes : le retriever RAG (`knowledge\_retriever=combined\_law\_documents`) qui lui fournit les extraits de loi réels, un outil de recherche web (`DuckDuckGoTools`) pour compléter l'information si le texte de loi ne suffit pas, et un `expected\_output` qui impose un gabarit de rapport en quatre sections fixes (export UK, import Maroc, taxes, résumé). Le choix d'un modèle Groq plus petit (20B au lieu de 120B) pour cet agent précis est dicté par une contrainte technique : c'est lui qui reçoit le plus gros volume de contexte (les deux extraits de loi), et le modèle 120B aurait dépassé le quota de tokens par minute du tier gratuit. Les instructions très détaillées et le format de sortie imposé forcent le modèle à produire des réponses concrètes et sourcées plutôt que des généralités vagues.

Etape 6 — Agent de vérification de conformité

Lignes 150-200

```
# Agent 4 – Compliance Verification
verification_agent = Agent(
    name="Compliance Verification Agent",
    model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),
    instructions=dedent("""
    You are a regulatory compliance auditor specializing in international trade.

    Your job is to audit an export-import compliance report that was generated by another expert agent.
    You must carefully review the content and assess:

    1. **Legal consistency**:
    - Are the legal steps and procedures correctly sequenced?
    - Are licenses, permits, and document names aligned with real trade practice?

    2. **Accuracy of references**:
    - Are any numbers, codes, rates or penalties properly supported?
    - Are there unsupported claims, vague terms, or approximations?

    3. **Completeness**:
    - Are all sections fully covered: UK export, Moroccan import, taxes, and summary?
    - Are essential steps or legal documents missing?

    Your output should:
    - Clearly identify each issue, grouped by section,
    - Suggest accurate corrections where needed,
    - Confirm when a section is compliant.

    You must use markdown to organize your response under:
    - UK Export Section
    - Moroccan Import Section
    - Taxation Section
    - General Observations

    Be precise, legal, and concise – no filler or speculation.
    """),
    expected_output=dedent("""
    ### UK Export Section
    [ or X Summary of findings with details and line references]

    ### Moroccan Import Section
    [ or X Summary of findings with suggested corrections]

    ### Taxation Section
    [ or X Breakdown of duties, VAT accuracy, correction if needed]

    ### General Observations
    [Highlight formatting, clarity, completeness, and red flags if any]
    """),
    knowledge=None,
    search_knowledge=False,
    tools=[],
)
```

Cet agent ne génère pas de nouveau contenu métier : il relit le rapport produit par l'agent spécialiste et joue le rôle d'auditeur, en évaluant la cohérence juridique, la présence de sources chiffrées, et l'exhaustivité par rapport aux quatre sections attendues. Il n'a ni `retrieve` ni outil de recherche (`tools=[]`, `search_knowledge=False`) car sa tâche est purement une relecture critique du texte qu'on lui donne, pas une recherche d'information nouvelle. Ce garde-fou est essentiel dans un système RAG : même avec des extraits de loi réels en entrée, le modèle qui rédige peut mal interpréter ou survendre une information, et cette seconde passe indépendante sert de filet de sécurité avant qu'un humain ne s'appuie sur le rapport.

Etape 7 — Agent d'analyse de risques

Lignes 202-240

```
# Agent 5 – Risk Analysis Agent
risk_agent = Agent(
    name="Risk Analysis Agent",
    model=Groq(id="openai/gpt-oss-120b", api_key=groq_key),
    instructions=dedent("""
    You are a senior risk analysis advisor for international trade operations between the UK and Morocco.

    Based on a complete export-import compliance report, your task is to identify and evaluate potential
    risks across three main areas:

    1. Legal Risks:
    - Missing or ambiguous license requirements
    - Penalties for non-compliance
    - Contradictions or lack of jurisdiction clarity

    2. Operational Risks:
    - Risks of shipment delay due to customs, documents, unclear timelines
    - Problems with quantity limits, permit validation, or procedural bottlenecks

    3. Financial Risks:
    - Unexpected taxes, fees, or duties
    - Currency conversion, VAT surprises, or payment processing complications

    You must respond in clear markdown format, with bold headers for each section.
    Use concrete details from the report to justify your findings.
    Do not invent risks – base your output only on what is found in the report.

    You may recommend mitigation strategies if applicable.
    """),
    expected_output=dedent("""
    ### Legal Risks
    [Bullet list of legal vulnerabilities based on the report]

    ### Operational Risks
    [Bullet list of logistics, customs or documentation-related risks]
    """),
    knowledge=None,
    search_knowledge=False,
    tools=[],
)
```

Dernier maillon de la chaîne, cet agent relit lui aussi le rapport du spécialiste (pas le texte de loi brut) mais avec un angle différent : traduire les constats de conformité en risques concrets pour l'entreprise, classés en juridique, opérationnel et financier. L'instruction explicite "Do not invent risks — base your output only on what is found in the report" vise directement le risque d'hallucination : plutôt que de laisser le modèle imaginer des scénarios catastrophes génériques, on le contraint à justifier chaque risque par un élément réellement présent dans le rapport déjà généré et ancré dans les lois.

27- ImportExportSpecialist/app.py

Interface Streamlit qui orchestre l'exécution séquentielle du pipeline (OCR, traduction, analyse spécialiste, vérification, risques) et affiche les résultats à l'utilisateur.

Etape 1 — Mise en place de l'interface

Lignes 1-8

```
import streamlit as st
from import_export_agent import *

# Streamlit UI
if __name__ == "__main__":
    st.title("Import Export Specialist Multi-Agent System")

    product_details = st.text_input("Enter product and shipment details")
```

Le fichier importe tout le contenu de `import_export_agent.py` (les agents et fonctions définis précédemment) et construit une interface web minimaliste avec Streamlit : un titre et un champ de texte où l'utilisateur décrit le produit et le trajet d'expédition. C'est le point d'entrée visuel qui va déclencher tout le pipeline multi-agent défini dans l'autre fichier.

Etape 2 — Déclenchement et extraction OCR des deux lois

Lignes 10-14

```
if st.button("Submit"):
    with st.spinner("Running multi-agent workflow..."):
        # Get PDFs and process them
        ocr_pdf("Documents/uk-export-law.pdf", "uk-export-law")
        ocr_pdf("Documents/morocco-import-law.pdf", "morocco-import-law")
```

Quand l'utilisateur clique sur "Submit", le pipeline démarre par la conversion des deux PDF de loi en Markdown via ``ocr_pdf``, condition préalable indispensable puisque tous les agents suivants (retrieve RAG, traduction) dépendent de fichiers texte déjà présents sur disque. Le ``st.spinner`` donne un retour visuel pendant ce traitement, qui peut prendre du temps car il implique des appels réseau à l'API OCR.

Etape 3 — Traduction du texte marocain

Lignes 16-19

```
# Translation step
translation_response = translation_agent.run(import_law_document(None, None))
with open("Documents/morocco-import-law-en.md", "w") as f:
    f.write(translation_response.content)
```

Le texte de loi marocain, extrait en français par l'OCR, est lu puis envoyé à l'agent de traduction pour être converti en anglais, et le résultat est sauvegardé dans un nouveau fichier. Cette étape prépare une version anglaise du document qui pourrait être réutilisée ailleurs, même si l'agent spécialiste, lui, continue de s'appuyer sur son propre retrieveur combiné plutôt que sur ce fichier traduit.

Etape 4 — Génération et affichage du rapport de conformité

Lignes 21-26

```
# Specialist analysis (le retriever combiné est déjà configuré à la construction de l'agent)
specialist_response = specialist_agent.run(product_details)

# Display specialist report
st.subheader(" Specialist Compliance Report")
st.markdown(specialist_response.content)
```

La description du produit saisie par l'utilisateur est transmise à l'agent spécialiste, qui va automatiquement récupérer les extraits de loi via son retrieveur RAG et générer le rapport de conformité structuré. Le résultat est affiché directement dans l'interface sous forme de Markdown, rendant visibles à l'utilisateur les sections, titres et mises en forme produits par l'agent.

Etape 5 — Vérification et analyse de risques

Lignes 28-36

```
# Verification step
verification_response = verification_agent.run(specialist_response.content)
st.subheader(" Verification Feedback")
st.markdown(verification_response.content)

# Risk analysis
risk_response = risk_agent.run(specialist_response.content)
st.subheader(" Risk Analysis")
st.markdown(risk_response.content)
```

Le rapport produit par l'agent spécialiste est transmis successivement à l'agent de vérification puis à l'agent d'analyse de risques, chacun affichant son propre résultat dans une section dédiée de l'interface. L'utilisateur voit donc apparaître trois blocs complémentaires (rapport, relecture critique, risques) plutôt qu'une seule réponse, ce qui matérialise concrètement le principe de contrôle croisé entre agents spécialisés.

Projet 29 — 29-Investment_insight

Profil client + recommandation d'investissement

LE BUT DE CE PROJET

Le but de ce projet est d'automatiser le travail qu'un conseiller financier fait avant un rendez-vous client : rassembler tout ce qu'on sait déjà du client, compléter par une recherche publique si le client est inconnu en interne, puis produire une analyse financière rigoureuse d'une action boursière qui l'intéresse, jusqu'à une recommandation d'investissement formatée et chiffrée. Le problème que ça résout : ce travail de préparation implique normalement de fouiller dans des notes de réunion internes, de chercher le client sur le web s'il n'a pas de dossier, puis de croiser des données financières en temps réel (cours de bourse, ratios, avis d'analystes) avec des rapports qualitatifs — un processus lent, difficile à standardiser d'un conseiller à l'autre, et où l'on oublie facilement une information.

La technique démontrée est celle d'une équipe d'agents ("team") qui se relaient selon une condition, plutôt qu'un empilement d'agents qui tournent systématiquement tous en même temps : un agent de recherche interne consulte d'abord la base de données des réunions passées, et ce n'est que s'il répond explicitement "CLIENT NOT FOUND" qu'un second agent de recherche web prend le relais pour chercher des informations publiques sur le client. C'est le patron "l'orchestrateur délègue à un agent spécialisé en fonction du résultat précédent", plus économe et plus pertinent qu'exécuter toutes les recherches à chaque fois. En aval, un agent d'analyse financière s'appuie sur une API boursière pour produire un rapport détaillé et le sauvegarde dans un fichier, qu'un agent de recommandation relit ensuite pour rédiger la conclusion finale — une communication entre agents qui passe par un fichier partagé plutôt que par un simple échange de texte en mémoire.

Pourquoi ça compte : cette architecture montre comment automatiser un flux de décision qui, normalement, exige de croiser plusieurs sources hétérogènes à la main, tout en gardant chaque étape traçable et vérifiable (chaque agent produit une sortie visible ou sauvegardée) plutôt qu'une réponse unique et opaque.

29-Investment_insight/main.py

Définit une équipe d'agents agno qui recherche un client (en interne puis sur le web si besoin) et produit une analyse financière et une recommandation d'investissement sur une action.

Etape 1 — Configuration et clés API

Lignes 1-19

```
from agno.agent import Agent
from agno.team.team import Team
from agno.models.openai import OpenAIChat
from agno.models.google import Gemini
from agno.tools.scrapegraph import ScrapeGraphTools
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.file import FileTools
from agno.tools.yfinance import YFinanceTools
from agno.tools.exa import ExaTools
```

```

from dotenv import load_dotenv
import os

# Configuration
load_dotenv()

sgai_api_key = os.getenv("SGAI_API_KEY")
exa_api_key = os.getenv("EXA_API_KEY")
api_key_openai = os.getenv("OPENAI_API_KEY")

```

Cette étape charge les briques agno nécessaires : la classe `Agent` pour les agents individuels, `Team` pour l'orchestration collective, plusieurs outils (fichiers locaux, recherche web ScrapeGraph et Exa, données boursières YFinance) et les clés API associées, lues depuis un fichier `.env`. Le fait de disposer de plusieurs outils différents dès l'import montre que ce projet combine plusieurs sources de données hétérogènes (fichier interne, web, marché financier) au sein d'un même flux.

Etape 2 — Agent de recherche interne

Lignes 21-57

```

# Agent 1: Internal Search Agent
internal_search = Agent(
    name="internal search agent",
    # model=Gemini(id=id, api_key=api_key),
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    tools=[FileTools()],
    description="""
A data analyst agent that retrieves and summarizes historical client meeting records
from the internal database ('29-Investment_insight/Data/data.json').
The agent provides concise, insight-driven summaries based on each meeting entry.
""",
    instructions="""
You are given access to a JSON file containing client information, including detailed meeting notes.

Your task is to:
1. Search for the specified client by full name in the '29-Investment_insight/Data/data.json' file.
2. If the client is found, focus on the 'meeting_notes' field.
3. For each note, write a **professional, well-structured summary, and actions** using markdown bullet
format like this:
# Internal Search:
- <Date>: Summary
- **Action Items**:

Notes:
- Summarize clearly; do not just copy/paste raw content.
- Group related ideas. Be concise but informative.
- Ensure the tone is suitable for executive review.

Final response must be:
- Well-structured using Markdown
- Professional in tone
- Only include one of the two sections depending on the results.

If the client is not found, respond exactly with: CLIENT NOT FOUND.
""",
    stream=True
)

```

Cet agent est le premier réflexe du système : il utilise `FileTools` pour lire un fichier JSON local contenant l'historique des réunions clients, cherche le client par nom, et résume ses notes de réunion en un format Markdown professionnel. La consigne finale ("If the client is not found, respond exactly with: CLIENT NOT FOUND") est cruciale : c'est un signal texte exact et prévisible que les autres composants du système (l'agent web, l'équipe orchestratrice) vont pouvoir détecter pour décider de la suite du flux, transformant une réponse en langage naturel en un déclencheur logique fiable.

Etape 3 — Agent de recherche web (filet de sécurité)

Lignes 59-91

```
# Agent 2: Web Search Agent
web_search = Agent(
    name="web search agent",
    # model=Gemini(id=id, api_key=api_key),
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    tools=[ScrapeGraphTools(api_key=sgai_api_key, markdownify=True), ExaTools(api_key=exa_api_key)],
    description=""

A web intelligence agent activated only when a client is not found in the internal records.
It searches the web to uncover publicly available, up-to-date information about the client,
focusing on professional activity and recent public appearances.
""",
    instructions=""

    Only respond if the internal search agent confirms: "CLIENT NOT FOUND".

Step 1: Use ExaTools to search the web for the client's full name.
Step 2: Identify relevant links such as:
- Personal websites
- LinkedIn, GitHub, or other professional profiles
- News articles or blog posts
- Conference appearances, guest speaking events, or publications

Step 3: Use ScrapeGraphTools to scrape only the most relevant and recent pages.
Step 4: Summarize any recent or notable findings into a clear, executive-level markdown summary.

Formatting guidelines:
- Use bullet points per finding.
- Each point should include a concise description of the event, post, or appearance.
- Keep it factual, neutral, and informative – suitable for a professional briefing.
- If no relevant information is found, respond with: "No public information found."
""",
    show_tool_calls=True,
    stream=True,
)
```

Cet agent ne se déclenche, par instruction explicite, que si l'agent interne a confirmé "CLIENT NOT FOUND" : c'est le filet de sécurité qui prend le relais uniquement en cas de client inconnu en interne, évitant une recherche web systématique et inutile. Il combine deux outils complémentaires : Exa pour trouver les liens pertinents (profils professionnels, articles, conférences) puis ScrapeGraph pour extraire le contenu réel de ces pages, avant de produire un résumé factuel neutre destiné à un usage professionnel.

Etape 4 — Équipe de recherche client (orchestration conditionnelle)

Lignes 93-129

```
# Team: Client Research Team
client_research = Team(
    name="Client Research Agent",
    # model=Gemini(id=id, api_key=api_key),
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    members=[internal_search, web_search],
    # model="coordinate",
    tools=[],
    description=""

A high-level research agent that coordinates two specialized sub-agents:
- An internal search agent that queries local client records.
- A web search agent that performs external research if internal data is missing.

This agent orchestrates both and produces a professional, structured summary about the client.
""",
    instructions=""

Your goal is to gather insights about a client using two research sources:

1. First, delegate the task to the internal search agent to locate any existing internal meeting records about the client.
2. If the internal agent returns information, format it under the section titled:
```

```
# Internal Search:
- Date: Summary
- **Action Items**: if any

3. If the internal agent replies "CLIENT NOT FOUND", delegate the task to the **web search agent**.
4. If the web search agent returns anything, format it under the section:
# Web Search:
- Summarized bullet points from recent or relevant online content

Final response must be:
- Well-structured using Markdown
- Professional in tone
- Only include one of the two sections depending on the results.
"""
    stream=True,
    show_members_responses=True,
)

```

La classe `Team` d'agno est ici l'orchestrateur : elle reçoit les deux agents précédents comme `members` et applique elle-même la logique conditionnelle décrite en toutes lettres dans ses instructions — d'abord déléguer à la recherche interne, puis ne déléguer à la recherche web que si le résultat est "CLIENT NOT FOUND". C'est la démonstration centrale du fichier du patron coordinateur/travailleurs spécialisés : une couche de décision au-dessus des agents individuels choisit dynamiquement qui doit agir, plutôt que d'exécuter une chaîne fixe et systématique.

Etape 5 — Agent d'analyse financière

Lignes 131-204

```
# Agent 3: Financial Data Agent
financial_data_agent = Agent(
    name="Financial Intelligence Agent",
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    tools=[
        YFinanceTools(
            stock_price=True,
            company_info=True,
            stock_fundamentals=True,
            key_financial_ratios=True,
            analyst_recommendations=True,
        ),
        FileTools(),
        ExaTools(api_key=exa_api_key),
        ScrapeGraphTools(api_key=sgai_api_key, markdownify=True),
    ],
    description="""
A financial analysis expert agent responsible for researching specific investment opportunities based on
client context.
This agent can extract financial data, summarize earnings calls, and provide balanced investment
recommendations.

It uses insights from the client's research and combines them with real-time financial data
and qualitative sources to support decision-making.
""",
    instructions="""
You are a financial analyst agent tasked with evaluating a stock as an investment opportunity. Your output
must strictly follow the structure outlined below.

Start by understanding the client's investment preferences, goals, and risk tolerance. Tailor your
recommendations to align with that profile.

End by Saving the output in a file called 'financial_data.md'.

---
Analyze [Stock Name] as an investment opportunity using this exact structure:

---
**[Stock Name] Overview**

```

[1-2 sentences summarizing the company's core business and differentiation.]

Pros of Investing in [Stock Name]

- **Innovative Technology**: [Detail unique tech/advantages, if applicable.]
- **Strategic Partnerships**: [List key partnerships/alliances.]
- **Market Demand**: [Highlight growth sectors/tailwinds.]
- **Regulatory Support**: [Mention favorable policies/risks.]
- **Revenue Potential**: [Identify high-margin segments or opportunities.]

Cons of Investing in [Stock Name]

- **Regulatory Challenges**: [Permitting delays/policy risks.]
- **Financial Health**: [Losses/debt/cash burn concerns.]
- **Market Risks**: [Supply chain/competition/volatility.]
- **Execution Risks**: [Management hurdles/operational uncertainties.]

Overall: [Stock Name] presents [summary of opportunity], but investors should weigh [key risks].

Use the following data sources to inform your analysis (in priority order):

1. Latest earnings calls (Q2/Q3 2024).
2. SEC filings (10-K/10-Q).
3. Analyst reports (e.g., Bloomberg, Reuters).
4. Industry trends related to the company (e.g., AI, energy transition, EV, etc.).
5. Financial metrics using YFinance (stock fundamentals, key ratios, analyst recommendations).

Important:

Before saving your output to 'financial_data.md', perform this final step:

- Replace all smart punctuation (like '„', '„', '■', '„', '„') with standard ASCII characters (e.g., '„', '„', '„').
- Save the output in UTF-8 encoding using FileTools.

Your output must be:

- Clean, well-formatted Markdown
- Free of encoding issues
- Saved in a file called 'financial_data.md'

```
"""
    success_criteria="Your task is only complete when the output is saved to 'financial_data.md'.",
    stream=True,
)
```

Cet agent combine un outil de données boursières (`YFinanceTools`, qui donne accès au cours, aux ratios financiers et aux avis d'analystes en temps réel) avec des outils de recherche et de scraping web pour compléter avec du contexte qualitatif (résultats trimestriels, tendances sectorielles). Il doit produire une analyse "avantages/inconvénients" selon un gabarit strict, puis sauvegarder physiquement le résultat dans un fichier `financial\_data.md` — le `success\_criteria` explicite indique à agno que la tâche n'est considérée terminée que lorsque ce fichier existe, garantissant que l'agent suivant trouvera bien la donnée dont il a besoin. Le nettoyage de la ponctuation "intelligente" avant sauvegarde évite des problèmes d'encodage lors de la relecture du fichier par l'étape suivante.

Etape 6 — Agent de recommandation (gabarit tableau)

[Lignes 206-244](#)

```
# Agent 4: Recommendation Agent
recommendation_agent = Agent(
    name="Financial Advisor Agent",
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    tools=[FileTools(), YFinanceTools(
        stock_price=True,
        company_info=True,
        stock_fundamentals=True,
        key_financial_ratios=True,
        analyst_recommendations=True,
    ), ExaTools(api_key=exa_api_key)],
    description=""
```

```

A financial advisory agent responsible for issuing structured investment recommendations based on prior
financial analysis.
It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed
template,
including both qualitative insights and quantitative metrics.
"""
    instructions="""
You are tasked with generating a professional stock recommendation report and save it as a
'recommendation.md' file.

Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's
performance, risks, and potential.

Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown
template with realistic and accurate values:

---
# **Company Details**:
```

Name	Rating	Target	ISIN	Last Price	Currency	Sector	Sector Rating
[Company]	[Rating]	[Price]	[ISIN]	\$XX.XX	[Ccy]	[Sector]	[Sector Rating]

```

Notes:
- Make sure all values are consistent with the financial insights in the input file.
- Maintain a professional tone suitable for investment committees or client reports.
- Use markdown formatting to ensure the output is clean and readable.

Step 3: Save the final output as a 'recommendation.md' file.
"""
    stream=True,
)

```

Cet agent illustre la communication inter-agents par fichier partagé : au lieu de recevoir le texte de l'analyse financière directement en mémoire, ses instructions lui demandent explicitement de lire le fichier `financial\_data.md` produit par l'agent précédent, puis de compléter un tableau de métriques (note, cours, ISIN, secteur) avant de sauvegarder le tout dans `recommendation.md`. C'est un gabarit orienté "fiche synthèse tabulaire", pensé pour un usage type comité d'investissement.

Etape 7 — Agent de recommandation (gabarit narratif alternatif)

[Lignes 246-298](#)

```

# Agent 5: Financial Advisor Agent (Alternative Template)
recommendation_agent_v2 = Agent(
    name="Financial Advisor Agent",
    model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
    tools=[FileTools(), YFinanceTools(
        stock_price=True,
        company_info=True,
        stock_fundamentals=True,
        key_financial_ratios=True,
        analyst_recommendations=True,
    ), ExaTools(api_key=exa_api_key)],
    description="""
A financial advisory agent responsible for issuing structured investment recommendations based on prior
financial analysis.
It reads data from 'financial_data.md' and produces a detailed recommendation report using a fixed
template,
including both qualitative insights and quantitative metrics.
"""
    instructions="""
You are tasked with generating a professional stock recommendation report and save it as a
'recommendation.md' file.

Step 1: Load and read the file 'financial_data.md'. It contains a prior detailed analysis of the stock's
performance, risks, and potential.

```

Step 2: Based on the data in that file and YFinance tools, complete the following structured markdown template with realistic and accurate values:

```
---
[Company Name] Stock Recommendation

**Rating**: [Buy / Hold / Sell]
**Target Price**: [$XX.XX]

**Rationale**:

- **Financial Health**: [Brief summary of financials - EPS, revenue trends, profitability]
- **Regulatory Environment**: [Summary of policy or compliance issues]
- **Growth Initiatives**: [Acquisitions, partnerships, product pipelines]
- **Market Position**: [Competitive strengths or threats]
- **Risk Factors**: [Short/long-term risks or volatility factors]

**Overall**: [1-2 sentence summary – should reflect the company's outlook and your advised action]

**Key Metrics**:

| Metric | Value |
|-----|-----|
| Open Price | $XX.XX |
| Market Cap | $X.XXB |
| 52-Week High | $XX.XX |
| 52-Week Low | $XX.XX |
| P/E Ratio | XX.XX |
| Volume | X,XXX,XXX |
| Volatility (30D) | XX% |
"""
    stream=True,
)
```

Cette variante reprend exactement la même mécanique que l'agent précédent (lire `financial\_data.md`, s'appuyer sur YFinance, sauvegarder `recommendation.md`) mais avec un gabarit de sortie différent : une fiche narrative de type "note d'analyste" avec note Achat/Conserver/Vendre, prix cible et rationale qualitative, complétée d'un tableau de métriques de marché plus détaillé. Le fichier propose ainsi deux formats de restitution interchangeables pour le même besoin, à choisir selon le public visé (comité vs. conseiller individuel).

Etape 8 — Exemples d'exécution

Lignes 300-309

```
# Exemples d'utilisation
if __name__ == "__main__":
    # Test des agents
    internal_search.print_response("give me info about Nicole Robinson", markdown=True)

    client_research.print_response("I'm Nicole Robinson, I'm a Dental Hygienist, do you have any info about me?", stream=True, markdown=True)

    financial_data_agent.print_response("Summarize what Binance does and provide the pros and cons of investing into it based on the earnings call Q2/Q3 2024", stream=True, markdown=True)

    recommendation_agent.print_response("Nicole Robinson is interested in Binance stock. What is our recommendation for this stock?", stream=True, markdown=True)
```

Ce bloc final sert de démonstration exécutable : il appelle successivement l'agent de recherche interne seul, puis l'équipe complète, puis l'agent financier, puis l'agent de recommandation, chacun avec une requête d'exemple concrète (une cliente fictive et une action, Binance). `print\_response` affiche directement le résultat dans la console avec mise en forme Markdown, ce qui permet de vérifier rapidement que chaque agent du pipeline fonctionne indépendamment avant de les enchaîner dans une application réelle.

Projet 30 — 30-compliance_agent

Rapport de conformité réglementaire UE, export Word

LE BUT DE CE PROJET

Le but de ce projet est de produire automatiquement, pour une entreprise donnée, un rapport de conformité réglementaire européenne complet et prêt à partager sous forme de document Word professionnel. Le problème que ça résout : une entreprise qui opère sur le marché européen doit respecter un enchevêtrement de règles qui se chevauchent et évoluent souvent (protection des données avec le RGPD, encadrement de l'intelligence artificielle, obligations environnementales, règles sectorielles, sécurité des produits), et produire une synthèse claire de ces obligations demande normalement l'intervention d'un juriste ou d'un consultant spécialisé qui lit les textes officiels, catégorise les risques par criticité, et rédige un rapport présentable, par exemple pour un comité de direction.

La technique démontrée est double. D'une part, un agent volontairement restreint dans ses sources : ses recherches web sont limitées par domaine à eur-lex.europa.eu et ec.europa.eu (les sites officiels de la législation européenne), combiné à un outil de "raisonnement" explicite, dans le but de réduire le risque que le modèle invente une règle ou une échéance qui n'existe pas. D'autre part, une conversion automatique du rapport, d'abord rédigé en Markdown par l'agent, vers un vrai document Word (.docx) mis en forme : page de titre, titres hiérarchisés, listes à puces ou numérotées, texte en gras/italique et pied de document, réalisée ligne par ligne avec la bibliothèque python-docx plutôt que par un simple copier-coller.

Pourquoi ça compte : ce projet montre comment transformer une tâche de recherche réglementaire et de rédaction qui prendrait normalement plusieurs jours à un consultant en un script reproductible qui produit un livrable directement partageable, tout en limitant délibérément les sources d'information de l'IA à des textes officiels pour réduire le risque de conseils réglementaires erronés, qui pourraient avoir de vraies conséquences juridiques si on leur faisait confiance sans vérification.

30-compliance_agent/main.py

Définit une classe qui interroge un agent de conformité UE limité aux sources officielles, sauvegarde sa réponse en Markdown, puis convertit ce Markdown en un rapport Word mis en forme.

Etape 1 — Correction d'encodage console et imports

Lignes 1-21

```
import sys

# La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas
# les emojis utilisés dans les messages de ce script (ex: , ).
for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import os
import re
```

```

from datetime import datetime
from pathlib import Path
from typing import Optional
from agno.agent import Agent
from agno.models.groq import Groq
from agno.tools.exa import ExaTools
from agno.tools.reasoning import ReasoningTools
from docx import Document
from docx.shared import Pt
from docx.enum.text import WD_PARAGRAPH_ALIGNMENT
from dotenv import load_dotenv

```

Avant même d'importer le framework d'agents, le script force la console Windows à utiliser l'encodage UTF-8, car son encodage par défaut (cp1252) ne sait pas afficher les emojis utilisés plus loin dans les messages de statut ; sans ce correctif, le script planterait dès son premier print contenant un emoji. Le reste de l'étape importe les briques nécessaires aux deux moitiés du fichier : agno et Exa côté agent de recherche, python-docx (`Document`, `Pt`, `WD\_PARAGRAPH\_ALIGNMENT`) côté génération du rapport Word.

Etape 2 — Initialisation de la classe et configuration de l'agent

Lignes 23-69

```

class AnalyseurConformiteUE:
    """Système d'Analyse de Conformité UE"""

    def __init__(self, config_path: Optional[str] = None):
        """Initialise l'analyseur de conformité"""
        load_dotenv(config_path)

        # Récupération des clés API
        self.exa_api_key = os.getenv("EXA_API_KEY")
        # Le script original utilisait OpenAI (gpt-4-turbo) ; remplacé par Groq
        # pour réutiliser la clé GROQ_API_KEY déjà disponible.
        self.groq_api_key = os.getenv("GROQ_API_KEY")

        # Validation des clés
        if not self.groq_api_key:
            raise ValueError("Clé API Groq manquante")

        # Initialisation de l'agent
        self._configurer_agent()

    def _configurer_agent(self):
        """Configure l'agent d'analyse de conformité"""
        outils = [ReasoningTools(add_instructions=True)]

        # Ajout d'ExaTools si la clé est disponible
        if self.exa_api_key:
            outils.append(ExaTools(
                api_key=self.exa_api_key,
                include_domains=["eur-lex.europa.eu", "ec.europa.eu"]
            ))

        self.agent = Agent(
            name="Analyseur de Conformité UE",
            model=Groq(
                id="openai/gpt-oss-120b",
                api_key=self.groq_api_key
            ),
            tools=outils,
            debug_mode=True,
            description="""
            Agent spécialisé dans l'analyse de la conformité réglementaire européenne.
            Fournit des évaluations de conformité structurées pour les entreprises
            opérant sur le marché européen.
            """,
            instructions=self._obtenir_instructions_agent(),

```

```
        markdown=True,  
    )
```

Le projet est organisé en classe plutôt qu'en simples fonctions au niveau module : le constructeur charge les clés API, valide qu'une clé Groq est bien présente (sinon il arrête tout de suite avec une erreur explicite plutôt que d'échouer plus tard au premier appel), puis délègue la configuration de l'agent à une méthode dédiée. Le point clé de cette configuration est `include_domains=["eur-lex.europa.eu", "ec.europa.eu"]` : l'outil de recherche web n'est pas laissé libre de fouiller tout Internet, il est explicitement cantonné aux deux sites officiels de la législation européenne, ce qui réduit directement le risque que l'agent s'appuie sur une source non fiable ou obsolète. L'ajout d'un `ReasoningTools` donne par ailleurs à l'agent une capacité de raisonnement explicite avant de répondre, plutôt qu'une réponse en un seul jet.

Etape 3 — Instructions détaillées du cadre d'analyse

[Lignes 71-112](#)

```
def _obtenir_instructions_agent(self) -> str:  
    """Instructions détaillées pour l'agent de conformité"""  
    return """  
    Vous êtes un expert en conformité UE. Votre mission est de fournir des analyses  
    réglementaires complètes pour les entreprises opérant sur les marchés européens.  
  
    CADRE D'ANALYSE :  
    1. PROFIL DE L'ENTREPRISE  
        - Identifier les activités principales  
        - Déterminer la classification sectorielle  
        - Évaluer l'étendue géographique des opérations UE  
  
    2. CARTOGRAPHIE RÉGLEMENTAIRE  
        - Rechercher les directives et règlements applicables  
        - Identifier les exigences sectorielles spécifiques  
        - Suivre les calendriers de mise en œuvre  
  
    3. ÉVALUATION DES RISQUES DE CONFORMITÉ  
        - Catégoriser les règlements par criticité (Élevée/Moyenne/Faible)  
        - Identifier les lacunes potentielles de conformité  
        - Évaluer les sanctions et pénalités  
  
    4. DOMAINES RÉGLEMENTAIRES PRINCIPAUX :  
        - Protection des données : RGPD, ePrivacy  
        - IA & Numérique : Loi sur l'IA, Loi sur les services numériques  
        - Environnement : Pacte vert, CSRD, Taxonomie  
        - Sectoriel : Automobile, Services financiers  
        - Sécurité des produits : Marquage CE, Cybersécurité  
  
    5. STRUCTURE DU RAPPORT :  
        - Résumé exécutif avec priorités clés  
        - Calendrier réglementaire  
        - Matrice des risques  
        - Feuille de route de conformité  
        - Exigences en ressources  
  
    STRATÉGIE DE RECHERCHE :  
        - Utiliser uniquement des sources officielles UE  
        - Inclure des références spécifiques aux articles  
        - Fournir les dates de publication et d'entrée en vigueur  
        - Vérifier la cohérence entre les sources  
    """
```

Cette méthode isole, dans une fonction séparée, le texte long du prompt système envoyé à l'agent, ce qui rend le code de configuration plus lisible. Le contenu impose une méthodologie d'analyse en cinq étapes (profil de l'entreprise, cartographie réglementaire, évaluation des risques par criticité, domaines réglementaires couverts, structure du rapport final) et répète explicitement dans la "stratégie de recherche" de n'utiliser que des sources officielles UE avec des références précises aux articles et aux dates — un rappel redondant avec la restriction de domaine technique vue à l'étape précédente, qui renforce à deux niveaux (technique et instruction) la fiabilité des

réponses produites.

Etape 4 — Lancement de l'analyse pour une entreprise

Lignes 114-141

```
def analyser_entreprise(self, nom_entreprise: str, secteur: str,
                        contexte_supplementaire: str = "",
                        chemin_markdown: str = "conformite_automobile.md") -> str:
    """Analyse les exigences de conformité pour une entreprise spécifique"""

    requete = f"""
    Effectuez une analyse complète de conformité réglementaire UE pour :

    Entreprise : {nom_entreprise}
    Secteur : {secteur}
    Contexte supplémentaire : {contexte_supplementaire}

    Identifiez tous les règlements UE applicables, leurs exigences de conformité,
    les délais de mise en œuvre et les impacts potentiels sur l'activité.
    Fournissez des conseils de conformité spécifiques et actionnables.
    """

    print(f" Début de l'analyse de conformité pour {nom_entreprise}...")
    # print_response() n'imprime que dans la console et ne retourne rien
    # d'exploitable ; run() donne accès au contenu pour le sauvegarder en
    # markdown, nécessaire pour l'étape de génération du rapport Word.
    reponse = self.agent.run(requete)
    print(reponse.content)

    with open(chemin_markdown, "w", encoding="utf-8") as fichier:
        fichier.write(reponse.content)

    return reponse.content
```

Cette méthode construit dynamiquement la requête envoyée à l'agent à partir du nom de l'entreprise, de son secteur et d'un contexte libre, puis appelle `self.agent.run(...)` plutôt que la méthode `print\_response()` habituellement utilisée pour un simple affichage console : `run()` renvoie un objet contenant le texte de la réponse, indispensable ici puisque ce texte doit être écrit dans un fichier Markdown pour être ensuite converti en document Word. C'est ce fichier Markdown, sauvegardé en UTF-8, qui fait le pont entre la partie "génération de contenu par l'IA" et la partie "mise en forme du document" du script.

Etape 5 — Orchestration de la génération du rapport Word et page de titre

Lignes 143-194

```
def generer_rapport_word(self, chemin_markdown: str,
                        chemin_sortie: Optional[str] = None) -> str:
    """Convertit un rapport markdown en document Word formaté"""

    # Lecture du contenu markdown
    try:
        with open(chemin_markdown, "r", encoding="utf-8") as fichier:
            contenu = fichier.read()
    except FileNotFoundError:
        raise FileNotFoundError(f"Fichier markdown introuvable : {chemin_markdown}")

    # Initialisation du document Word
    doc = Document()

    # Ajout de la page de titre
    self._ajouter_page_titre(doc)

    # Traitement du contenu
    self._traiter_contenu_markdown(doc, contenu)

    # Ajout du pied de page
    self._ajouter_pied_document(doc)
```

```

# Sauvegarde du document
if not chemin_sortie:
    horodatage = datetime.now().strftime("%Y%m%d_%H%M%S")
    chemin_sortie = f"Rapport_Conformite_UE_{horodatage}.docx"

doc.save(chemin_sortie)
print(f" Rapport Word sauvegardé : {chemin_sortie}")

return chemin_sortie

def _ajouter_page_titre(self, doc: Document):
    """Ajoute une page de titre professionnelle"""
    # Titre
    titre = doc.add_heading("Analyse de Conformité Réglementaire UE", 0)
    titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER

    # Sous-titre
    sous_titre = doc.add_paragraph("Rapport d'Intelligence Réglementaire")
    sous_titre.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
    sous_titre_format = sous_titre.runs[0].font
    sous_titre_format.size = Pt(14)
    sous_titre_format.italic = True

    # Date
    date_para = doc.add_paragraph(f"Généré le : {datetime.now().strftime('%d %B %Y')}")
    date_para.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER

    # Saut de page
    doc.add_page_break()

```

`generer\_rapport\_word` est la méthode chef d'orchestre de la conversion Markdown vers Word : elle relit le fichier Markdown généré précédemment, crée un document Word vierge avec python-docx, puis appelle dans l'ordre les méthodes qui ajoutent la page de titre, le corps du rapport et le pied de page, avant de sauvegarder le fichier avec un nom horodaté si aucun chemin de sortie n'est précisé. `\_ajouter\_page\_titre` construit une première page professionnelle centrée (titre, sous-titre en italique, date du jour) suivie d'un saut de page, pour que le rapport ait l'apparence d'un vrai livrable d'entreprise plutôt que d'un simple export brut.

Etape 6 — Conversion du contenu Markdown en éléments Word formatés

[Lignes 196-257](#)

```

def _traiter_contenu_markdown(self, doc: Document, contenu: str):
    """Traite le contenu markdown et l'ajoute au document Word"""
    lignes = contenu.splitlines()

    for ligne in lignes:
        ligne = ligne.strip()
        if not ligne:
            continue

        # En-têtes
        if ligne.startswith("# "):
            doc.add_heading(ligne[2:], level=1)
        elif ligne.startswith("## "):
            doc.add_heading(ligne[3:], level=2)
        elif ligne.startswith("### "):
            doc.add_heading(ligne[4:], level=3)
        elif ligne.startswith("#### "):
            doc.add_heading(ligne[5:], level=4)

        # Listes
        elif ligne.startswith("* ") or ligne.startswith("- "):
            self._ajouter_puce(doc, ligne[2:].strip())
        elif re.match(r'^\d+\.', ligne):
            self._ajouter_numero(doc, re.sub(r'^\d+\.', '', ligne))

```

```

        # Paragraphes normaux
        else:
            self._ajouter_paragraphe_formate(doc, ligne)

def _ajouter_puce(self, doc: Document, texte: str):
    """Ajoute une puce formatée"""
    p = doc.add_paragraph(style="List Bullet")
    self._ajouter_texte_formate(p, texte)

def _ajouter_numero(self, doc: Document, texte: str):
    """Ajoute un élément numéroté"""
    p = doc.add_paragraph(style="List Number")
    self._ajouter_texte_formate(p, texte)

def _ajouter_paragraphe_formate(self, doc: Document, texte: str):
    """Ajoute un paragraphe avec formatage"""
    p = doc.add_paragraph()
    self._ajouter_texte_formate(p, texte)

def _ajouter_texte_formate(self, paragraphe, texte: str):
    """Ajoute du texte avec formatage gras/italique"""
    # Traitement du texte en gras (**texte**)
    parties = re.split(r"(\*\.?*\*)", texte)
    for partie in parties:
        if partie.startswith("***") and partie.endswith("***"):
            run = paragraphe.add_run(partie[2:-2])
            run.bold = True
        else:
            # Traitement du texte en italique (*texte*)
            parties_italique = re.split(r"(\.?*\.?)", partie)
            for partie_italique in parties_italique:
                if (partie_italique.startswith("(") and partie_italique.endswith(")")
                    and not partie_italique.startswith("***")):
                    run = paragraphe.add_run(partie_italique[1:-1])
                    run.italic = True
            else:
                paragraphe.add_run(partie_italique)

```

C'est le cœur technique de la conversion : le Markdown n'est pas transformé par une bibliothèque toute faite, mais analysé ligne par ligne à la main. Chaque ligne est testée contre des motifs simples (préfixe `#`, `##`, `###`, `####` pour les titres, `\*/`-` ` pour les puces, une expression régulière `^\d+\. ` pour les listes numérotées) afin de choisir l'élément Word correspondant (`add\_heading`, style "List Bullet", style "List Number", paragraphe normal). Le texte de chaque ligne passe ensuite par `\_ajouter\_texte\_formate`, qui découpe la chaîne autour des motifs `\*\*gras\*\*` et `\*italique\*` avec des expressions régulières pour appliquer le bon style à chaque fragment (`run.bold`, `run.italic`) plutôt que d'écrire le texte tel quel avec ses astérisques visibles. C'est un mini-parseur Markdown artisanal, suffisant pour les éléments que le prompt système demande à l'agent de produire (titres, listes, gras).

Etape 7 — Pied de document et point d'entrée du script

Lignes 259-307

```

def _ajouter_pied_document(self, doc: Document):
    """Ajoute un pied de document avec métadonnées"""
    doc.add_page_break()
    doc.add_heading("Informations du Document", level=2)

    infos_pied = [
        f"Généré le : {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}",
        "Source : Sources officielles UE (eur-lex.europa.eu, ec.europa.eu)",
        "Avertissement : Ce rapport est à des fins d'information uniquement.",
        "Recommandation : Consultez un conseil juridique pour des questions de conformité"
        "spécifiques."
    ]

    for info in infos_pied:
        p = doc.add_paragraph(info)

```

```

        p.style = "Caption"

def main():
    """Fonction d'exécution principale"""
    try:
        # Initialisation de l'analyseur
        analyseur = AnalyseurConformiteUE()

        # Exemple d'utilisation
        nom_entreprise = "Renault"
        secteur = "Industrie Automobile"
        contexte_supplementaire = "Focus sur les systèmes IA, réglementations d'émissions et reporting de durabilité"

        # Lancement de l'analyse
        print(" Début de l'analyse de conformité UE...")
        reponse = analyseur.analyser_entreprise(
            nom_entreprise=nom_entreprise,
            secteur=secteur,
            contexte_supplementaire=contexte_supplementaire
        )

        # Conversion en Word (en supposant qu'un fichier markdown a été généré)
        chemin_markdown = "conformite_automobile.md"
        if Path(chemin_markdown).exists():
            chemin_rapport_word = analyseur.generer_rapport_word(chemin_markdown)
            print(f" Analyse terminée ! Rapport sauvegardé : {chemin_rapport_word}")
        else:
            print(" Fichier markdown introuvable. Veuillez vérifier le chemin.")

    except Exception as e:
        print(f" Erreur lors de l'analyse : {e}")
        raise

if __name__ == "__main__":
    main()

```

`\_ajouter\_pied\_document` termine le rapport par une page de métadonnées (date de génération, rappel des sources officielles utilisées, et surtout un avertissement explicite invitant à consulter un vrai conseil juridique) — une précaution logique compte tenu du sujet traité, qui rappelle que le rapport généré par l'IA reste une aide et non un avis juridique engageant. La fonction `main()` sert d'exemple concret bout en bout : elle instancie la classe, lance une analyse de conformité pour l'entreprise fictive "Renault" dans le secteur automobile avec un contexte spécifique (IA embarquée, émissions, durabilité), puis, si le fichier Markdown a bien été créé, déclenche la conversion en document Word — le tout entouré d'un bloc `try/except` qui affiche une erreur lisible et relance l'exception plutôt que d'échouer silencieusement.

Projet 31 — 31-Procurement-agent

Recherche de fournisseurs pour une liste de produits

LE BUT DE CE PROJET

Le but de ce projet est d'automatiser une tâche que fait normalement un acheteur ou un responsable achats à la main : chercher sur internet des fournisseurs pour une liste de produits, comparer leurs prix, leurs délais de livraison et leurs conditions, puis choisir le meilleur. Le problème que ça résout : quand une entreprise veut importer ou acheter en gros un produit (par exemple des bananes ou des pommes), il n'existe pas de moteur de recherche dédié qui liste tous les fournisseurs avec leurs prix et conditions — il faut ouvrir des dizaines de sites, lire des pages entières, et noter à la main les informations utiles dans un tableau, ce qui prend des heures et est source d'erreurs. Ce projet automatise tout le pipeline : un premier composant (Exa, un moteur de recherche pensé pour l'IA) va chercher sur le web des pages de fournisseurs et en récupère le contenu texte, puis un agent basé sur un modèle de langage (GPT-4o) lit tout ce texte brut, en extrait les informations structurées (nom du produit, prix, devise, site web, délai de livraison...), les écrit lui-même dans un fichier CSV en générant et en exécutant du code Python, puis rédige un résumé exécutif qui recommande le meilleur fournisseur par produit avec ses raisons. Ça illustre un concept clé des agents IA : donner à un modèle de langage la capacité d'agir (chercher sur le web, écrire et exécuter du code) plutôt que de se contenter de répondre en texte, ce qui transforme un simple chatbot en un assistant capable de produire un vrai livrable exploitable (le fichier CSV) sans intervention humaine.

31-Procurement-agent/procurement_agent.py

Workflow agno qui cherche des fournisseurs via l'API Exa, puis délègue l'extraction des données et la génération du CSV à un agent LLM outillé de PythonTools.

Etape 1 — Imports et configuration de la clé Exa

Lignes 1-20

```
import os
from exa_py import Exa
from textwrap import dedent
from typing import Iterator
from dotenv import load_dotenv
from agno.workflow import Workflow
from agno.agent import Agent, RunResponse
from agno.models.openai import OpenAIChat
from agno.tools.python import PythonTools
from agno.utils.pprint import pprint_run_response

load_dotenv()

### Exa Setup ###
exa_api_key = os.environ.get('EXA_API_KEY')

if not exa_api_key:
    raise ValueError("EXA_API_KEY environment variable is required")

exa = Exa(api_key=exa_api_key)
```

Cette section pose les fondations du script : les imports rassemblent les briques nécessaires (dotenv pour les secrets, agno pour construire l'agent et le workflow, exa_py pour la recherche web, PythonTools pour donner à l'agent la capacité d'exécuter du code). La clé API Exa est chargée depuis un fichier .env plutôt qu'écrite en dur dans le code, une pratique standard pour ne jamais exposer de secrets dans un dépôt Git, et le script s'arrête immédiatement avec un message clair si elle est absente, ce qui évite un échec cryptique plus tard lors du premier appel réseau.

Etape 2 — Construction et exécution de la recherche Exa

Lignes 22-39

```
# Exa search function using basic search and contents
def exa_search(product_list: str, location: str):
    """
    Search for products and vendors using Exa's search and contents functionality
    """
    try:
        # Create a comprehensive search query
        search_query = f"vendors suppliers {product_list} {location} prices purchasing procurement"

        print(f"Searching for: {search_query}")

        # Perform search and get contents
        results = exa.search_and_contents(
            query=search_query,
            num_results=15, # Get more results for better coverage
            text=True, # Include full text content
            type="auto" # Use auto type for best results
        )
```

Cette étape construit dynamiquement une requête de recherche en ajoutant des mots-clés métier (vendors, suppliers, prices, procurement) autour des produits et de la localisation fournis par l'utilisateur : cette technique aide le moteur à cibler des pages commerciales pertinentes plutôt que des résultats génériques. L'appel à `search_and_contents` combine en une seule requête la recherche web et la récupération du texte complet des pages trouvées, ce qui évite d'avoir à scraper chaque site séparément ; demander 15 résultats avec du texte complet vise à maximiser la couverture d'information que l'agent LLM pourra ensuite analyser.

Etape 3 — Mise en forme Markdown des résultats

Lignes 41-72

```
        # Format the results for the procurement agent
        formatted_results = f"""
# Procurement Research Results for {location}

**Products searched:** {product_list}
**Location:** {location}
**Search performed:** {search_query}

## Search Results:

"""

        for i, result in enumerate(results.results, 1):
            formatted_results += f"""
### Result {i}: {result.title or 'No title'}

**URL:** {result.url}
**Author:** {result.author or 'Not specified'}
**Published Date:** {result.published_date or 'Not specified'}
**Score:** {result.score or 'N/A'}

**Content Preview:**
{result.text[:1500] if result.text else 'No content available'}...

---
"""
```

```

        print("----- EXA SEARCH COMPLETION -----")
        print(f"Found {len(results.results)} results")
        print("-----")
    )

    return formatted_results

```

Les résultats bruts renvoyés par Exa sont transformés en un document Markdown structuré (titres, gras, séparateurs) plutôt que transmis tels quels : présenter l'information de façon organisée facilite grandement sa lecture et son interprétation par le modèle de langage qui la recevra ensuite comme contexte. Le texte de chaque page est volontairement tronqué à 1500 caractères pour limiter la taille totale transmise au LLM (qui a une fenêtre de contexte et un coût par token limités) tout en gardant l'essentiel de l'information utile à l'extraction.

Etape 4 — Repli (fallback) en cas d'échec de la recherche

Lignes 74-115

```

except Exception as e:
    print(f"Error in exa_search: {str(e)}")
    # Fallback to a simpler search
    try:
        print("Trying fallback search...")
        results = exa.search(
            query=f"{product_list} suppliers {location}",
            num_results=10
        )

        fallback_results = f"""
# Fallback Search Results for {location}

**Products searched:** {product_list}
**Location:** {location}

## Basic Search Results:

"""

        for i, result in enumerate(results.results, 1):
            fallback_results += f"""
### Result {i}: {result.title or 'No title'}
- **URL:** {result.url}
- **Score:** {result.score or 'N/A'}
- **Published:** {result.published_date or 'Not specified'}

"""

        return fallback_results

    except Exception as fallback_error:
        print(f"Fallback search also failed: {str(fallback_error)}")
        return f"""
# Search Error

Unable to perform search for products: {product_list} in location: {location}

Error details: {str(e)}

Please check your Exa API key and try again.
"""

```

Cette étape met en place une robustesse à deux niveaux : si la recherche riche (avec récupération de contenu) échoue, le code retente une recherche plus simple (sans contenu, moins gourmande) avant d'abandonner. Si même ce repli échoue, la fonction ne plante pas et ne renvoie pas non plus une chaîne vide incompréhensible : elle retourne un message d'erreur structuré expliquant le problème, que l'agent LLM pourra lire et relayer à l'utilisateur de façon claire plutôt que de faire planter tout le programme.

Etape 5 — Définition de l'agent et de ses instructions d'extraction

Lignes 117-167

```
class ProcurementAgent(Workflow):
    procurement_agent: Agent = Agent(
        name='Procurement Agent',
        model=OpenAIChat(id='gpt-4o'),
        instructions=dedent("""
You are a procurement analysis agent.

Your goal is to:
1. Parse the search results from Exa containing information about vendors and suppliers.
2. For each relevant product/vendor combination found, extract the following fields:
    - Product Name
    - Vendor Name
    - Product Title
    - Price (if mentioned, convert to numeric if possible)
    - Currency (if mentioned)
    - Vendor Website or Purchase Link
    - Short Product Description
    - Minimum Order Quantity (if available)
    - Shipping Time (if available)
    - Bulk Discounts or Deals (if available)
    - Vendor Location (if mentioned)

3. After extracting the data, write and execute a Python script that creates a file named
`data.csv`
    - Use the standard `csv` module
    - Make the columns in this order: Product Name, Vendor Name, Product Title, Price, Currency,
Bulk Discounts or Deals, Vendor Website, Short Product Description, Minimum Order Quantity, Shipping
Time, Vendor Location
    - The script should write a header row followed by one row per vendor/product combination
    - IMPORTANT: When opening the file, always use encoding='utf-8' in open(), e.g.
open('data.csv', 'w', newline='', encoding='utf-8')
    - Then use the PythonTools tool to run the script and save the data
    - Create rows for each relevant data point found in the search results

Then:

4. Analyze the data across all products and vendors.
    - Compare vendors based on pricing, shipping times, minimum quantities, and available deals.
    - Prioritize vendors who:
        - Deliver to the specified location
        - Offer competitive prices
        - Have favorable shipping times or bulk deals
        - Appear reliable (from established marketplaces or verified sellers)

5. Write an executive summary that includes:
    - Recommended vendor(s) per product (if clear recommendations can be made)
    - Reasons for the recommendation (price, location, delivery, etc.)
    - Any noteworthy observations (e.g. pricing differences, available options)
    - Limitations of the search results (if search quality was limited)

IMPORTANT:
    - Use PythonTools to write the extracted data to `data.csv`.
    - If search results are limited or don't contain specific pricing/vendor info, acknowledge this
in your analysis.
    - Focus on extracting whatever relevant information is available from the search results.
"""),
```

ProcurementAgent hérite de Workflow, la classe agno qui permet d'orchestrer plusieurs étapes (recherche puis analyse) en code Python classique plutôt que de tout confier au raisonnement libre d'un seul agent. Le cœur de cette étape est le prompt système donné à procurement_agent (modèle GPT-4o) : il détaille précisément les champs à extraire, l'ordre exact des colonnes du futur CSV, et impose l'usage de l'encodage utf-8 pour éviter les erreurs avec des caractères spéciaux issus du web. C'est cette instruction qui pousse l'agent à réellement écrire et exécuter un script Python via PythonTools, plutôt que de simplement décrire ce qu'il faudrait faire.

Etape 6 — Format de sortie attendu (expected_output)

Lignes 168-199

```
expected_output=dedent("""
The output should include:

1. Data Summary:
- Number of relevant results processed
- Total vendors/suppliers identified
- Location considered for delivery: <city>, <country>
- Data quality assessment and any limitations

2. Recommendations Per Product:
For each product found (e.g., "banane", "pomme"), provide:

### Product: <Product Name>

**Available Options Found:**
- List the vendors/suppliers found with their key details

**Best Option (if determinable):** <Vendor Business Name>
**Why Recommended:**
- Reason 1 (e.g. local supplier, competitive pricing mentioned)
- Reason 2 (e.g. established marketplace presence)
- Reason 3 (e.g. good shipping options or bulk availability)

**Alternative Options:**
- Mention other viable vendors if found

**Data Quality Note:**
- Acknowledge any limitations in the search results
- Mention if specific pricing or delivery info was not available

IMPORTANT: Confirm that the CSV file was created and contains the extracted data.
"""),
```

En plus des instructions de traitement, l'agent reçoit un gabarit exact du format de sortie attendu (résumé des données, recommandations par produit, alternatives, limites des données). Donner explicitement ce squelette est une technique de prompt engineering qui augmente fortement la cohérence et la lisibilité des réponses générées d'une exécution à l'autre, au lieu de laisser le modèle improviser librement sa mise en forme.

Etape 7 — Outils de l'agent et méthode run() du workflow

Lignes 200-207

```
tools=[PythonTools()],
show_tool_calls=True,
markdown=True
)

def run(self, product_list: str, location: str):
    research_response = exa_search(product_list, location)
    yield from self.procurement_agent.run(research_response, stream=True)
```

`tools=[PythonTools()]` est la ligne qui donne concrètement à l'agent la capacité d'agir : sans elle, il ne pourrait que décrire le CSV à créer, jamais l'écrire réellement sur le disque. La méthode `run()` est le point d'entrée du workflow : elle enchaîne d'abord la recherche Exa, puis transmet les résultats formatés à l'agent LLM avec `stream=True`, ce qui fait que la réponse (l'analyse, le CSV, le résumé) arrive morceau par morceau au fur et à mesure de sa génération plutôt que d'un bloc à la fin, donnant une sensation de réactivité à l'utilisateur.

Etape 8 — Point d'entrée en ligne de commande

Lignes 209-229

```
if __name__ == '__main__':
    from rich.prompt import Prompt

    # Check if API key is available
```

```

if not os.environ.get('EXA_API_KEY'):
    print(" Error: EXA_API_KEY environment variable is not set.")
    print("Please set your Exa API key in your .env file:")
    print("EXA_API_KEY=your_api_key_here")
    exit(1)

product_list = Prompt.ask('Enter your products list separated by commas')
location = Prompt.ask('Enter your business location (city, country)')

if product_list and location:
    try:
        workflow = ProcurementAgent()
        response: Iterator[RunResponse] = workflow.run(product_list=product_list, location=location)
        pprint_run_response(response, markdown=True)
    except Exception as e:
        print(f" Error running procurement agent: {str(e)}")
        print("Please check your API keys and try again.")

```

Cette dernière section transforme le script en petit outil en ligne de commande utilisable directement : elle vérifie la présence de la clé API, demande interactivement à l'utilisateur sa liste de produits et sa localisation via des invites soignées (rich.prompt), puis instancie et lance le workflow en affichant le résultat en streaming avec une mise en forme Markdown. Le tout est protégé par un bloc try/except final pour afficher un message d'erreur clair plutôt qu'une trace d'erreur brute si quelque chose échoue.

31-Procurement-agent/create_data_csv.py

Script autonome qui génère un fichier data.csv d'exemple à partir de données de fournisseurs codées en dur, servant de jeu de données de démonstration indépendant de l'agent de recherche.

Etape 1 — Import du module CSV

Lignes 1-1

```
import csv
```

Le script importe uniquement le module standard csv, qui fournit des outils pour écrire des fichiers CSV en gérant correctement l'échappement des virgules, guillemets et retours à la ligne dans les champs, sans dépendance externe puisqu'aucun appel réseau n'est nécessaire ici.

Etape 2 — Données de démonstration codées en dur

Lignes 3-56

```

data = [
    {
        "Product Name": "Bananas",
        "Vendor Name": "United States",
        "Product Title": "Bananas",
        "Price": 1.50,
        "Currency": "USD",
        "Vendor Website": "https://www.e-fresco.io/bananas-4dqa.html",
        "Short Product Description": "Bananas for retailers, available through an online platform.",
        "Minimum Order Quantity": "",
        "Shipping Time": "",
        "Bulk Discounts or Deals": "",
        "Vendor Location": "United States"
    },
    {
        "Product Name": "",
        "Vendor Name": "Wali Trading",
        "Product Title": "",
        "Price": "",
        "Currency": "",
        "Vendor Website": "https://wali-trading.com/",
        "Short Product Description": "Offers a wide range of fruits, including bananas, from Morocco.",
        "Minimum Order Quantity": "",

```

```

        "Shipping Time": "",
        "Bulk Discounts or Deals": "",
        "Vendor Location": "Morocco"
    },
    {
        "Product Name": "",
        "Vendor Name": "Zenkai Trading SARL",
        "Product Title": "",
        "Price": "",
        "Currency": "",
        "Vendor Website": "https://www.zenkaltrading.com/",
        "Short Product Description": "Global exporter of fresh fruits and vegetables, offering various
products from Morocco.",
        "Minimum Order Quantity": "",
        "Shipping Time": "",
        "Bulk Discounts or Deals": "",
        "Vendor Location": "Casablanca, MA"
    },
    {
        "Product Name": "Pomme products",
        "Vendor Name": "Pomme de Pain Maroc",
        "Product Title": "",
        "Price": "",
        "Currency": "",
        "Vendor Website": "https://www.linkedin.com/company/pomme-de-pain-maroc",
        "Short Product Description": "Offers a variety of pomme (apple) products along with other fresh
items.",
        "Minimum Order Quantity": "",
        "Shipping Time": "",
        "Bulk Discounts or Deals": "",
        "Vendor Location": "Casablanca, MA"
    }
]

```

Cette étape définit une liste de quatre dictionnaires Python, chacun représentant une future ligne du CSV (un couple produit/fournisseur), avec des champs volontairement laissés vides quand l'information équivalente ne serait pas disponible dans un vrai résultat de recherche. Ces données fixes ne proviennent pas d'une recherche en temps réel : elles servent de jeu d'exemple ou de secours pour tester rapidement le format du CSV attendu sans dépendre de l'API Exa.

Etape 3 — Ouverture du fichier et préparation du writer CSV

Lignes 58-61

```

# Create CSV
with open('data.csv', 'w', newline='', encoding='utf-8') as csvfile:
    fieldnames = ["Product Name", "Vendor Name", "Product Title", "Price", "Currency", "Bulk Discounts or
Deals", "Vendor Website", "Short Product Description", "Minimum Order Quantity", "Shipping Time", "Vendor
Location"]
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

```

Le fichier data.csv est ouvert en écriture avec `newline=""` (recommandation standard du module csv pour éviter des lignes vides parasites sous Windows) et l'encodage utf-8 pour gérer correctement les caractères spéciaux. `csv.DictWriter` est utilisé plutôt qu'un simple `writer` : il sait écrire directement des dictionnaires Python comme des lignes de CSV en respectant l'ordre de colonnes donné par `fieldnames`, ce qui évite d'avoir à convertir manuellement chaque dictionnaire en liste ordonnée de valeurs.

Etape 4 — Écriture de l'en-tête et des lignes de données

Lignes 63-65

```

writer.writeheader()
for entry in data:
    writer.writerow(entry)

```

La dernière étape écrit d'abord la ligne d'en-tête à partir des noms de colonnes, puis parcourt chaque dictionnaire de la liste data pour l'écrire comme une ligne du fichier, chaque valeur étant placée automatiquement sous la colonne correspondant à sa clé. Le résultat est un fichier data.csv exploitable directement dans un tableur, avec la même structure que celui que produirait l'agent de recherche.

Projet 32 — 32-Advance-Research-agent

Assistant de recherche academique avec base de connaissances

LE BUT DE CE PROJET

Le but de ce projet est de transformer un article de recherche scientifique au format PDF (potentiellement long, technique et parfois même scanné) en un assistant interactif capable de le résumer et de répondre à des questions précises dessus, en citant systématiquement la source exacte. Le problème que ça résout : lire et comprendre un papier académique complet prend du temps, et un modèle de langage classique, interrogé directement sur un document qu'il n'a jamais vu, soit ne peut pas répondre, soit invente une réponse plausible mais fausse (une hallucination) car il n'a aucun accès au contenu réel du PDF. La solution technique employée ici s'appelle le RAG (Retrieval-Augmented Generation, ou génération augmentée par récupération) : au lieu de faire confiance uniquement à ce que le modèle a appris pendant son entraînement, on découpe le document en morceaux, on transforme chaque morceau en un vecteur numérique qui capture son sens (l'embedding), et on stocke ces vecteurs dans une base de données spécialisée (ici PostgreSQL avec l'extension pgvector). Quand l'utilisateur pose une question, sa question est elle aussi transformée en vecteur, ce qui permet de retrouver mathématiquement les passages du document les plus proches en sens, de les injecter dans le contexte du modèle, et de le forcer à répondre en s'appuyant sur ces extraits réels plutôt qu'en inventant. Concrètement, le pipeline complet est : upload d'un PDF dans une interface web (Streamlit), OCR (reconnaissance optique) du PDF via l'API Mistral pour en extraire le texte même s'il est scanné, indexation de ce texte dans la base vectorielle, génération automatique d'un résumé, puis mise à disposition d'un agent de questions-réponses qui cherche d'abord dans le document et, seulement s'il ne trouve rien, va chercher sur des sources externes (Semantic Scholar, puis le web) en citant toujours sa source.

32-Advance-Research-agent/research_assistant_agent.py

Application Streamlit qui fait l'OCR d'un PDF de recherche via Mistral, indexe le contenu dans une base de connaissances vectorielle (RAG) et expose un agent de résumé ainsi qu'un agent de questions-réponses consultant cette base.

Etape 1 — Reconfiguration de l'encodage et désactivation de TensorFlow

Lignes 1-13

```
import sys

for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import os

# sentence-transformers (utilisé pour les embeddings locaux) importe transformers,
# qui essaie par défaut de charger TensorFlow en plus de PyTorch. La version de
# TensorFlow installée sur cette machine est incompatible avec la version de
# protobuf présente : on désactive TF avant tout import de la lib.
os.environ.setdefault("USE_TF", "0")
```

Avant même les imports métier, le script sécurise l'environnement d'exécution. La reconfiguration de stdout/stderr en utf-8 avec remplacement des caractères non décodables évite des plantages liés à l'encodage sur un terminal Windows quand du texte OCR ou des réponses de modèle contiennent des caractères spéciaux. La variable USE_TF=0 empêche la librairie transformers (utilisée en interne par l'embedder local) de tenter de charger TensorFlow, qui est incompatible avec la version de protobuf installée sur la machine — ce réglage doit impérativement précéder tout import de ces bibliothèques.

Etape 2 — Imports du framework agent et configuration des clés

Lignes 15-34

```
import requests
import streamlit as st
from textwrap import dedent
from agno.agent import Agent
from mistralai.client import Mistral
from agno.models.groq import Groq
from agno.vectordb.pgvector import PgVector
from agno.tools.reasoning import ReasoningTools
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.knowledge.knowledge import Knowledge
from agno.knowledge.embedder.sentence_transformer import SentenceTransformerEmbedder
from dotenv import load_dotenv
load_dotenv()

mistral_key = os.getenv("MISTRAL_API_KEY")
# Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq
# pour le chat (clé GROQ_API_KEY déjà disponible) et par un embedder local
# (sentence-transformers) pour les embeddings, gratuit et sans limite de débit.
groq_key = os.getenv("GROQ_API_KEY")
client = Mistral(api_key=mistral_key)
```

Cette étape rassemble toutes les briques de l'application : Streamlit pour l'interface web, le client Mistral pour l'OCR, Groq comme modèle de chat rapide et gratuit, PgVector comme base de données vectorielle, ReasoningTools pour structurer le raisonnement de l'agent, DuckDuckGoTools pour la recherche web de secours, et surtout Knowledge + SentenceTransformerEmbedder qui forment ensemble le cœur du système RAG (stockage vectoriel + génération d'embeddings locaux, gratuits et sans limite de débit contrairement à une API payante). Les clés API sont chargées depuis .env plutôt qu'écrites en dur dans le code.

Etape 3 — Fonction ocr_pdf — conversion du PDF en Markdown via Mistral

Lignes 37-63

```
def ocr_pdf(pdf_path):
    # check if the file exists
    if not os.path.exists(pdf_path):
        st.error("PDF file not found")
        return

    uploaded_pdf = client.files.upload(
        file={
            "file_name": pdf_path,
            "content": open(pdf_path, "rb"),
        },
        purpose="ocr"
    )

    signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = client.ocr.process(
        model="mistral-ocr-latest",
        document={
            "type": "document_url",
            "document_url": signed_url.url,
        },
        include_image_base64=True
    )
```

```
with open(f"DocumentMarkdown/ocr_document.md", "w", encoding="utf-8") as f:
    f.write("\n".join([page.markdown for page in ocr_response.pages]))
```

Cette fonction convertit le PDF en texte exploitable en trois temps : upload du fichier vers l'API Mistral (obligatoire avant tout traitement), récupération d'une URL signée temporaire pointant vers ce fichier, puis lancement du traitement OCR proprement dit avec le modèle spécialisé mistral-ocr-latest, capable de reconnaître texte et mise en page même dans un PDF scanné. Le résultat, découpé par page, est rassemblé en un seul fichier Markdown sauvegardé sur disque : c'est ce fichier qui sera indexé à l'étape suivante et qui sert de socle à tout le système RAG.

Etape 4 — knowledge_base_setup — indexation vectorielle (RAG)

[Lignes 65-81](#)

```
def knowledge_base_setup():
    """ Setup Knowledge Base """
    # MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
    # unifiée Knowledge : le contenu se charge via add_content(path=...) plutôt
    # qu'un argument du constructeur + .load(recreate=...).
    knowledge_base = Knowledge(
        vector_db=PgVector(
            table_name="markdown_documents",
            db_url="postgresql+psycopg://ai:ai@localhost:5532/ai",
            embedder=SentenceTransformerEmbedder(),
        ),
        # La valeur par défaut (10 documents) fait dépasser la limite Groq
        # (8000 tokens/minute en tier gratuit) en un seul appel.
        max_results=3,
    )
    knowledge_base.add_content(path="DocumentMarkdown/ocr_document.md")
    return knowledge_base
```

C'est ici que se met en place le mécanisme de RAG : PgVector configure PostgreSQL (via son extension pgvector) comme base vectorielle, chargée de stocker les morceaux du document sous forme de vecteurs numériques (embeddings) générés localement par SentenceTransformerEmbedder. L'appel add_content() découpe automatiquement le fichier Markdown en morceaux (chunking), calcule leur embedding, et les stocke dans PgVector — rendant le document interrogeable par similarité de sens plutôt que par simple recherche de mots-clés. max_results=3 limite volontairement le nombre de passages renvoyés par recherche, pour rester sous la limite de tokens par minute du tier gratuit de Groq.

Etape 5 — Outil de recherche académique externe (Semantic Scholar)

[Lignes 83-88](#)

```
def semantic_scholar_search(query):
    url = "https://api.semanticscholar.org/graph/v1/paper/search"
    params = {"query": query}
    headers = {"x-api-key": "1234567890"}
    response = requests.get(url, params=params, headers=headers)
    return response.json()
```

Cette fonction Python simple, de par sa signature explicite, peut être directement passée à un agent comme "outil" (function tool) que le modèle de langage peut décider d'appeler lui-même. Elle interroge l'API publique de Semantic Scholar, un moteur de recherche académique gratuit, et sert de première ligne de repli quand la base de connaissances interne (limitée au PDF indexé) ne contient pas la réponse à une question.

Etape 6 — summary_agent — agent de résumé automatique

[Lignes 90-105](#)

```
def summary_agent():
    knowledge_base = knowledge_base_setup()
    summary_agent = Agent(
        name="Summary Agent",
        model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
        instructions=dedent("""
```

```

        You are a summary agent designed to summarize the document.
        Read the entire document stored in the knowledge base (in markdown format) and produce a concise
summary in plain English. Keep it focused on the paper's objective, methodology, and key findings.
        """),
        knowledge=knowledge_base,
        search_knowledge=True,
        # Sans limite, l'agent relance plusieurs recherches pour "tout lire", ce qui
        # accumule le contexte et dépasse la limite Groq (8000 tokens/minute).
        tool_call_limit=1,
    )
    return summary_agent

```

Cette fonction construit un agent dédié exclusivement au résumé : il utilise le modèle Groq openai/gpt-oss-20b (choisi pour son quota plus généreux sur le tier gratuit) et reçoit la base de connaissances via `search_knowledge=True`, qui active la récupération automatique de passages pertinents avant de répondre. `tool_call_limit=1` est un réglage important : sans lui, l'agent relancerait plusieurs recherches successives pour tenter de "tout lire", ce qui accumulerait du contexte et dépasserait la limite de tokens par minute de Groq.

Etape 7 — agent_setup — agent de questions-réponses avec repli externe

[Lignes 107-141](#)

```

def agent_setup():
    knowledge_base = knowledge_base_setup()
    research_agent = Agent(
        name="Research Agent",
        model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
        instructions=dedent("""
        You are a research assistant designed to simplify and answer questions about academic papers.
        Your behavior follows this logic:

        1. Answering Questions:
        - When a user asks a question, try to answer it using the internal knowledge base first.
        - If you find an answer, cite the exact source from the markdown document (quote the relevant
sentence or paragraph).
        - Format the answer clearly and showably: include the answer, the citation snippet, and its
section or approximate heading if available.

        2. Fallback to External Tools:
        - If the knowledge base doesn't contain enough information to answer:
            - Use `semantic_shcolar_search(query)` to look for external answers.
            - If that fails, retry once.
            - If it still fails, use `GoogleSearchTool(query)` as a backup.
        - When using external tools, always show the source link or citation (title, author if
possible, and source URL).
        - Keep external answers clean, brief, and properly attributed.

        3. Formatting:
        - Always use clean markdown formatting.
        - Highlight the source of every answer clearly.
        - Avoid hallucinating—if you don't know, say you'll look it up.

        Your top priorities are accuracy, traceability (always show the source), and clarity.
        """),
        knowledge=knowledge_base,
        search_knowledge=True,
        tools=[ReasoningTools(add_instructions=True), semantic_scholar_search, DuckDuckGoTools()],
        compress_tool_results=True,
    )

    return research_agent

```

C'est l'agent principal de questions-réponses : son prompt système impose une logique en cascade — chercher d'abord dans la base de connaissances interne et citer la source exacte (extrait du document) si une réponse y est trouvée, sinon basculer vers Semantic Scholar puis vers la recherche web DuckDuckGo, en citant toujours la source externe. ReasoningTools force l'agent à décomposer explicitement son raisonnement plutôt que de répondre d'un

bloc, ce qui améliore la fiabilité sur des questions complexes, tandis que `compress_tool_results=True` réduit la taille des résultats d'outils réinjectés dans le contexte pour rester sous les quotas de Groq.

Etape 8 — Interface Streamlit et orchestration du pipeline complet

Lignes 143-186

```
if __name__ == "__main__":
    if "ocr_done" not in st.session_state:
        st.session_state.ocr_done = False

    if "knowledge_base" not in st.session_state:
        st.session_state.knowledge_base = None

    if "summary_agent" not in st.session_state:
        st.session_state.summary_agent = None

    if "agent" not in st.session_state:
        st.session_state.agent = None

    st.title("Advanced Research Assistant")
    uploaded_file = st.file_uploader("Upload your research paper", type=["pdf"])
    if uploaded_file and not st.session_state.ocr_done:
        with open(f"DocumentMarkdown/{uploaded_file.name}", "wb") as f:
            f.write(uploaded_file.read())
        with st.spinner("Performing OCR..."):
            ocr_pdf(f"DocumentMarkdown/{uploaded_file.name}")
        st.success("OCR completed successfully")
        st.session_state.ocr_done = True
        st.session_state.summary_agent = summary_agent()
        with st.spinner("Generating summary..."):
            summary_response = st.session_state.summary_agent.run("Summarize the document")
            st.write(summary_response.content)
        # st.session_state.knowledge_base = knowledge_base_setup()
        st.session_state.agent = agent_setup()

    if st.session_state.ocr_done:
        user_input = st.text_input("Enter a question: ")
        if st.button("Submit"):
            with st.spinner('Response is being generated in your terminal'):
                st.session_state.agent.print_response(
                    user_input,
                    stream=True,
                    show_full_reasoning=True,
                    stream_intermediate_steps=True
                )
            st.success('Response generated successfully')
            # response = research_agent.run(user_input)
            # st.write(response)
```

Cette dernière étape assemble tout le pipeline dans une interface web Streamlit : le `session_state` Streamlit conserve l'état entre les interactions (le PDF a-t-il déjà été traité, les agents sont-ils déjà créés) pour éviter de relancer l'OCR à chaque clic. Dès qu'un PDF est déposé, il est sauvegardé, envoyé à l'OCR, puis un résumé est généré automatiquement et affiché ; ensuite l'utilisateur peut poser librement des questions via un champ de texte, la réponse de l'agent de questions-réponses étant affichée en streaming avec son raisonnement complet visible (`show_full_reasoning=True`), ce qui rend transparent le cheminement de l'agent entre base de connaissances et sources externes.

Projet 33 — 33-FreightAgent

Estimation de couts de fret via WhatsApp

LE BUT DE CE PROJET

Le but de ce projet est d'automatiser, directement depuis WhatsApp, l'estimation du coût et du délai de transport d'une marchandise pour une petite entreprise qui importe ou exporte des produits. Le problème que ça résout : comparer les tarifs de plusieurs transporteurs (UPS, FedEx, DHL) selon le poids, les dimensions et la destination d'un colis demande normalement de consulter plusieurs grilles tarifaires complexes (souvent au format PDF) ou d'appeler chaque transporteur, ce qui est long pour une PME qui n'a pas de service logistique dédié. Ce projet met en place un petit robot conversationnel accessible directement dans WhatsApp (le canal que les gens utilisent déjà) : l'utilisateur peut soit décrire son besoin en texte, soit envoyer une simple photo du produit à expédier. Si c'est une photo, un premier agent basé sur la vision (GPT-4o mini, capable d'analyser une image) identifie automatiquement le produit, ses dimensions probables et sa matière. Cette description est ensuite transmise à un second agent spécialisé dans le fret, qui s'appuie sur une base de connaissances construite à partir de grilles tarifaires réelles de transporteurs (converties de PDF en texte via OCR, puis indexées dans une base vectorielle pour faire du RAG comme dans un moteur de recherche sémantique) et complète si besoin avec une recherche web en direct, pour produire une comparaison chiffrée des options d'expédition. Toute la mécanique de réception et d'envoi des messages WhatsApp (vérification du webhook, téléchargement des images envoyées, envoi de la réponse finale) est gérée par un petit serveur web (FastAPI) qui fait le lien entre l'API WhatsApp Business de Meta et les agents IA.

33-FreightAgent/freight_agent.py

Agent RAG spécialisé fret qui indexe des grilles tarifaires de transporteurs (OCR + base vectorielle LanceDB) et répond aux demandes d'estimation de coûts d'expédition en combinant cette base avec une recherche web DDGS.

Etape 1 — Imports et désactivation de TensorFlow

Lignes 1-19

```
import os

# sentence-transformers (utilisé pour les embeddings locaux) importe transformers,
# qui essaie par défaut de charger TensorFlow en plus de PyTorch. La version de
# TensorFlow installée sur cette machine est incompatible avec la version de
# protobuf présente : on désactive TF avant tout import de la lib.
os.environ.setdefault("USE_TF", "0")

from textwrap import dedent
from agno.agent import Agent
from mistralai.client import Mistral
from agno.models.groq import Groq
from agno.vectordb.lancedb import LanceDb, SearchType
from agno.tools.reasoning import ReasoningTools
from ddgs import DDGS
from agno.knowledge.knowledge import Knowledge
from agno.knowledge.embedder.sentence_transformer import SentenceTransformerEmbedder
```

```
from dotenv import load_dotenv
load_dotenv()
```

Comme dans le projet de recherche académique, `USE_TF=0` doit être défini avant tout import des librairies d'embeddings pour éviter un conflit de versions entre TensorFlow et protobuf sur la machine. Les imports rassemblent ensuite les briques nécessaires : Mistral pour l'OCR, Groq comme modèle de chat, LanceDb (une base vectorielle embarquée, sans serveur à installer) pour le stockage, ReasoningTools pour structurer le raisonnement, et DDGS pour la recherche web de secours.

Etape 2 — Configuration des chemins et vérification des clés API

Lignes 21-42

```
# Dossier du script : les chemins sont résolus par rapport à lui, pas au
# répertoire courant, pour que le script fonctionne peu importe d'où il est lancé.
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MARKDOWN_DIR = os.path.join(SCRIPT_DIR, "Markdown")
DOCUMENT_DIR = os.path.join(SCRIPT_DIR, "Document")
LANCEDB_URI = os.path.join(SCRIPT_DIR, "tmp", "lancedb")

mistral_key = os.getenv("MISTRAL_API_KEY")
# Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq
# pour le chat (clé GROQ_API_KEY déjà disponible) et par l'embedder Mistral
# pour les embeddings (Groq n'a pas d'API d'embeddings).
groq_key = os.getenv("GROQ_API_KEY")

if mistral_key:
    client = Mistral(api_key=mistral_key)
else:
    print("ERREUR: MISTRAL_API_KEY requis pour OCR")
    exit()

if not groq_key:
    print("ERREUR: GROQ_API_KEY requis")
    exit()
```

Les chemins des dossiers (Markdown, Document, base LanceDB) sont calculés à partir de l'emplacement du script lui-même plutôt que du répertoire courant, ce qui garantit que le script fonctionne correctement peu importe d'où il est lancé — un détail important puisque ce fichier sera importé comme module par `main.py` et `whatsapp_utils.py` situés ailleurs dans l'arborescence. Les deux clés API indispensables (Mistral pour l'OCR, Groq pour le chat) sont vérifiées explicitement au démarrage, avec arrêt immédiat et message clair si l'une d'elles manque, pour éviter un échec tardif et confus en pleine conversation WhatsApp.

Etape 3 — ocr_pdf — conversion des grilles tarifaires PDF en Markdown

Lignes 44-93

```
## OCR Function améliorée ###
def ocr_pdf(pdf_path, output_name=None):
    """Convertit un PDF en Markdown via OCR Mistral"""
    if not os.path.exists(pdf_path):
        print(f"ERREUR: PDF non trouvé - {pdf_path}")
        return False

    print(f"Traitement OCR de {pdf_path}...")

    try:
        # Upload du PDF
        uploaded_pdf = client.files.upload(
            file={
                "file_name": os.path.basename(pdf_path),
                "content": open(pdf_path, "rb"),
            },
            purpose="ocr"
        )
```

```

# Obtenir URL signée
signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

# Traitement OCR
ocr_response = client.ocr.process(
    model="mistral-ocr-latest",
    document={
        "type": "document_url",
        "document_url": signed_url.url,
    },
    include_image_base64=True
)

# Générer nom de sortie
if output_name is None:
    base_name = os.path.splitext(os.path.basename(pdf_path))[0]
    output_name = f"{base_name}.md"

# Sauvegarder en Markdown
os.makedirs(MARKDOWN_DIR, exist_ok=True)
output_path = os.path.join(MARKDOWN_DIR, output_name)

with open(output_path, "w", encoding="utf-8") as f:
    f.write("\n".join([page.markdown for page in ocr_response.pages]))

print(f"SUCCESS: {output_path} créé ({len(ocr_response.pages)} pages)")
return True

except Exception as e:
    print(f"ERREUR OCR {pdf_path}: {e}")
    return False

```

Cette fonction reprend le même principe d'OCR que dans le projet de recherche académique (upload du PDF, URL signée, traitement par mistral-ocr-latest) mais l'applique ici aux grilles tarifaires 2024 des transporteurs UPS, FedEx et DHL fournies en PDF. Elle est encapsulée dans un try/except qui retourne un booléen de succès/échec plutôt que de lever une exception, ce qui permettra à la fonction appelante de traiter plusieurs PDF à la suite sans que l'échec de l'un n'interrompe le traitement des autres.

Etape 4 — DDGSTools — outil de recherche web de secours

[Lignes 94-107](#)

```

class DDGSTools:
    def __init__(self):
        self.ddgs = DDGS()

    def search(self, query, max_results=5):
        try:
            results = list(self.ddgs.text(query, max_results=max_results))
            return results
        except Exception as e:
            print(f"Erreur recherche: {e}")
            return []

# Dans votre agent
ddgs_tool = DDGSTools()

```

Cette petite classe encapsule la librairie DDGS (recherche DuckDuckGo) dans un outil réutilisable par l'agent : elle expose une méthode search() protégée par un try/except qui renvoie une liste vide en cas d'échec plutôt que de faire planter l'agent. C'est ce mécanisme qui permet au Freight Agent de compléter les informations de sa base de connaissances figée (les grilles tarifaires 2024) par des données de transport plus récentes ou manquantes trouvées en direct sur le web.

Etape 5 — process_all_pdfs — traitement en masse du dossier de documents

[Lignes 109-138](#)

```

## Traitement de tous vos PDFs ###
def process_all_pdfs():
    """Traite tous les PDFs du dossier Document/"""
    document_folder = DOCUMENT_DIR

    if not os.path.exists(document_folder):
        print(f"ERREUR: Dossier {document_folder} non trouvé")
        return

    # Lister tous les PDFs
    pdf_files = [f for f in os.listdir(document_folder) if f.lower().endswith('.pdf')]

    if not pdf_files:
        print("Aucun PDF trouvé dans Document/")
        return

    print(f"PDFs trouvés: {pdf_files}")

    success_count = 0
    for pdf_file in pdf_files:
        pdf_path = os.path.join(document_folder, pdf_file)
        if ocr_pdf(pdf_path):
            success_count += 1

    print(f"\nTraitement terminé: {success_count}/{len(pdf_files)} PDFs convertis")

    # Lister les Markdown créés
    if os.path.exists(MARKDOWN_DIR):
        md_files = [f for f in os.listdir(MARKDOWN_DIR) if f.endswith('.md')]
        print(f"Fichiers Markdown créés: {md_files}")

```

Cette fonction balaie le dossier Document/, y repère tous les fichiers PDF, et appelle `ocr_pdf()` sur chacun d'eux en comptant les succès. C'est l'étape de préparation ponctuelle (à exécuter une fois, pas à chaque requête utilisateur) qui transforme l'ensemble des grilles tarifaires PDF des transporteurs en fichiers Markdown exploitables, avant qu'ils ne soient indexés dans la base vectorielle à l'étape suivante.

Etape 6 — Construction de la base de connaissances vectorielle (RAG)

[Lignes 140-158](#)

```

## Configuration base de connaissances ###
# MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
# unifiée Knowledge : le contenu se charge via add_content(path=...) plutôt
# qu'un argument du constructeur + .load(recreate=...).
knowledge_base = Knowledge(
    vector_db=LanceDb(
        table_name="freight_documents",
        uri=LANCEDB_URI,
        search_type=SearchType.hybrid,
        # L'API d'embeddings gratuite de Mistral est trop vite rate-limitée pour
        # indexer plusieurs documents d'un coup ; embedder local à la place.
        embedder=SentenceTransformerEmbedder(),
    ),
    # La valeur par défaut (10 documents) fait dépasser la limite Groq (8000
    # tokens/minute en tier gratuit) en un seul appel.
    max_results=3,
)
if os.path.exists(MARKDOWN_DIR) and any(f.endswith(".md") for f in os.listdir(MARKDOWN_DIR)):
    knowledge_base.add_content(path=MARKDOWN_DIR)

```

C'est le cœur du système RAG de ce projet : LanceDb (une base vectorielle légère, embarquée directement dans un dossier local plutôt que sur un serveur PostgreSQL comme dans le projet 32) stocke les embeddings des grilles tarifaires, avec `search_type=SearchType.hybrid` combinant recherche sémantique (par sens) et recherche par mots-clés pour de meilleurs résultats sur des documents tarifaires structurés. L'embedder local sentence-transformers est utilisé pour la même raison que dans les autres projets : éviter les limites de débit d'une API d'embeddings payante. Le contenu n'est indexé que si des fichiers Markdown existent déjà dans MARKDOWN_DIR, évitant une erreur si l'étape OCR n'a pas encore été exécutée.

Etape 7 — Définition de l'agent Freight Agent

Lignes 160-181

```
freight_agent = Agent(
    name="Freight Agent",
    # Le contexte (base de connaissances + ReasoningTools) dépasse régulièrement la
    # limite Groq du modèle 120B (8000 tokens/minute) ; le modèle 20B a un quota plus
    # élevé sur le tier gratuit.
    model=Groq(id="openai/gpt-oss-20b", api_key=groq_key),
    instructions=dedent("""
    You are a Freight Cost Estimator Agent that helps small import/export businesses estimate and compare
    shipping costs and timelines across major carriers.

    You have access to:
    - A knowledge base built from 2024 rate guides and surcharge tables for UPS, FedEx, and DHL (converted
    from PDF via OCR)
    - DDGS search for live data and missing information

    When analyzing shipping requests, search your knowledge base FIRST for official rates, then
    supplement with web search if needed.

    Always provide detailed cost breakdowns, delivery estimates, and clear recommendations based on the
    specific product dimensions and requirements.
    """),
    tools=[ReasoningTools(add_instructions=True), ddgs_tool],
    knowledge=knowledge_base,
    search_knowledge=True,
    compress_tool_results=True,
)
```

L'agent central du projet est configuré avec des instructions qui imposent un ordre de priorité clair : consulter d'abord la base de connaissances officielle (les grilles tarifaires indexées) avant de recourir à la recherche web via l'outil DDGS. Le modèle choisi (openai/gpt-oss-20b) est délibérément le plus petit disponible sur Groq car le contexte combiné de la base de connaissances et de ReasoningTools dépasse régulièrement la limite de tokens par minute du modèle plus gros sur le tier gratuit ; `compress_tool_results=True` réduit encore la taille du contexte réinjecté pour rester sous ce quota.

Etape 8 — Fonctions d'utilisation — chargement, réponse et test

Lignes 183-215

```
def load_knowledge_base():
    """Le contenu est déjà chargé à la construction de knowledge_base via add_content()."""
    if not (os.path.exists(MARKDOWN_DIR) and any(f.endswith('.md') for f in os.listdir(MARKDOWN_DIR))):
        print("Aucun fichier Markdown trouvé - utilisation de DDGS uniquement")

def full_response(message):
    print('-----Freight Agent Triggered-----')
    try:
        response = freight_agent.run(message)
        return response.content
    except Exception as e:
        print(f"Erreur: {e}")
        return f"Erreur lors du traitement: {str(e)}"

if __name__ == '__main__':
    print("=== FREIGHT AGENT OCR SETUP ===")

    # Étape 1: Traiter les PDFs (décommentez pour exécuter)
    #process_all_pdfs()
    #exit() # Arrête après traitement OCR

    # Étape 2: Charger la base de connaissances
    load_knowledge_base()

    # Test
    test_message = """
    User Question: Ship from Chicago, IL to Miami, FL
```

```

Product Specifications: Machine parts, Weight: 300 lbs, Dimensions: 36"x24"x18", Industrial equipment
"""

print("\nTest agent...")
result = full_response(test_message)
print(f"\nRésultat:\n{result}")

```

`full_response()` est la fonction d'interface publique appelée depuis `whatsapp_utils.py` : elle encapsule l'appel à `freight_agent.run()` dans un `try/except` pour ne jamais faire planter le serveur WhatsApp même en cas d'erreur, et renvoie toujours une chaîne de texte exploitable. Le bloc `__main__` sert uniquement de test local en ligne de commande, avec un message d'exemple simulant une vraie demande d'expédition, ce qui permet de vérifier que l'agent répond correctement avant de le brancher sur WhatsApp.

33-FreightAgent/main.py

Serveur web FastAPI qui expose le webhook WhatsApp (vérification et réception des messages) ainsi que quelques routes de test pour l'API WhatsApp Business de Meta.

Etape 1 — Imports, variables d'environnement et création de l'application

Lignes 1-20

```

import os
import json
import uvicorn
import logging
import requests
from threading import Thread
from dotenv import load_dotenv
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import PlainTextResponse, JSONResponse
from whatsapp_utils import process_whatsapp_message, is_valid_whatsapp_message

load_dotenv()

VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')
WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
RECIPIENT_PHONE_NUMBER = os.getenv('RECIPIENT_PHONE_NUMBER')
VERIFY_TOKEN = os.getenv('VERIFY_TOKEN')

app = FastAPI()

```

Ce fichier met en place le serveur web qui reçoit les messages WhatsApp. FastAPI est utilisé pour sa simplicité à définir des routes HTTP asynchrones ; Thread permettra plus loin de traiter les messages en arrière-plan sans bloquer la réponse au webhook. Toutes les informations sensibles propres à l'API WhatsApp Business de Meta (token d'accès, identifiant du numéro de téléphone, token de vérification) sont chargées depuis les variables d'environnement plutôt qu'écrites en dur dans le code.

Etape 2 — Route racine de test

Lignes 22-24

```

@app.get("/")
def read_root():
    return {"message": "Welcome to the portal!"}

```

Une route minimale qui répond simplement par un message de bienvenue en JSON : elle sert à vérifier rapidement que le serveur est en ligne et répond, sans dépendre d'aucune configuration WhatsApp.

Etape 3 — verify — vérification du webhook exigée par Meta

Lignes 26-42

```
# Required webhook verification for WhatsApp
async def verify(request: Request):
    params = dict(request.query_params)
    mode = params.get("hub.mode")
    token = params.get("hub.verify_token")
    challenge = params.get("hub.challenge")

    if mode and token:
        if mode == "subscribe" and token == VERIFY_TOKEN:
            logging.info("WEBHOOK_VERIFIED")
            return PlainTextResponse(content=challenge, status_code=200)
        else:
            logging.warning("VERIFICATION_FAILED")
            raise HTTPException(status_code=403, detail="Verification failed")
    else:
        logging.warning("MISSING_PARAMETER")
        raise HTTPException(status_code=400, detail="Missing parameters")
```

Cette fonction implémente la procédure de vérification imposée par Meta pour tout webhook WhatsApp Business : quand on configure le webhook dans l'interface Meta, celle-ci envoie une requête GET avec un mode, un token et un challenge, et le serveur doit renvoyer exactement ce challenge tel quel si le token reçu correspond au VERIFY_TOKEN secret configuré côté serveur — c'est cette vérification bilatérale qui prouve que le serveur appartient bien au propriétaire de l'application avant que Meta n'active l'envoi réel des messages entrants.

Etape 4 — handle_message — réception asynchrone des messages entrants

[Lignes 44-68](#)

```
async def handle_message(request: Request):
    try:
        body = await request.json()
    except json.JSONDecodeError:
        logging.error("Failed to decode JSON")
        return JSONResponse(status_code=400, content={"status": "error", "message": "Invalid JSON provided"})

    if (
        body.get("entry", [{}])[0]
        .get("changes", [{}])[0]
        .get("value", {})
        .get("statuses")
    ):
        logging.info("Received a WhatsApp status update.")
        return JSONResponse(content={"status": "ok"})

    if is_valid_whatsapp_message(body):
        try:
            # Run message processing in a background thread
            Thread(target=process_whatsapp_message, args=(body,)).start()
        except Exception as e:
            logging.error(f"Error starting background message thread: {e}")
            return JSONResponse(content={"status": "ok"})
    else:
        return JSONResponse(status_code=404, content={"status": "error", "message": "Not a WhatsApp API event"})
```

Cette fonction filtre et route le corps JSON reçu de Meta : elle ignore silencieusement les simples accusés de statut (message envoyé/lu) qui ne nécessitent aucune action, puis vérifie que le corps correspond bien à un vrai message utilisateur avant de le traiter. Le traitement effectif est délégué à un thread en arrière-plan (`Thread(target=process_whatsapp_message,...).start()`) : c'est un choix important, car l'API WhatsApp attend une réponse HTTP rapide au webhook, alors que le traitement réel (appel aux agents IA, potentiellement plusieurs secondes) prendrait trop de temps s'il bloquait la réponse.

Etape 5 — Routes de test manuel de l'API WhatsApp

[Lignes 72-109](#)

```

@app.get("/test")
def test():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'template',
        'template': {
            'name': 'hello_world',
            'language': {
                'code': 'en_US'
            }
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Test successful!", "response": response.json()}

@app.get("/send_message")
def send_message():
    url = f'https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages'
    headers = {
        'Authorization': f'Bearer {WHATSAPP_ACCESS_TOKEN}',
        'Content-Type': 'application/json'
    }
    data = {
        'messaging_product': 'whatsapp',
        'to': RECIPIENT_PHONE_NUMBER,
        'type': 'text',
        'text': {
            'body': 'Hello, how are you?'
        }
    }
    response = requests.post(url, headers=headers, json=data)
    return {"message": "Message sent successfully!", "response": response.json()}

```

Ces deux routes ne font pas partie du flux applicatif normal : elles servent d'outils de diagnostic manuel pour vérifier que la connexion à l'API WhatsApp Business fonctionne. `/test` envoie le message-modèle standard `hello_world` (le seul type de message qu'on peut envoyer sans que le destinataire ait initié la conversation dans les 24 dernières heures), tandis que `/send_message` envoie un simple message texte libre, utile pour tester rapidement l'envoi sans passer par tout le pipeline agent.

Etape 6 — Déclaration des routes du webhook et point d'entrée du serveur

[Lignes 115-124](#)

```

@app.get("/webhook")
async def webhook_verify(request: Request):
    return await verify(request)

@app.post("/webhook")
async def webhook_message(request: Request):
    return await handle_message(request)

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

La même URL `/webhook` est utilisée pour deux méthodes HTTP différentes, comme l'exige l'API WhatsApp Business : GET pour la vérification initiale du webhook (déléguée à `verify()`) et POST pour chaque message réellement reçu (délégué à `handle_message()`). Le bloc final lance le serveur avec `uvicorn` sur le port 8000, accessible sur toutes les interfaces réseau (0.0.0.0), ce qui permet de l'exposer publiquement (par exemple via un tunnel) pour que Meta puisse effectivement l'atteindre.

33-FreightAgent/agent_service.py

Agent de vision qui analyse soit un texte, soit une image envoyée par l'utilisateur, pour identifier le produit à expédier ainsi que son nom, ses dimensions et sa matière.

Etape 1 — Imports et configuration de la clé OpenAI

Lignes 1-11

```
import os
import base64
# from letta_client import Letta
from dotenv import load_dotenv
from agno.agent import Agent
from agno.media import Image
from agno.models.openai import OpenAIChat

load_dotenv()

openai_key = os.getenv("OPENAI_API_KEY")
```

Ce fichier charge les briques nécessaires à un agent multimodal : `agno.media.Image` permettra de transmettre une image au modèle, et `OpenAIChat` servira de moteur au futur agent de vision. L'import commenté de `letta_client` laisse une trace d'une intégration alternative envisagée puis abandonnée (un service de mémoire d'agent), montrant que le projet a évolué vers une solution plus simple basée uniquement sur `agno`.

Etape 2 — `get_response` — détection texte ou image et création de l'agent

Lignes 29-46

```
def get_response(message):
    message = message
    image_data = None
    print('-----Agent triggered-----')
    if isinstance(message, dict):
        with open(message.get("image_path"), "rb") as f:
            image_data = f.read()
        message = message.get('caption')

    print('---message---', message)
    agent = Agent(
        model=OpenAIChat(id="gpt-4o-mini", api_key=openai_key),
        instructions="""
        If an image is given, you are responsible for defining the object on the image, (if it's a product
        define what is it in details),
        along with it's name, dimensions and material.
        IMPORTANT: Do not ask back, only give product specifications, dimensions and material!
        """,
    )
```

Cette fonction accepte deux formes d'entrée possibles : un simple texte, ou un dictionnaire contenant un chemin d'image et une légende (produit plus loin par `whatsapp_utils.py` quand l'utilisateur envoie une photo). Si c'est un dictionnaire, le fichier image est lu en binaire pour être transmis au modèle. Un agent GPT-4o mini est ensuite instancié avec des instructions strictes : décrire précisément l'objet visible (nom, dimensions, matière) sans jamais poser de question de clarification en retour, ce qui est essentiel dans un flux automatisé WhatsApp où il n'y a pas d'échange interactif possible à cette étape.

Etape 3 — Exécution de l'agent et construction de la réponse combinée

Lignes 47-56

```
if image_data:
    response = agent.run('Please define the object on the image, along with it\'s name, dimensions
and material.', images=[Image(content=image_data)])
else:
    response = agent.run(message)
```

```

print('---response---', response.content)
full_response = f"""
User Question: {message}
Product Specifications: {response.content}
"""
return full_response

```

Selon qu'une image a été fournie ou non, l'agent est exécuté avec ou sans le paramètre images. Dans les deux cas, le résultat est reformaté en une chaîne combinant la question/légende d'origine et les spécifications produit identifiées par l'agent : c'est ce texte structuré, et non la réponse brute, qui sera ensuite transmis au Freight Agent (dans freight_agent.py) comme entrée pour l'estimation du coût de transport, faisant le pont entre l'agent de vision et l'agent de fret.

Etape 4 — Point d'entrée de test

Lignes 59-60

```

if __name__ == '__main__':
    print(get_response('hello'))

```

Un test minimal en ligne de commande qui appelle get_response() avec un simple texte, permettant de vérifier rapidement que l'agent répond correctement sans avoir à passer par WhatsApp ni à fournir une image.

33-FreightAgent/whatsapp_utils.py

Fonctions utilitaires qui font le lien entre l'API WhatsApp Business (validation des messages entrants, téléchargement des images, envoi des réponses) et les deux agents du projet (agent_service pour la vision, freight_agent pour l'estimation).

Etape 1 — Imports et configuration

Lignes 1-16

```

import os
import re
import json
import logging
import requests
from time import sleep
from dotenv import load_dotenv
from freight_agent import full_response
from agent_service import get_response
from fastapi.responses import JSONResponse

load_dotenv()

WHATSAPP_ACCESS_TOKEN = os.getenv('WHATSAPP_ACCESS_TOKEN')
VERSION = os.getenv('VERSION')
PHONE_NUMBER_ID = os.getenv('PHONE_NUMBER_ID')

```

Ce fichier importe directement les deux agents des autres modules du projet (full_response du Freight Agent et get_response de l'agent de vision), ce qui en fait le point de jonction central du pipeline complet WhatsApp. Les identifiants nécessaires pour appeler l'API Graph de Meta (token d'accès, version d'API, identifiant du numéro de téléphone) sont chargés depuis l'environnement.

Etape 2 — Fonctions utilitaires de journalisation et de formatage

Lignes 18-31

```

def log_http_response(response):
    logging.info(f"Status: {response.status_code}")
    logging.info(f"Content-type: {response.headers.get('content-type')}")
    logging.info(f"Body: {response.text}")

```

```
def get_text_message_input(recipient, text):
    return {
        "messaging_product": "whatsapp",
        "recipient_type": "individual",
        "to": recipient,
        "type": "text",
        "text": {"preview_url": False, "body": text},
    }
```

`log_http_response()` centralise l'affichage des détails d'une réponse HTTP (statut, type de contenu, corps), utile pour diagnostiquer un problème avec l'API WhatsApp. `get_text_message_input()` construit le format JSON exact exigé par l'API Graph de Meta pour envoyer un message texte à un destinataire donné, évitant de dupliquer cette structure à chaque endroit du code qui doit envoyer un message.

Etape 3 — `send_message` — envoi robuste du message vers l'API WhatsApp

Lignes 39-56

```
def send_message(data):
    headers = {
        "Content-type": "application/json",
        "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
    }
    url = f"https://graph.facebook.com/{VERSION}/{PHONE_NUMBER_ID}/messages"

    try:
        response = requests.post(url, json=data, headers=headers, timeout=10)
        response.raise_for_status()
        log_http_response(response)
        return response
    except requests.Timeout:
        logging.error("Timeout occurred while sending message")
        return JSONResponse({"status": "error", "message": "Request timed out"}, status_code=408)
    except requests.RequestException as e:
        logging.error(f"Request failed due to: {e}")
        return JSONResponse({"status": "error", "message": "Failed to send message"}, status_code=500)
```

Cette fonction envoie effectivement le message final à l'API Graph de Meta, authentifiée par le token d'accès. Un timeout de 10 secondes évite que la requête ne reste bloquée indéfiniment, et deux cas d'erreur distincts sont gérés séparément (dépassement de délai vs autre erreur réseau/HTTP), chacun renvoyant une réponse JSON d'erreur explicite plutôt que de laisser une exception non gérée remonter et potentiellement faire planter le thread de traitement du message.

Etape 4 — `is_valid_whatsapp_message` — validation défensive du corps reçu

Lignes 59-67

```
def is_valid_whatsapp_message(body: dict):
    return (
        body.get("object")
        and body.get("entry")
        and body["entry"][0].get("changes")
        and body["entry"][0]["changes"][0].get("value")
        and body["entry"][0]["changes"][0]["value"].get("messages")
        and body["entry"][0]["changes"][0]["value"]["messages"][0]
    )
```

Le format JSON envoyé par Meta pour un webhook WhatsApp est profondément imbriqué et peut varier selon le type d'événement (message, statut, erreur...). Cette fonction vérifie, niveau par niveau avec des accès chaînés en court-circuit (`and`), que la structure attendue pour un vrai message utilisateur est bien présente avant de tenter de la traiter plus loin, évitant ainsi une exception de type `KeyError` si un événement d'une autre forme était reçu.

Etape 5 — `process_whatsapp_message` — extraction du message texte ou image

Lignes 70-92

```
def process_whatsapp_message(body: dict):
    value = body["entry"][0]["changes"][0]["value"]
    wa_id = value["contacts"][0]["wa_id"]
    name = value["contacts"][0]["profile"]["name"]
    message = value["messages"][0]
    if message["type"] == "text":
        message_body = message["text"]["body"]
    elif message["type"] == "image":
        message_body = message["image"]
        id = message_body.get("id")
        headers = {
            "Content-type": "application/json",
            "Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}",
        }
        metadata_res = requests.get(f"https://graph.facebook.com/{VERSION}/{id}", headers=headers)
        image_url = metadata_res.json().get("url")
        image_res = requests.get(image_url, headers={"Authorization": f"Bearer {WHATSAPP_ACCESS_TOKEN}"})
        with open(f"33-FreightAgent/images/{id}.jpg", "wb") as f:
            f.write(image_res.content)
        if message_body.get("caption"):
            message_body = {'image_path': f'33-FreightAgent/images/{id}.jpg', 'caption':
message_body.get("caption")}
        else:
            message_body = {'image_path': f'33-FreightAgent/images/{id}.jpg', 'caption': ''}
```

Cette étape extrait l'identifiant du contact et le message lui-même du corps de la requête, puis traite deux cas différemment. Pour un message texte, le contenu est utilisé directement. Pour une image, WhatsApp ne transmet jamais le fichier lui-même mais seulement un identifiant : il faut donc faire un premier appel à l'API Graph pour obtenir l'URL réelle et temporaire de l'image, puis un second appel pour télécharger effectivement le contenu binaire et le sauvegarder localement sur le disque, avant de construire un dictionnaire {image_path, caption} — c'est précisément ce format que get_response() dans agent_service.py sait interpréter comme une entrée image.

Etape 6 — Enchaînement des deux agents et envoi de la réponse finale

[Lignes 93-96](#)

```
response_text = get_response(message_body)
full_text = full_response(response_text)
data = get_text_message_input(wa_id, full_text)
send_message(data)
```

Cette dernière étape orchestre tout le pipeline métier en trois appels successifs : get_response() (l'agent de vision) analyse le message ou l'image et produit une description structurée du produit, full_response() (le Freight Agent) reçoit cette description et calcule une estimation de coût et de délai de transport, puis le résultat est formaté au format attendu par l'API WhatsApp et envoyé au contact d'origine via send_message(). C'est cet enchaînement qui relie concrètement les deux agents indépendants du projet en une seule conversation WhatsApp fluide pour l'utilisateur final.

Projet 34 — 34-ImportExportSpecialist

Variante autonome (CLI) : conformité import/export, export Word direct

LE BUT DE CE PROJET

Quand une entreprise veut exporter un produit électronique du Royaume-Uni vers le Maroc, elle doit respecter deux jeux de règles complètement différents : les lois d'export britanniques (licences, taxes, documents obligatoires) et les lois d'import marocaines (droits de douane, TVA, produits interdits, permis). Aujourd'hui, une équipe conformité doit éplucher des textes de loi PDF longs et arides, croiser les informations des deux pays, et vérifier qu'elles sont toujours à jour - un travail lent, répétitif et où la moindre erreur peut coûter cher (colis bloqué en douane, amende, produit refusé). Le but de ce projet est de montrer comment automatiser ce travail avec un agent IA qui lit et comprend de vrais documents légaux (en les convertissant d'abord de PDF en texte grâce à l'OCR), les indexe dans une base de connaissances consultable par similarité de sens, puis s'appuie sur cette base - complétée par une recherche web en direct quand une information manque - pour rédiger un rapport de conformité complet, chiffré et sourcé, converti automatiquement en document Word prêt à être partagé. La technique centrale démontrée est le RAG (Retrieval-Augmented Generation, "génération augmentée par récupération") : au lieu de laisser le modèle de langage deviner ou inventer un taux de TVA ou un numéro de licence, on le force à aller chercher la réponse dans de vrais documents avant de répondre, ce qui rend ses affirmations vérifiables plutôt qu'hallucinées.

34-ImportExportSpecialist/full_import_export_agent.py

Agent Agno qui effectue l'OCR de documents légaux PDF, les indexe dans une base de connaissances vectorielle locale (RAG), puis génère un rapport de conformité import-export au format DOCX.

Etape 1 — Préparation de la console et imports

Lignes 1-21

```
import sys

# La console Windows utilise par défaut l'encodage cp1252, qui ne supporte pas
# les emojis utilisés dans les messages de ce script (ex: , ).
for _stream in (sys.stdout, sys.stderr):
    if hasattr(_stream, "reconfigure"):
        _stream.reconfigure(encoding="utf-8", errors="replace")

import os
import markdown2
from textwrap import dedent
from agno.agent import Agent
from mistralai.client import Mistral
from html2docx import html2docx
from agno.models.groq import Groq
from agno.vectordb.lancedb import LanceDb, SearchType # Changement ici
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.knowledge.knowledge import Knowledge
from agno.knowledge.embedder.mistral import MistralEmbedder
from dotenv import load_dotenv
load_dotenv()
```

Avant même de faire quoi que ce soit d'utile, le script corrige un problème pratique : la console Windows n'affiche pas nativement les emojis utilisés plus loin dans les messages (,), donc on force stdout/stderr en UTF-8 pour éviter un plantage au premier print(). Ensuite viennent tous les imports qui posent les fondations du script : Agno (le framework d'agents IA), Mistral (client OCR), html2docx (conversion HTML vers Word), Groq (fournisseur d'inférence rapide pour le chat), LanceDB (base vectorielle locale) et son embedder Mistral, DuckDuckGoTools (recherche web), et enfin dotenv pour charger les clés API depuis un fichier .env plutôt que de les coder en dur. Cette étape pose tout l'outillage nécessaire aux étapes suivantes : OCR, RAG, génération de rapport, export DOCX.

Etape 2 — Résolution des chemins et chargement des clés API

Lignes 25-42

```
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MARKDOWN_DIR = os.path.join(SCRIPT_DIR, "Markdown")
LANCEDB_URI = os.path.join(SCRIPT_DIR, "tmp", "lancedb")

mistral_key = os.getenv("MISTRAL_API_KEY")
# Le script original utilisait OpenAI (chat + embeddings) ; remplacé par Groq
# pour le chat (clé GROQ_API_KEY déjà disponible) et par l'embedder Mistral
# pour les embeddings (Groq n'a pas d'API d'embeddings).
groq_key = os.getenv("GROQ_API_KEY")

if not groq_key:
    print("ERREUR: GROQ_API_KEY manquant")
    exit()

if mistral_key:
    client = Mistral(api_key=mistral_key)
else:
    print("WARNING: MISTRAL_API_KEY manquant - OCR désactivé")
```

Les chemins des dossiers (Markdown, base LanceDB) sont calculés à partir de l'emplacement du script lui-même plutôt que du répertoire courant, pour que le script fonctionne peu importe d'où il est lancé. Les clés API Mistral et Groq sont ensuite lues depuis l'environnement. La gestion de leur absence illustre deux niveaux de criticité différents : Groq est indispensable au fonctionnement des deux agents de chat définis plus bas, donc son absence arrête le script immédiatement (fail fast) ; Mistral ne sert qu'à la fonctionnalité optionnelle d'OCR, donc son absence ne fait qu'émettre un avertissement et désactiver cette seule fonctionnalité, sans bloquer le reste - un exemple de dégradation gracieuse plutôt que d'arrêt total.

Etape 3 — Fonction d'OCR du PDF réglementaire

Lignes 45-78

```
def ocr_pdf(pdf_path):
    if not mistral_key:
        print("MISTRAL_API_KEY requis pour OCR")
        return

    # check if the file exists
    if not os.path.exists(pdf_path):
        print("PDF file not found")
        return

    uploaded_pdf = client.files.upload(
        file={
            "file_name": os.path.basename(pdf_path),
            "content": open(pdf_path, "rb"),
        },
        purpose="ocr"
    )

    signed_url = client.files.get_signed_url(file_id=uploaded_pdf.id)

    ocr_response = client.ocr.process(
        model="mistral-ocr-latest",
        document={
```

```

        "type": "document_url",
        "document_url": signed_url.url,
    },
    include_image_base64=True
)

# Créer le dossier s'il n'existe pas
os.makedirs(MARKDOWN_DIR, exist_ok=True)

with open(os.path.join(MARKDOWN_DIR, "morocco-import-law.md"), "w", encoding="utf-8") as f:
    f.write("\n".join([page.markdown for page in ocr_response.pages]))

```

Cette fonction transforme un PDF de loi (potentiellement scanné) en texte exploitable : elle téléverse le fichier vers l'API Mistral, récupère une URL signée temporaire pointant vers ce fichier hébergé, puis lance le modèle spécialisé "mistral-ocr-latest" sur cette URL. Le résultat est renvoyé page par page en Markdown, que la fonction concatène et écrit dans un fichier .md local. C'est une étape de prétraitement obligatoire : une base de connaissances RAG a besoin de texte structuré, pas d'un PDF brut, donc l'OCR sert de pont entre le document légal original et l'index vectoriel construit à l'étape suivante. L'appel à cette fonction est volontairement laissé en commentaire plus bas dans le fichier (lignes 80-82) car l'OCR est coûteux et n'a besoin d'être exécuté qu'une seule fois pour produire le fichier Markdown, pas à chaque lancement du script.

Etape 4 — Construction de la base de connaissances RAG (LanceDB)

[Lignes 85-116](#)

```

def setup_knowledge_base():
    """Configure la base de connaissances avec LanceDB"""
    markdown_path = MARKDOWN_DIR

    if not os.path.exists(markdown_path):
        print(f"Dossier {markdown_path} non trouvé")
        return None

    md_files = [f for f in os.listdir(markdown_path) if f.endswith('.md')]
    if not md_files:
        print(f"Aucun fichier Markdown dans {markdown_path}")
        return None

    print(f"Fichiers Markdown trouvés: {md_files}")

    # MarkdownKnowledgeBase(path=..., vector_db=...) a été remplacé par la classe
    # unifiée Knowledge : le contenu se charge maintenant via add_content(path=...)
    # plutôt qu'un argument du constructeur + .load(recreate=...).
    knowledge_base = Knowledge(
        vector_db=LanceDb(
            table_name="import_export_documents",
            uri=LANCEDB_URI, # Stockage local
            search_type=SearchType.hybrid,
            embedder=MistralEmbedder(api_key=mistral_key),
        ),
    )
    knowledge_base.add_content(path=markdown_path)

    return knowledge_base

# Configuration de la base de connaissances
knowledge_base = setup_knowledge_base()

```

Cette fonction construit le cœur du système RAG : après avoir vérifié que le dossier Markdown existe et contient bien des fichiers .md (sinon elle échoue proprement en renvoyant None), elle crée un objet Knowledge appuyé sur LanceDB, une base vectorielle embarquée qui stocke ses données directement dans un dossier local plutôt que sur un serveur externe à héberger. Le mode de recherche "hybrid" combine recherche par mots-clés et recherche par similarité sémantique, ce qui rattrape les cas où l'une des deux approches seule raterait la bonne information. L'appel add_content() déclenche l'ingestion effective : les fichiers Markdown sont découpés en morceaux, transformés en vecteurs via MistralEmbedder, puis stockés dans la table LanceDB - c'est ce qui permettra ensuite

aux agents de retrouver les passages pertinents des lois d'export/import à chaque question posée.

Etape 5 — Définition de l'agent de conformité export-import

Lignes 118-187

```
import_export_agent = Agent(
    name='Import Export Agent',
    model=Groq(id='openai/gpt-oss-120b', api_key=groq_key),
    instructions=dedent("""
    You are an Export-Import Compliance Expert with access to two knowledge sources:
    - The official UK export law document for electronics (including licences, restrictions, taxes,
    fees),
    - The official Moroccan import law document for electronics (including permits, quantity limits,
    import duties, VAT, prohibited items, and customs procedures).

    You also have access to GoogleSearchTools() to find up-to-date, authoritative data when it is missing
    or unclear.

    Your task is: given detailed product info (type, quantity, raw materials) and the shipment route (UK
    to Morocco), provide a full, precise, step-by-step explanation of:
    [...]
    Use only facts and figures extracted from your knowledge base or from live, authoritative
    sources. Do not guess or be vague.

    If any data is missing in your knowledge, immediately perform a GoogleSearchTools() lookup for
    official government or customs sites and incorporate exact data found.
    [...]
    """),
    expected_output=dedent("""
    IMPORTANT: Output the final result in raw HTML, starting directly with <h1> and without wrapping it in
    Markdown code blocks like `html ...`.

    <h1 style="text-align: center;"> Import-Export Compliance Report</h1>
    <hr>
    [...]
    """),
    knowledge=knowledge_base,
    search_knowledge=True if knowledge_base else False,
    tools=[DuckDuckGoTools()]
)
```

Cette étape définit le premier agent, spécialisé dans le passage de la frontière (export UK + import Maroc). Le paramètre instructions est le "prompt système" : il cadre le rôle de l'agent, lui rappelle ses deux sources documentaires, lui interdit d'inventer des chiffres, et lui demande de basculer sur une recherche web quand une donnée manque dans sa base - un garde-fou anti-hallucination classique. Le paramètre expected_output est un second bloc de prompt, distinct, qui impose un gabarit HTML précis pour la réponse : cette technique de prompt engineering (séparer "comment te comporter" de "quel format produire") fiabilise la mise en forme, ce qui compte ici car la sortie sera directement convertie en document Word plus loin. Enfin, knowledge et search_knowledge branchent effectivement la base RAG construite à l'étape précédente sur cet agent (avec une désactivation propre si la base n'a pas pu être créée), et tools lui donne la capacité d'aller chercher sur le web ce qu'il ne trouve pas dans ses documents.

Etape 6 — Définition de l'agent de réglementation locale

Lignes 195-266

```
local_regulations_agent = Agent(
    name='Local Regulations Agent',
    model=Groq(id='openai/gpt-oss-120b', api_key=groq_key),
    instructions=dedent("""
    You are a Local Regulation Checker Agent.

    Your task is to research local regulations in a specific country related to the imported product.
    Focus on the rules and requirements for legally selling that product in the local market after import.

    Based on the provided product and target import country, gather clear and up-to-date information from
```

```

official sources (e.g., government websites, trade portals, customs authorities). Use reliable data to
answer the following:
[...]
Only return factual and country-specific results with references when possible. If something is
unclear or unavailable, state that clearly.
"""),
expected_output=dedent("""
IMPORTANT: Output the final result in raw HTML, starting directly with <h1> and without wrapping it in
Markdown code blocks like `html ...`.

<h1 style="text-align: center;"> Local Regulation Report</h1>
<hr>
[...]
"""),
tools=[DuckDuckGoTools()],
)

```

Ce second agent complète le premier avec un rôle différent et complémentaire : là où `import_export_agent` traite le passage en douane, `local_regulations_agent` recherche les règles de vente une fois le produit arrivé sur le marché marocain (étiquetage, certifications type CE/FDA/ISO/halal, licences de vente, distribution, taxes locales). Il ne possède volontairement aucune base de connaissances RAG (pas de paramètre `knowledge`) : il s'appuie entièrement sur son outil de recherche web en direct, car ce type d'information (réglementation commerciale locale) évolue et se trouve mieux via une recherche à jour que via un document statique indexé une fois. Ce choix illustre qu'un système multi-agent peut combiner, pour deux sous-tâches différentes, deux stratégies différentes d'accès à l'information (RAG statique vs recherche live), plutôt que d'imposer la même approche partout.

Etape 7 — Orchestration des deux agents et export en DOCX

[Lignes 269-301](#)

```

def generate_report(product_description):
    """Génère un rapport d'import-export complet"""
    try:
        print(f"Génération du rapport pour: {product_description}")

        output_html = ""

        # Agent 1: Import-Export Compliance
        print("Exécution de l'agent Import-Export...")
        agent1 = import_export_agent.run(product_description)
        output_html += agent1.content
        output_html += "<hr>"

        # Agent 2: Local Regulations
        print("Exécution de l'agent Local Regulations...")
        agent2 = local_regulations_agent.run(product_description)
        output_html += agent2.content

        # Conversion en DOCX
        print("Conversion en DOCX...")
        docx_bytes = html2docx(output_html, title="Import-Export Regulation Report")

        # Sauvegarde
        filename = "import_export_regulation_report.docx"
        with open(filename, "wb") as f:
            f.write(docx_bytes.getvalue())

        print(f" Rapport sauvegardé: {filename}")
        return filename

    except Exception as e:
        print(f"Erreur lors de la génération: {e}")
        return None

```

Cette fonction relie tout ce qui précède en un pipeline complet : elle exécute d'abord l'agent de conformité douanière, puis l'agent de réglementation locale, sur la même description produit, et concatène leurs deux sorties HTML (séparées par une ligne horizontale) en un seul document. Ce HTML combiné est ensuite passé à `html2docx`,

qui le convertit directement en un fichier Word binaire, sauvegardé sur disque. C'est l'étape qui transforme deux réponses de modèle de langage en un livrable bureautique concret et partageable, sans intervention manuelle de mise en forme. Le bloc try/except englobant garantit qu'une erreur à n'importe quelle étape (appel agent, conversion, écriture disque) est journalisée clairement plutôt que de faire planter le script avec une trace peu lisible.

Etape 8 — Point d'entrée du script

Lignes 304-306

```
if __name__ == "__main__":  
    product = "An electronic air purifier exported from the UK and imported into Morocco"  
    generate_report(product)
```

Le script se termine par un exemple concret d'utilisation : un purificateur d'air électronique expédié du Royaume-Uni vers le Maroc. Cette garde `if __name__ == "__main__"` garantit que cette exécution de démonstration ne se déclenche que si le fichier est lancé directement (pas s'il est importé comme module ailleurs), et sert à la fois de test manuel et d'exemple documenté de la façon d'appeler `generate_report()`.

Projet 35 — 35-HR-Assistant

Assistant RH (conges, projets, approbations) via Airtable

LE BUT DE CE PROJET

Dans une entreprise, le service RH reçoit en permanence les mêmes questions ("combien de jours de congés me reste-t-il ?", "c'est quoi mon projet actuel ?") et doit traiter des demandes de congés qui nécessitent une validation humaine avant d'être actées. Répondre à la main à chaque question et faire circuler les demandes d'approbation par email est lent et consomme du temps RH qui pourrait être utilisé ailleurs. Le but de ce projet est de montrer comment construire une petite équipe d'agents IA spécialisés - un agent congés, un agent projets et un agent d'approbation - pilotée par un agent "manager" qui lit chaque question et l'aiguille automatiquement vers le bon spécialiste, exactement comme un standard téléphonique achemine un appel vers le bon service plutôt que de laisser un seul agent généraliste tout gérer. Les agents ne travaillent pas dans le vide : ils lisent et écrivent de vraies données dans une base Airtable (soldes de congés, projets en cours), ce qui illustre comment donner à un LLM la capacité d'agir sur un système métier réel via des fonctions (tool use) plutôt que de simplement discuter. Le projet montre aussi comment exposer ces agents derrière une API web consultable depuis une interface de chat séparée, et comment mettre en place une vraie boucle d'approbation humaine : la demande de congé génère un email avec un lien cliquable vers une page web d'approbation, rendue accessible depuis l'extérieur via un tunnel public même si le service tourne sur une machine locale.

35-HR-Assistant/rhAgent.py

Backend Flask et équipe d'agents Agno qui gèrent les congés, les projets et les approbations RH via Airtable, exposés via une API HTTP et un tunnel ngrok public.

Etape 1 — Imports, journalisation et validation des variables d'environnement

Lignes 1-47

```
# Imports
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from dotenv import load_dotenv
import os
from agno.tools.file import FileTools
from agno.tools.calculator import CalculatorTools
from agno.tools.email import EmailTools
from agno.team import Team
from agno.storage.sqlite import SqliteStorage
from pyairtable import Api
from flask import Flask, request, render_template_string, jsonify
from flask_cors import CORS
import uuid
import threading
import time
import logging
import requests
import pyngrok.ngrok as ngrok

# Set up logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)
```

```

load_dotenv()

# Load environment variables
api_key_openai = os.getenv("OPENAI_API_KEY")
AIRTABLE_TOKEN = os.getenv("AIRTABLE_TOKEN")
BASE_ID = os.getenv("AIRTABLE_BASE_ID")
NGROK_AUTH_TOKEN = os.getenv("NGROK_AUTH_TOKEN")
HR_EMAIL = os.getenv("HR_EMAIL")
db_file = os.getenv("SQLITE_DB_FILE", "hr_assistant_sessions.db")

# Validation des variables d'environnement critiques
required_env_vars = {
    "OPENAI_API_KEY": api_key_openai,
    "AIRTABLE_TOKEN": AIRTABLE_TOKEN,
    "AIRTABLE_BASE_ID": BASE_ID,
    "NGROK_AUTH_TOKEN": NGROK_AUTH_TOKEN,
    "HR_EMAIL": HR_EMAIL
}

missing_vars = [var for var, value in required_env_vars.items() if not value]
if missing_vars:
    logger.error(f"Variables d'environnement manquantes: {'', '.join(missing_vars)}")
    raise ValueError(f"Variables d'environnement requises manquantes: {'', '.join(missing_vars)}")

```

Le fichier commence par importer tous les briques nécessaires : Agno pour les agents et le multi-agent (Team), Flask et Flask-CORS pour exposer une API web, pyairtable pour parler à la base de données RH, pyngrok pour créer un tunnel public, et des modules standards (uuid, threading, time, logging) qui serviront à la logique applicative. Un logger est configuré pour remplacer les simples print() par des messages structurés et horodatés, plus faciles à diagnostiquer qu'un flot de texte brut. Les variables d'environnement critiques (clé OpenAI, jeton et base Airtable, jeton ngrok, email RH) sont ensuite lues puis vérifiées en bloc plutôt qu'une par une : si l'une d'elles manque, le script s'arrête immédiatement avec une erreur explicite (fail fast) plutôt que de démarrer un service RH mal configuré qui échouerait de façon confuse plus tard.

Etape 2 — Connexion Airtable et mise en place de l'application Flask

Lignes 50-99

```

try:
    api = Api(AIRTABLE_TOKEN)
    base = api.base(BASE_ID)
    holiday_table = base.table("employee_holidays")
    project_table = base.table("employee_projects")
    request_table = base.table("holiday_requests")
    logger.info("Connexion aux tables Airtable réussie")
except Exception as e:
    logger.error(f"Erreur de connexion Airtable: {e}")
    raise

# Flask app for approval/disapproval
app = Flask(__name__)

CORS(app, resources={r"/chat": {"origins": "http://localhost:8000"}})
CORS(app, resources={r"/text": {"origins": "http://localhost:8000"}})

APPROVAL_PAGE = """
<!DOCTYPE html>
<html>
<head>
    <title>Holiday Request Approval</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; }
        .container { max-width: 600px; margin: auto; }
        .button { padding: 10px 20px; margin: 10px; font-size: 16px; cursor: pointer; }
        .approve { background-color: #4CAF50; color: white; border: none; }
        .disapprove { background-color: #f44336; color: white; border: none; }
    </style>

```

```

</style>
</head>
<body>
  <div class="container">
    <h2>Holiday Request Approval</h2>
    <p><strong>Employee ID:</strong> {{ employee_id }}</p>
    <p><strong>Name:</strong> {{ name }}</p>
    <p><strong>Requested Days:</strong> {{ requested_days }}</p>
    <p><strong>Remaining Holiday Days:</strong> {{ remaining_days }}</p>
    <p><strong>Request Date:</strong> {{ request_date }}</p>

    <form method="POST" action="/process/{{ request_id }}">
      <button type="submit" name="action" value="approved" class="button approve">Approve</button>
      <button type="submit" name="action" value="disapproved" class="button
disapprove">Disapprove</button>
    </form>
  </div>
</body>
</html>
"""

# Variable globale pour stocker l'URL publique
public_url = None

```

Cette étape connecte le script aux trois tables Airtable qui font office de base de données RH (congrés, projets, demandes de congés), avec un arrêt immédiat en cas d'échec puisque Airtable est la source de vérité de toutes les données utilisées ensuite. L'application Flask est ensuite créée, avec CORS activé de façon volontairement restreinte (seules les routes /chat et /text, uniquement depuis localhost:8000, le frontend Streamlit) plutôt qu'ouvert à tous, pour limiter la surface d'attaque. Le gabarit HTML APPROVAL_PAGE, écrit en dur dans une chaîne Python, définit la page qu'un membre RH verra en cliquant sur le lien d'approbation reçu par email : les {{ ... }} sont des emplacements que Flask remplira dynamiquement avec les vraies données de la demande au moment du rendu.

Etape 3 — Routes web de la boucle d'approbation humaine

[Lignes 101-146](#)

```

@app.route('/request/<request_id>')
def show_request(request_id):
    try:
        logger.info(f"Attempting to retrieve request with request_id: {request_id}")
        records = request_table.all(formula=f"{{request_id}}='{{request_id}}'")
        if not records:
            logger.error(f"No record found for request_id: {request_id}")
            return "Request not found", 404
        record = records[0]
        fields = record['fields']
        logger.info(f"Retrieved record: {record}")
        return render_template_string(
            APPROVAL_PAGE,
            employee_id=fields.get("employee_id"),
            name=fields.get("full_name"),
            requested_days=fields.get("requested_days"),
            remaining_days=fields.get("remaining_days"),
            request_date=fields.get("request_date"),
            request_id=request_id
        )
    except Exception as e:
        logger.error(f"Error retrieving request {request_id}: {str(e)}")
        return f"Error retrieving request: {str(e)}", 500

@app.route('/process/<request_id>', methods=['POST'])
def process_request(request_id):
    try:
        logger.info(f"Processing request with request_id: {request_id}")
        action = request.form.get('action')
        records = request_table.all(formula=f"{{request_id}}='{{request_id}}'")
        if not records:

```

```

        logger.error(f"No record found for request_id: {request_id}")
        return "Request not found", 404

    record = records[0]
    request_table.update(record['id'], {'status': action})
    logger.info(f"Updated request {request_id} with status: {action}")

    # Trigger approval agent to notify employee
    response = approval_agent.run(f"Process holiday request {request_id} with action {action}",
    markdown=True)
    print(response.content)
    logger.info(f"Approval agent response: {response.content}")
    return f"Request {action} successfully. The employee will be notified."
except Exception as e:
    logger.error(f"Error processing request {request_id}: {str(e)}")
    return f"Error processing request: {str(e)}", 500

```

Ces deux routes Flask forment le cœur de la boucle d'approbation humaine (human-in-the-loop). La première (/request/) affiche à la personne RH les détails d'une demande de congé précise en allant les chercher dans Airtable, puis rend la page HTML définie plus haut. La seconde (/process/) reçoit la soumission du formulaire (bouton "Approve" ou "Disapprove"), met à jour le statut de la demande directement dans Airtable, puis - point important - délègue la suite du travail (notifier l'employé) à `approval_agent.run(...)`, un agent IA plutôt qu'un simple envoi d'email codé en dur : c'est l'agent qui décide, selon ses instructions, comment formuler et envoyer la notification. Cela illustre comment un clic humain sur une page web peut réveiller un agent IA pour terminer une tâche métier.

Etape 4 — API de chat et intégration vocale, démarrage du serveur Flask

[Lignes 148-251](#)

```

@app.route('/chat', methods=['POST'])
def chat_handler():
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "No JSON data provided"}), 400

        message = data.get('message')

        if not message:
            return jsonify({"error": "Missing message"}), 400

        # Call the team agent
        response = ChatBot_Team.run(message, markdown=True)

        return jsonify({"response": response.content})
    except Exception as e:
        logger.error(f"Error in chat handler: {str(e)}")
        return jsonify({"error": str(e)}), 500

@app.route('/text', methods=['POST'])
def voice_handler():
    try:
        data = request.get_json()
        if not data:
            return jsonify({"error": "No JSON data provided"}), 400

        print(data)
        print(data.get("transcript"))

        transcript = data.get("transcript")
        print(transcript)

        if not transcript:
            return jsonify({"error": "Missing transcript"}), 400

```

```

# Optional: customer_name = data.get("customer_name", "User")

# Step 1: Create Chat Session
try:
    create_chat_response = requests.post(
        "https://api.retellai.com/create-chat",
        headers={
            "Authorization": f"Bearer sk-***REDACTED***",
            "Content-Type": "application/json"
        },
        json={
            "agent_id": "agent_4cb8cf66474b91a09d19419",
            "metadata": {}
        },
        timeout=30
    )
except requests.RequestException as e:
    logger.error(f"Error creating chat: {e}")
    return jsonify({"error": "Failed to connect to Retell API"}), 500

print(create_chat_response) # If response 402, means payment required!!!
if create_chat_response.status_code != 200:
    return jsonify({"error": f"Failed to create chat: {create_chat_response.text}"}), 500

chat_id = create_chat_response.json().get("chat_id")
if not chat_id:
    return jsonify({"error": "No chat_id returned"}), 500

# RETELL_API_KEY corrigé
RETELL_API_KEY = "sk-***REDACTED***"
try:
    completion_response = requests.post(
        "https://api.retellai.com/create-chat-completion",
        headers={
            "Authorization": f"Bearer {RETELL_API_KEY}",
            "Content-Type": "application/json"
        },
        json={
            "chat_id": chat_id,
            "content": transcript
        },
        timeout=30
    )
except requests.RequestException as e:
    logger.error(f"Error getting completion: {e}")
    return jsonify({"error": "Failed to get completion from Retell API"}), 500

if completion_response.status_code != 200:
    return jsonify({"error": f"Failed to get completion: {completion_response.text}"}), 500

messages = completion_response.json().get("messages", [])
agent_reply = messages[-1]["content"] if messages else "No reply from agent."

return jsonify({
    "response": agent_reply,
    "chat_id": chat_id
})

except Exception as e:
    logger.error(f"Error in voice handler: {str(e)}")
    return jsonify({"error": str(e)}), 500

def start_flask():
    logger.info("Starting Flask app on port 5000")
    try:
        app.run(port=5000, debug=False)
    except Exception as e:
        logger.error(f"Error starting Flask app: {e}")

```

La route /chat est le point d'entrée principal utilisé par le frontend : elle reçoit un message texte et le transmet tel quel à ChatBot_Team (l'équipe d'agents définie plus loin), qui se charge de router la question vers le bon spécialiste. La route /text illustre une intégration vocale : elle reçoit une transcription (probablement produite ailleurs par un service de reconnaissance vocale), puis appelle directement l'API tierce Retell (deux requêtes HTTP successives : création d'une session de chat, puis envoi du message) pour obtenir une réponse, sans passer par les agents Agno de ce fichier - un chemin parallèle et indépendant du reste du système. Notez que la clé d'API Retell utilisée ici est codée en dur dans le fichier source (remplacée ci-dessus par un espace réservé pour ne pas la divulguer) : c'est une mauvaise pratique de sécurité, une clé API ne devrait jamais être écrite en clair dans le code mais toujours lue depuis une variable d'environnement comme le sont les autres clés du projet. Enfin, `start_flask()` encapsule le lancement du serveur HTTP, destiné à tourner dans un thread séparé pour ne pas bloquer le reste du script.

Etape 5 — Fonctions outils Airtable utilisées par les agents

Lignes 254-365

```
def get_employee_holiday(employee_id: int) -> dict:
    """Retrieve employee holiday information from Airtable by employee_id."""
    try:
        records = holiday_table.all()
        for record in records:
            fields = record["fields"]
            if fields.get("employee_id") == employee_id:
                logger.info(f"Retrieved holiday data for employee_id: {employee_id}")
                return {
                    "name": fields.get("full_name"),
                    "email": fields.get("email"),
                    "total": int(fields.get("total_holiday_days", 0)),
                    "taken": int(fields.get("holidays_taken", 0)),
                    "last_date": fields.get("last_holiday_taken"),
                }
            logger.warning(f"No holiday data found for employee_id: {employee_id}")
        return None
    except Exception as e:
        logger.error(f"Error retrieving employee holiday data: {e}")
        return None

def get_employee_project(employee_id: int):
    """Retrieve employee project information from Airtable by employee_id."""
    try:
        records = project_table.all()
        for record in records:
            fields = record["fields"]
            if fields.get("employee_id") == employee_id:
                logger.info(f"Retrieved project data for employee_id: {employee_id}")
                return {
                    "name": fields.get("employee_name"),
                    "project": fields.get("project_name"),
                    "role": fields.get("role"),
                    "start_date": fields.get("start_date"),
                    "end_date": fields.get("end_date"),
                    "next_project": {
                        "project_name": fields.get("next_project_name", ""),
                        "start_date": fields.get("next_project_start_date", ""),
                        "role": fields.get("next_project_role", "")
                    }
                }
            logger.warning(f"No project data found for employee_id: {employee_id}")
        return None
    except Exception as e:
        logger.error(f"Error retrieving employee project data: {e}")
        return None

def update_next_project(employee_id: int, project_name: str, start_date: str, role: str):
    """Update the next project information for an employee in Airtable."""
    try:
```

```

records = project_table.all()
for record in records:
    fields = record["fields"]
    if fields.get("employee_id") == employee_id:
        project_table.update(record["id"], {
            "next_project_name": project_name,
            "next_project_start_date": start_date,
            "next_project_role": role
        })
        logger.info(f"Updated next project for employee_id: {employee_id}")
        return True
    logger.warning(f"No project record found for employee_id: {employee_id}")
    return False
except Exception as e:
    logger.error(f"Error updating next project: {e}")
    return False

def create_holiday_request(employee_id: int, requested_days: int):
    """Create a new holiday request in Airtable."""
    try:
        employee = get_employee_holiday(employee_id)
        if not employee:
            logger.error(f"No employee found for employee_id: {employee_id}")
            return None

        request_id = str(uuid.uuid4())
        remaining_days = employee['total'] - employee['taken']

        request_table.create({
            "request_id": request_id,
            "employee_id": employee_id,
            "full_name": employee['name'],
            "requested_days": requested_days,
            "remaining_days": remaining_days,
            "status": "pending",
            "request_date": time.strftime("%Y-%m-%d")
        })
        logger.info(f"Created holiday request {request_id} for employee_id: {employee_id}")
        return request_id
    except Exception as e:
        logger.error(f"Failed to create holiday request for employee_id {employee_id}: {str(e)}")
        return None

def update_holiday_taken(employee_id: int, additional_days: int):
    """Update holidays_taken for an employee in Airtable."""
    try:
        records = holiday_table.all()
        for record in records:
            fields = record["fields"]
            if fields.get("employee_id") == employee_id:
                current_taken = int(fields.get("holidays_taken", 0))
                holiday_table.update(record["id"], {
                    "holidays_taken": current_taken + additional_days,
                    "last_holiday_taken": time.strftime("%Y-%m-%d")
                })
                logger.info(f"Updated holidays_taken for employee_id: {employee_id}")
                return True
        logger.warning(f"No holiday record found for employee_id: {employee_id}")
        return False
    except Exception as e:
        logger.error(f"Error updating holiday taken: {e}")
        return False

```

Ces cinq fonctions Python ordinaires sont les "outils" (tool use / function calling) que les agents IA définis plus loin pourront appeler eux-mêmes pendant leur raisonnement, exactement comme des fonctions métier classiques : lire le solde de congés ou le projet d'un employé, créer une demande de congé, mettre à jour le prochain projet, ou incrémenter les jours pris après une approbation. Chacune encapsule un accès direct à Airtable (parcours des

enregistrements, filtrage par employee_id, lecture ou écriture de champs) avec gestion d'erreur et journalisation systématiques. C'est le mécanisme qui transforme un LLM, capable seulement de générer du texte, en un agent capable d'agir réellement sur une base de données d'entreprise : le modèle décide quand appeler get_employee_holiday ou create_holiday_request selon la question posée, et le code ci-dessus exécute l'action demandée de façon déterministe et fiable.

Etape 6 — Configuration Gmail, tunnel ngrok et démarrage du serveur en arrière-plan

Lignes 368-392

```
# Configuration des credentials Gmail
gmail_credentials_path = os.path.join(os.path.dirname(__file__), "client_secret.json")
if not os.path.exists(gmail_credentials_path):
    logger.warning(f"Gmail credentials file not found at {gmail_credentials_path}")
    gmail_credentials_path = None

# Start ngrok
try:
    ngrok.set_auth_token(NGROK_AUTH_TOKEN)
    try:
        public_url = ngrok.connect(5000, bind_tls=True).public_url
        logger.info(f"ngrok tunnel \"{public_url}\" -> \"http://127.0.0.1:5000\"")
    except Exception as ngrok_error:
        logger.warning(f"ngrok HTTPS connection failed: {ngrok_error}")
        try:
            public_url = ngrok.connect(5000).public_url
            logger.info(f"ngrok tunnel (HTTP) \"{public_url}\" -> \"http://127.0.0.1:5000\"")
        except Exception:
            logger.warning("ngrok completely failed, using localhost")
            public_url = "http://localhost:5000"
except Exception as e:
    logger.error(f"Error setting up ngrok: {e}")
    public_url = "http://localhost:5000"

threading.Thread(target=start_flask, daemon=True).start()
time.sleep(2)
```

Cette étape prépare deux capacités externes. D'abord elle vérifie la présence d'un fichier d'identifiants Gmail (client_secret.json) qui conditionnera l'activation ou non de l'outil d'envoi d'email des agents. Ensuite elle ouvre un tunnel ngrok : ngrok crée une URL publique sur Internet qui redirige vers le serveur Flask tournant en local sur le port 5000, ce qui permet à un email RH cliqué depuis n'importe où (ou à un service externe comme Retell) d'atteindre ce serveur sans avoir à le déployer sur un vrai hébergeur. Le code tente d'abord une connexion HTTPS, puis HTTP en repli, puis retombe sur localhost en dernier recours si ngrok échoue complètement, un empilement de solutions de repli typique pour un service qui ne doit pas planter juste parce qu'une dépendance externe est indisponible. Enfin, le serveur Flask est lancé dans un thread séparé en arrière-plan (daemon=True), avec une courte pause pour lui laisser le temps de démarrer avant que la suite du script (les agents) ne s'initialise.

Etape 7 — Définition des agents spécialisés (congrés, approbation, projets)

Lignes 394-529

```
# Holiday Agent
try:
    email_tools = EmailTools() if gmail_credentials_path else None
    holiday_agent = Agent(
        name="Holiday Days Calculator Agent",
        model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
        markdown=True,
        show_tool_calls=True,
        tools=[
            FileTools(),
            CalculatorTools(),
            email_tools,
            get_employee_holiday,
            create_holiday_request
        ] if email_tools else [
```

```

        FileTools(),
        CalculatorTools(),
        get_employee_holiday,
        create_holiday_request
    ],
    description="""
An internal company assistant that helps employees check their holiday status, submit holiday requests,
and create new employee records.
This agent manages holiday data from an Airtable using employee IDs, creates holiday request entries for
HR approval, and can register new employees in the system.
It sends emails to HR with links to approve or disapprove requests.
""",
    instructions=f"""
- Your job is to retrieve holiday information or process holiday requests using the employee's unique ID.
- Use 'get_employee_holiday' to fetch:
    - Full name
    - Email
    - Total allocated holiday days
    - Number of holidays taken
    - Last holiday date
- For balance queries:
    - Calculate: remaining_days = total_holiday_days - holidays_taken
    - Return the information clearly and professionally.
- For holiday requests:
    - Extract the number of requested days and employee ID from the query.
    - Use 'create_holiday_request' to store the request in Airtable.
    - Generate a unique request ID and include it in an email to HR ({HR_EMAIL}) with a link to the
approval page: {public_url}/request/<request_id>
    - The email should include the employee's name, requested days, and remaining balance.
- Do not send holiday balance or request confirmation directly to the employee unless explicitly
requested.
- Only send data from Airtable. Do not guess or hallucinate employee info.
- If no matching employee_id is found, respond with: "No holiday data found for the provided employee ID."
"""
    )
    logger.info("Holiday Agent initialized successfully")
except Exception as e:
    logger.error(f"Error initializing Holiday Agent: {e}")
    raise

# Approval Agent
try:
    approval_agent = Agent(
        name="Holiday Approval Agent",
        model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
        markdown=True,
        show_tool_calls=True,
        tools=[
            email_tools,
            get_employee_holiday,
            update_holiday_taken
        ] if email_tools else [
            get_employee_holiday,
            update_holiday_taken
        ],
        description="""
An internal agent that processes HR holiday request approvals or disapprovals.
It updates the employee's holiday data and notifies the employee of the decision via email.
""",
        instructions="""
[...]
```

```

# Project Agent
try:
    project_agent = Agent(
        name="Project Management Agent",
        model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
        markdown=True,
        tools=[
            email_tools,
            FileTools(),
            get_employee_project,
            update_next_project
        ] if email_tools else [
            FileTools(),
            get_employee_project,
            update_next_project
        ],
        description="""
An internal company assistant that helps employees view and manage their current and future project
assignments.
This agent reads employee project data from an Airtable table using the employee's unique ID.
It can also update planned future projects when new information is provided.
""",
        instructions="""

[...]
```

Cette étape définit les trois agents spécialistes du système, chacun encapsulé dans son propre bloc try/except pour que l'échec d'un agent soit clairement identifié plutôt que de faire remonter une erreur ambiguë. Chaque agent reçoit le même modèle (GPT-4o-mini via OpenAI), mais un jeu d'outils et des instructions différents et taillés sur mesure pour son domaine : `holiday_agent` combine calcul, accès fichier, email et les fonctions Airtable de congés pour répondre aux soldes et créer des demandes ; `approval_agent` se concentre sur le traitement des décisions d'approbation et la notification de l'employé ; `project_agent` gère les données de projet. La liste d'outils de chaque agent est construite conditionnellement selon que `email_tools` est disponible ou non (dépendant des identifiants Gmail détectés à l'étape précédente), ce qui évite de planter si l'envoi d'email n'est pas configuré. Les instructions de chaque agent, écrites en langage naturel, jouent le rôle de spécification métier : elles indiquent précisément quelle fonction utiliser pour quelle situation, et rappellent explicitement de ne jamais halluciner de données employé.

Etape 8 — Équipe d'agents (routage) et boucle principale

[Lignes 533-609](#)

```

try:
    sqlite_storage = SqliteStorage(
        table_name="agent_sessions",
        db_file=db_file
    )
    logger.info("SQLite storage initialized successfully")

    ChatBot_Team = Team(
        name="HR Assistant Team Manager",
        members=[holiday_agent, project_agent, approval_agent],
        storage=sqlite_storage,
        add_history_to_messages=True,
        model=OpenAIChat(id="gpt-4o-mini", api_key=api_key_openai),
        description="""
An intelligent HR Assistant Team Manager responsible for handling employee queries related to holidays,
project assignments, and holiday request approvals.
This team manages delegated tasks to the most appropriate specialist agent:
- The holiday agent manages vacation day tracking and holiday request submissions.
- The project agent oversees project assignments, updates, and status inquiries.
- The approval agent processes HR approvals or disapprovals and notifies employees.
""")
```

```

The manager ensures that every incoming query is handled by the appropriate agent based on the user's
intent.
"""
    instructions="""
You are an intelligent routing agent that serves as the HR Assistant Team Manager.
Your role is to:
- Receive user questions related to employee holidays, project assignments, or holiday request approvals.
- Analyze the intent of the query.
- Forward the query to the correct agent:
    - If the query relates to holiday, time off, vacation leave balance, or holiday requests, assign it to
the **Holiday Days Calculator Agent**.
    - If the query relates to project names, roles, start/end dates, or updates, assign it to the
**Project Management Agent**.
    - If the query relates to holiday request approval or disapproval (e.g., from a web hook or internal
trigger), assign it to the **Holiday Approval Agent**.
- Never try to answer the user directly - your job is to decide which agent is most suited to respond.
- If a query could match multiple domains, use judgment based on parsing and specificity.
- If no clear match exists, return a polite message suggesting the user clarify their request.
Always strive for clarity and efficiency, ensuring accurate responses from the appropriate domain expert.
"""

    )
    logger.info("ChatBot Team Manager initialized successfully")
except Exception as e:
    logger.error(f"Error initializing ChatBot Team Manager: {e}")
    raise

# Main interaction loop (optional - can be uncommented for CLI testing)
def main_cli():
    """Optional CLI interface for testing"""
    print("HR Assistant System started. Type 'quit' to exit.")
    while True:
        try:
            user_input = input("You: ")
            if user_input.lower() in ['exit', 'quit']:
                print("Bot: Goodbye!")
                break
            response = ChatBot_Team.run(user_input, markdown=True)
            print("Bot:", response.content)
        except KeyboardInterrupt:
            print("\nBot: Goodbye!")
            break
        except Exception as e:
            logger.error(f"Error in main loop: {e}")
            print(f"Error: {e}")

if __name__ == "__main__":
    try:
        logger.info("HR System fully initialized and ready")
        print(f"System running at: {public_url}")
        print("Press Ctrl+C to stop")

        # Uncomment the next line if you want CLI interface
        # main_cli()

        # Keep the main thread alive
        while True:
            time.sleep(1)
    except KeyboardInterrupt:
        logger.info("Shutting down...")
        print("Shutting down...")
    except Exception as e:
        logger.error(f"Critical error: {e}")
        raise

```

La dernière pièce du puzzle est ChatBot_Team, un objet Team d'Agno qui regroupe les trois agents spécialistes sous un "manager" commun. Ses instructions ne lui demandent jamais de répondre lui-même à l'utilisateur : son unique rôle est d'analyser l'intention de la question et de la déléguer à l'agent le plus pertinent - c'est le pattern

d'architecture multi-agent illustré par tout ce fichier, où plusieurs experts étroits valent mieux qu'un seul généraliste. Le paramètre `storage (SqliteStorage)` et `add_history_to_messages=True` donnent à cette équipe une mémoire persistante entre les appels : les échanges sont sauvegardés dans un fichier SQLite local, ce qui permet à une conversation de garder son contexte plutôt que de repartir de zéro à chaque message. Le bloc final `if __name__ == "__main__"` garde simplement le processus principal vivant indéfiniment (le vrai travail se passe dans le thread Flask lancé plus haut), avec un arrêt propre sur Ctrl+C ; une fonction `main_cli()` optionnelle est laissée disponible mais désactivée par défaut, pour tester le système en ligne de commande sans passer par l'API HTTP.

35-HR-Assistant/app.py

Interface web Streamlit qui sert de client de chat pour interroger l'API Flask du backend HR Assistant.

Etape 1 — Configuration de la page Streamlit

Lignes 1-17

```
import streamlit as st
import requests
import json
from datetime import datetime

# Configuration de la page
st.set_page_config(
    page_title="HR Assistant",
    page_icon="",
    layout="centered"
)

# URL de votre API Flask
API_BASE_URL = "http://127.0.0.1:5000"

# Titre principal
st.title("HR Assistant System")
```

Ce fichier est un frontend séparé, écrit avec Streamlit (une bibliothèque qui transforme un script Python en interface web interactive sans écrire de HTML/JS). Il commence par importer Streamlit et requests, puis configure la page (titre d'onglet, icône, mise en page centrée). L'URL de base de l'API Flask du fichier `rhAgent.py` est codée en constante (`http://127.0.0.1:5000`) : ce frontend ne contient aucune logique métier ni aucun agent, il n'est qu'un client HTTP habillé d'une interface graphique, ce qui illustre une séparation nette entre le backend (agents + données) et la présentation.

Etape 2 — Saisie de l'identifiant employé et fonction d'appel à l'API

Lignes 20-41

```
col1, col2 = st.columns([1, 2])
with col1:
    employee_id = st.number_input("Employee ID:", min_value=1, value=1)
with col2:
    st.write("") # Espace vide

# Fonction pour appeler l'API
def call_hr_api(message):
    try:
        response = requests.post(
            f"{API_BASE_URL}/chat",
            json={"message": message},
            timeout=10
        )
        if response.status_code == 200:
            return response.json().get("response", "Pas de réponse")
        else:
            return f"Erreur API: {response.status_code}"
    except requests.exceptions.ConnectionError:
        return "Erreur: Serveur HR non accessible"
```

```
except Exception as e:
    return f"Erreur: {str(e)}"
```

Un champ numérique permet à l'utilisateur de saisir son identifiant employé, réutilisé dans tous les messages envoyés au backend puisque les agents identifient toujours un employé par son `employee_id` (jamais par son nom, comme le précisent les instructions des agents dans `rhAgent.py`). La fonction `call_hr_api()` centralise tous les appels réseau vers la route `/chat` du backend Flask : elle envoie le message en JSON, gère le cas normal (statut 200), le cas d'une erreur HTTP renvoyée par le serveur, et surtout le cas où le serveur Flask n'est simplement pas démarré (`ConnectionError`), avec un message d'erreur clair plutôt qu'une exception qui ferait planter l'interface.

Etape 3 — Zone de chat et historique de conversation

Lignes 44-58

```
# Zone de chat
st.subheader("Chat")

# Initialiser l'historique
if "messages" not in st.session_state:
    st.session_state.messages = []

# Afficher les messages
for message in st.session_state.messages:
    if message["role"] == "user":
        st.write(f"***Vous:** {message['content']}")
    else:
        st.write(f"***Assistant:** {message['content']}")

# Zone de saisie
user_input = st.text_input("Votre message:", placeholder="Ex: Combien de jours de congés me reste-t-il ?")
```

`st.session_state` est le mécanisme Streamlit qui conserve un état entre deux interactions utilisateur (par défaut, chaque clic recharge tout le script Python de haut en bas, donc sans cette persistance explicite l'historique de chat serait perdu à chaque message). La liste des messages est initialisée une seule fois puis réaffichée à chaque exécution, en distinguant visuellement les messages de l'utilisateur et ceux de l'assistant. Un champ texte libre permet ensuite de saisir n'importe quelle question, en plus des boutons rapides définis à l'étape suivante.

Etape 4 — Boutons d'action (envoi libre et raccourcis)

Lignes 61-98

```
col1, col2, col3 = st.columns(3)

with col1:
    if st.button("Envoyer") and user_input:
        # Ajouter Employee ID au message
        full_message = f"Employee ID: {employee_id}. {user_input}"

        # Ajouter à l'historique
        st.session_state.messages.append({"role": "user", "content": user_input})

        # Appeler l'API
        with st.spinner("En cours..."):
            response = call_hr_api(full_message)

        st.session_state.messages.append({"role": "assistant", "content": response})
        st.rerun()

with col2:
    if st.button("Mes congés"):
        message = f"Employee ID: {employee_id}. Quel est mon solde de congés ?"
        st.session_state.messages.append({"role": "user", "content": "Mes congés"})

        with st.spinner("En cours..."):
            response = call_hr_api(message)
```

```

        st.session_state.messages.append({"role": "assistant", "content": response})
        st.rerun()

with col3:
    if st.button("Mon projet"):
        message = f"Employee ID: {employee_id}. Quel est mon projet actuel ?"
        st.session_state.messages.append({"role": "user", "content": "Mon projet"})

        with st.spinner("En cours..."):
            response = call_hr_api(message)

        st.session_state.messages.append({"role": "assistant", "content": response})
        st.rerun()

```

Trois boutons côte à côte couvrent les scénarios les plus fréquents : envoyer un message libre, ou déclencher directement une question pré-écrite ("Mes congés" / "Mon projet") sans avoir à la taper. Dans les trois cas, le même schéma se répète : l'identifiant employé est préfixé au message envoyé à l'API (pour que le backend sache toujours de qui il s'agit), le message est ajouté à l'historique local, un spinner s'affiche pendant l'appel réseau, la réponse est ajoutée à son tour à l'historique, puis `st.rerun()` force Streamlit à réexécuter le script pour rafraîchir immédiatement l'affichage avec la conversation à jour.

Etape 5 — Réinitialisation du chat et test de connexion

[Lignes 101-112](#)

```

# Bouton pour vider l'historique
if st.button("Vider le chat"):
    st.session_state.messages = []
    st.rerun()

# Test de connexion
st.subheader("Test de connexion")
if st.button("Tester l'API"):
    try:
        response = requests.get(f"{API_BASE_URL}/", timeout=5)
        st.success("Connexion OK")
    except:
        st.error("Connexion échouée - Vérifiez que le serveur Flask est démarré")

```

Deux fonctionnalités utilitaires terminent l'interface : un bouton pour vider entièrement l'historique de conversation (utile pour repartir d'une session propre), et un bouton de diagnostic qui envoie une simple requête GET vers le serveur Flask pour vérifier qu'il répond, sans passer par un agent. C'est un outil de dépannage pensé pour l'utilisateur final : avant de se demander pourquoi le chat ne répond pas, il peut d'abord vérifier en un clic si le backend est seulement injoignable.

Projet 36 — 36-rfp-Agent

Remplit un questionnaire d'appel d'offres via Google Sheets

LE BUT DE CE PROJET

Quand une entreprise répond à un appel d'offres (RFP, Request for Proposal), elle reçoit souvent une longue feuille de calcul de questions précises sur son produit ou service - sécurité, intégrations, support, conformité - à laquelle il faut répondre une par une, en allant chercher l'information exacte dans la documentation interne de l'entreprise. C'est un travail répétitif et chronophage, et le risque de réponses incohérentes ou approximatives augmente avec le nombre de questions et le nombre de personnes qui y répondent au fil du temps. Le but de ce projet est de montrer comment un agent IA peut prendre en charge ce travail de bout en bout : il se connecte directement à un vrai Google Sheet, identifie les champs déjà remplis et ceux qui manquent, cherche la réponse dans une base de connaissances documentaire de l'entreprise plutôt que d'improviser, puis réécrit le tableau complété directement dans le même Google Sheet. La technique clé démontrée ici est le tool use : donner à un modèle de langage des fonctions concrètes (lister les onglets, lire une feuille, écrire une feuille) qu'il peut décider d'appeler lui-même pendant son raisonnement pour interagir avec un système externe, ce qui transforme un simple assistant conversationnel en un agent capable d'accomplir une tâche complète dans un outil métier réel, du début à la fin, sans intervention manuelle.

36-rfp-Agent/main.py

Agent Agno qui lit et complète un Google Sheet de réponses RFP en s'appuyant sur une base de connaissances documentaire de l'entreprise (RAG).

Etape 1 — Imports, variables d'environnement et autorisation Google Sheets

Lignes 1-25

```
import os
import pygsheets
import pandas as pd
from agno.tools import tool
from textwrap import dedent
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.vectordb.pgvector import PgVector
from agno.tools.reasoning import ReasoningTools
from agno.knowledge.markdown import MarkdownKnowledgeBase
from agno.vectordb.qdrant import Qdrant
from dotenv import load_dotenv
import json
from io import StringIO

# Charger les variables d'environnement
load_dotenv()

# Récupération des variables d'environnement
qdrant_api_key = os.getenv("QDRANT_API_KEY")
qdrant_url = os.getenv("QDRANT_URL")
openai_api_key = os.getenv("OPENAI_API_KEY")

# Autorisation Google Sheets
```

```
gc = pygsheets.authorize(service_account_file='36-rfp-Agent/client_secret.json')
```

Le script importe pygsheets (client Google Sheets), pandas (manipulation de tableaux de données), le décorateur @tool d'Agno (qui permet de transformer une fonction Python ordinaire en outil utilisable par un agent), et les briques du RAG (MarkdownKnowledgeBase, Qdrant). L'import de PgVector est présent mais n'est jamais utilisé plus loin dans le fichier - un vestige d'une version antérieure du script où PostgreSQL était peut-être envisagé comme base vectorielle avant d'opter pour Qdrant. Les clés API sont lues depuis l'environnement, puis la ligne gc = pygsheets.authorize(...) établit l'authentification auprès de l'API Google Sheets via un compte de service (fichier client_secret.json) : c'est cette connexion authentifiée, stockée dans la variable globale gc, qui sera réutilisée par les trois outils définis plus loin pour lire et écrire dans de vraies feuilles de calcul.

Etape 2 — Construction de la base de connaissances documentaire (RAG)

Lignes 27-38

```
### Setup Knowledge Base ###
vector_db = Qdrant(
    collection="docs_markdown",
    api_key=qdrant_api_key,
    url=qdrant_url,
)

knowledge_base = MarkdownKnowledgeBase(
    path="36-rfp-Agent/docs/markdown/",
    vector_db=vector_db,
    num_documents=5,
)
```

Contrairement au projet 34 qui utilisait LanceDB (une base vectorielle locale, embarquée dans un dossier), ici la base vectorielle est Qdrant, un service hébergé accessible par URL et clé API : ce choix permet à la base de connaissances d'être partagée entre plusieurs exécutions ou plusieurs machines, plutôt que d'être liée à un seul poste. MarkdownKnowledgeBase pointe ensuite vers un dossier local de fichiers Markdown contenant la documentation interne de l'entreprise (produit, sécurité, intégrations, etc.), avec num_documents=5 qui limite le nombre de passages les plus pertinents renvoyés à chaque recherche. C'est cette base qui permettra à l'agent RFP de répondre aux questions du client avec des faits réels issus de la documentation de l'entreprise, plutôt que d'inventer des réponses.

Etape 3 — Outil pour lister les onglets du Google Sheet

Lignes 40-61

```
@tool(
    name="list_worksheets",
    description="List all available worksheets in the Google Sheet.",
    show_result=True,
)
def list_worksheets(sheet_id: str) -> str:
    """
    List all available worksheets in a Google Sheet.

    Args:
        sheet_id (str): The unique ID of the Google Sheet.

    Returns:
        str: List of available worksheet titles.
    """
    try:
        sh = gc.open_by_key(sheet_id)
        worksheets = sh.worksheets()
        titles = [ws.title for ws in worksheets]
        return f"Available worksheets: {' '.join(titles)}"
    except Exception as e:
        return f"Error listing worksheets: {str(e)}"
```

Le décorateur `@tool` transforme cette fonction Python ordinaire en un outil que l'agent peut appeler de lui-même pendant sa génération : le nom, la description et la docstring (avec ses Args/Returns) ne sont pas de la simple documentation pour un humain, ils sont envoyés au modèle de langage pour qu'il comprenne quand et comment appeler cette fonction. Concrètement, elle ouvre le Google Sheet ciblé par son identifiant et renvoie la liste des noms d'onglets qu'il contient. C'est un outil de découverte, prévu pour être appelé en premier (comme le rappellent les instructions de l'agent plus bas) afin d'éviter que l'agent ne tente de lire un onglet dont il devine le nom, ce qui provoquerait une erreur.

Etape 4 — Outil pour lire une feuille comme un DataFrame

Lignes 63-94

```
@tool(
    name="read_sheet_as_df",
    description="Read an entire Google Sheets worksheet as a pandas DataFrame.",
    show_result=True,
)
def read_sheet_as_df(sheet_id: str, worksheet_title: str) -> str:
    """
    Use this function to read an entire Google Sheets worksheet and return its data as a JSON-formatted
    DataFrame.

    Args:
        sheet_id (str): The unique ID of the Google Sheet (from the URL).
        worksheet_title (str): The tab name of the worksheet within the Google Sheet.

    Returns:
        str: A JSON string representing the DataFrame containing the worksheet's data.
    """
    try:
        sh = gc.open_by_key(sheet_id)

        # Vérifier si l'onglet existe
        try:
            wks = sh.worksheet_by_title(worksheet_title)
        except:
            # Lister les onglets disponibles si celui demandé n'existe pas
            worksheets = sh.worksheets()
            available = [ws.title for ws in worksheets]
            return f"Worksheet '{worksheet_title}' not found. Available worksheets: {'',
''.join(available)}"

        df = wks.get_as_df()
        return df.to_json(orient="records")
    except Exception as e:
        return f"Error reading sheet: {str(e)}"
```

Ce second outil charge le contenu réel d'un onglet précis sous forme de DataFrame pandas (une structure tabulaire en mémoire), puis le sérialise en JSON pour pouvoir le faire transiter dans la conversation avec le modèle de langage (qui ne manipule que du texte, jamais d'objets Python directement). C'est cette lecture qui permet à l'agent d'inspecter concrètement quelles lignes de la feuille RFP sont déjà répondues et lesquelles sont vides. Si l'onglet demandé n'existe pas, la fonction ne renvoie pas juste une erreur brute : elle liste les onglets réellement disponibles dans le même message, ce qui donne à l'agent l'information nécessaire pour se corriger tout seul à l'appel suivant, sans intervention humaine.

Etape 5 — Outil pour écrire un DataFrame dans une feuille

Lignes 96-143

```
@tool(
    name="write_df_to_sheet",
    description="Write a DataFrame to a Google Sheets worksheet using the sheet ID.",
    show_result=True,
)
def write_df_to_sheet(
    df_json: str, sheet_id: str, worksheet_title: str, start_cell: list[int] = [1, 1]
```

```

) -> str:
    """
    Use this function to write a DataFrame (as JSON) to a Google Sheets worksheet.

    Args:
        df_json (str): JSON string representing the DataFrame to write.
        sheet_id (str): The unique ID of the Google Sheet (from the URL).
        worksheet_title (str): The tab name of the worksheet.
        start_cell (list[int], optional): [row, column] location to start writing. Defaults to [1, 1].

    Returns:
        str: Confirmation message upon successful write.
    """
    try:
        # Gestion améliorée du JSON
        if isinstance(df_json, str):
            # Nettoyer le JSON si nécessaire
            df_json = df_json.strip()
            if df_json.startswith("```json"):
                df_json = df_json.replace("```json", "").replace("```", "").strip()

            # Parser le JSON avec StringIO pour éviter le warning
            df = pd.read_json(StringIO(df_json))
        else:
            # Si ce n'est pas une string, essayer de convertir
            df = pd.DataFrame(df_json)

        sh = gc.open_by_key(sheet_id)
        try:
            wks = sh.worksheet_by_title(worksheet_title)
        except pygsheets.WorksheetNotFound:
            wks = sh.add_worksheet(worksheet_title)

        wks.clear()
        wks.set_dataframe(df, start=tuple(start_cell))
        return f"Successfully wrote DataFrame to worksheet '{worksheet_title}' in sheet '{sheet_id}'."
    Rows: {len(df)}, Columns: {len(df.columns)}

    except json.JSONDecodeError as e:
        return f"JSON parsing error: {str(e)}. Please provide valid JSON format."
    except Exception as e:
        return f"Error writing to sheet: {str(e)}"

```

Ce troisième outil ferme la boucle : il prend le JSON produit par l'agent après qu'il a complété les réponses manquantes, le reconvertit en DataFrame pandas, puis réécrit entièrement l'onglet correspondant dans le Google Sheet (en le créant s'il n'existe pas encore). Le nettoyage du JSON en entrée (suppression d'éventuels marqueurs ```json ajoutés par le modèle) est une précaution pragmatique : un LLM a tendance à entourer son JSON de blocs de code Markdown même quand on lui demande du JSON brut, donc le code défensif retire ce bruit avant de parser. C'est cet outil qui rend le résultat du travail de l'agent persistant et visible pour l'utilisateur final : sans lui, l'agent pourrait "savoir" les bonnes réponses mais ne jamais les faire apparaître dans la vraie feuille de calcul.

Etape 6 — Définition de l'agent RFP

[Lignes 145-220](#)

```

rfp_agent = Agent(
    name="RFP Agent",
    model=OpenAIChat(id="gpt-4o", api_key=openai_api_key),
    knowledge=knowledge_base,
    search_knowledge=True,
    show_tool_calls=True,
    tools=[ReasoningTools(add_instructions=True), list_worksheets, read_sheet_as_df, write_df_to_sheet],
    instructions=dedent(
        """
        You are an RFP (Request-for-Proposal) agent.

        Your role is to assist with completing RFP spreadsheets. You receive partially filled RFP sheets

```

and use the company knowledge base to fill in the missing fields accurately.

```
---

Tools You Have Access To:

1. list_worksheets(sheet_id: str)
  - Lists all available worksheets in a Google Sheet
  - Use this FIRST to check available worksheet names

2. read_sheet_as_df(sheet_id: str, worksheet_title: str)
  - Reads an entire Google Sheets worksheet and returns its contents as a pandas DataFrame
  (serialized as JSON).
  - Use this to:
    • Load the current contents of an RFP sheet
    • Inspect which fields are already filled and which are missing

3. write_df_to_sheet(df_json: str, sheet_id: str, worksheet_title: str, start_cell: List[int] =
[1, 1])
  - Writes a pandas DataFrame (provided as a JSON string) to a specific worksheet in a Google
Sheet, replacing all existing content.
  - IMPORTANT: df_json must be valid JSON format
  - Use this to:
    • Write the filled RFP sheet back to Google Sheets
    • Overwrite the worksheet with the completed version
[...]
Your Objective:

1. FIRST: Use list_worksheets to see available worksheet names
2. Read the data using read_sheet_as_df with the correct worksheet name
3. Analyze the user's instructions and the structure of the RFP sheet
4. Use the company knowledge base to fill in missing fields. If no relevant information is found
in the knowledge base, leave the field blank.
5. Fill in missing values using logic derived from the knowledge base or prompts
6. Write the updated data using write_df_to_sheet with proper JSON format
[...]
IMPORTANT: Always start by listing available worksheets to avoid WorksheetNotFound errors.

Respond like a knowledgeable and trustworthy analyst. Be clear, helpful, and direct.
---
"""
),
)
```

Cette étape assemble tout ce qui précède en un seul agent. Le paramètre `tools` lui donne accès aux trois fonctions définies plus haut ainsi qu'à ReasoningTools, un outil Agno qui pousse l'agent à expliciter ses étapes de raisonnement avant d'agir plutôt que de sauter directement à une conclusion. Les paramètres `knowledge` et `search_knowledge=True` branchent la base documentaire RAG construite plus tôt. Les instructions, très détaillées, décrivent explicitement à quoi sert chaque outil et dans quel ordre les utiliser (lister les onglets, puis lire, puis écrire), ce qui réduit le risque que l'agent se trompe d'ordre ou tente une opération invalide. Elles précisent aussi comment justifier chaque réponse remplie (formulations "Yes / No / Partial " avec explication courte), un format qui rend le travail de l'agent traçable et vérifiable par un humain plutôt qu'une simple case cochée sans justification.

Etape 7 — Chargement de la base de connaissances et exécution de l'agent

[Lignes 222-233](#)

```
rfp_agent.knowledge.load(recreate=True)

# Démarrage avec vérification des onglets disponibles
rfp_agent.print_response(
    """
    Answer all the questions in the RFP spreadsheet based on our company knowledge base. only the
    'Response / Answer' column should be updated.
    Sheet ID: '1yg8MQ7boc0lgURLQqZcX2fcY7AqjVlfQM8ZntwtbizA'

    IMPORTANT: First check what worksheets are available in this sheet, then use the correct worksheet
```

```
name.  
    "",  
    stream=True,  
)
```

La dernière étape déclenche concrètement le pipeline. `rfp_agent.knowledge.load(recreate=True)` force une réindexation complète de la base de connaissances (relecture des fichiers Markdown, recalcul des embeddings, réécriture de la collection Qdrant) avant chaque exécution : pratique en développement pour être sûr de travailler sur la documentation la plus à jour, mais coûteux en appels d'API si on le laisse tel quel en production à chaque lancement. `print_response()` lance ensuite l'agent avec une instruction utilisateur concrète, contenant l'identifiant du Google Sheet cible et la consigne de ne modifier que la colonne "Response / Answer" - c'est cette dernière ligne qui met en mouvement tout ce qui a été construit : l'agent va lister les onglets, lire le tableau, chercher chaque réponse manquante dans la base de connaissances, puis réécrire le Google Sheet avec les champs complétés. Le paramètre `stream=True` affiche la réponse au fur et à mesure qu'elle est générée plutôt que d'attendre la fin complète, ce qui donne une visibilité en direct sur le raisonnement de l'agent.